

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2024

JEUDI 18 AVRIL 2024

08h00 - 12h00

FILIERE MPI - Epreuve n° 7

INFORMATIQUE C (XULSR)

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

Égalités et différences

Le sujet comporte 17 pages, numérotées de 1 à 17 et une annexe.

Vue d'ensemble du sujet.

Ce sujet traite du problème de satisfiabilité de formules construites par la conjonction de contraintes d'égalité et de différence (dis-égalité) entre expressions. Ce problème se présente, par exemple, dans la compilation de programmes, quand on cherche à réduire des expressions complexes (nécessitant beaucoup de calculs) vers des expressions plus simples, en utilisant les propriétés des opérateurs apparaissant dans ces expressions. Par exemple, si x est une variable entière, $(x * 2) / 2 == x$ est une contrainte d'égalité entre deux expressions écrites en C. De même, si y et z sont des variables booléennes, l'égalité $(y || (y \&\& z)) == y$ permet de simplifier l'expression C écrite à gauche de l'égalité.

Il est important de savoir si un ensemble donné de contraintes d'égalité ou de différence est satisfiable. Dans la suite, P_L *est le problème suivant* : « **Étant donnée une formule F , conjonction de contraintes de la classe L , décider si F est satisfiable** ».

Ce sujet propose d'étudier la complexité du problème P_L et les algorithmes pour le résoudre dans le cas de quatre classes de contraintes :

La partie I considère la classe L_1 où les contraintes ont la forme $x \leftrightarrow y$ ou $x \leftrightarrow \neg z$ avec x , y et z des variables propositionnelles (c'est-à-dire, ayant des valeurs booléennes).

La partie II concerne la classe L_2 incluant les contraintes de la classe L_1 et celles de la forme $x \leftrightarrow (y \wedge z)$ (entre autres) avec x , y et z des variables propositionnelles. Les algorithmes des deux premières parties seront codés en OCaml.

La partie III s'intéresse à la classe L_3 des contraintes de la forme $x = y$ ou $x \neq y$ avec x et y des variables entières.

Enfin, la partie IV concerne les contraintes d'égalité et de différence entre les expressions construites à partir de variables entières et de fonctions à deux arguments. Les algorithmes des deux dernières parties seront codés en C.

Les parties III et IV peuvent être traitées indépendamment de parties I et II. Il est recommandé de traiter les parties I et II de manière séquentielle. De même pour les parties III et IV.

Notations

Les contraintes seront construites en utilisant un ensemble $\mathbb{X}$ de n variables notées X_i avec $1 \leq i \leq n$. Une valeur de vérité est un élément de $\mathbb{B} \stackrel{\text{def}}{=} \{0, 1\}$ où 1 représente le vrai et 0 le faux. Une valeur entière k est un élément de $\mathbb{Z}$ et sa valeur absolue est notée $|k|$. Étant donnés $k, m \in \mathbb{Z}$ avec $k \leq m$, l'intervalle $[k; m]$ est l'ensemble des $i \in \mathbb{Z}$ tels que $k \leq i \leq m$.

Logique propositionnelle. On utilisera les formules de la logique propositionnelle construites à partir de l'ensemble de variables propositionnelles $\mathbb{X}$, les constantes propositionnelles $\perp$ (faux) et $\top$ (vrai), les connecteurs logiques classiques $\neg$ (négation), $\wedge$ (conjonction), $\vee$ (disjonction). On définit $F \rightarrow G$ (implication) par $(\neg F \vee G)$, et $F \leftrightarrow G$ (équivalence) par $(F \rightarrow G) \wedge (G \rightarrow F)$. Pour les connecteurs $\wedge$ et $\vee$, on utilise leur forme k -aire avec $k \geq 0$. Par exemple, $X_1 \wedge X_2 \wedge \dots \wedge X_k$ est une formule utilisant une conjonction k -aire. Par convention, une conjonction 0-aire est $\top$, et une disjonction 0-aire est $\perp$; une conjonction (resp. disjonction) unaire est identique à son opérande.

Un **littéral** ℓ est soit une variable X_i soit sa négation $\neg X_i$. Une formule est en **forme normale conjonctive** (CNF) si c'est une conjonction de disjonctions de littéraux. Une formule est en forme k -CNF avec $k > 0$, si elle est en forme CNF et le nombre de littéraux différents dans chaque disjonction est au plus k . Par exemple, la formule $(X_2 \vee X_3 \vee \neg X_4) \wedge (\neg X_3 \vee X_4) \wedge X_1$ est en forme 3-CNF.

On rappelle qu'une formule propositionnelle F peut être **satisfaite** ou non par une valuation $v : \mathbb{X} \rightarrow \mathbb{B}$, qui assigne une valeur de vérité à chaque variable. On note $v \models F$ la relation de satisfaction. Une formule F' est **conséquence logique** de F , noté $F \models F'$, quand, pour toute valuation v telle que $v \models F$, on a aussi $v \models F'$. Deux formules sont **logiquement équivalentes** lorsque chacune est conséquence logique de l'autre (ou, de façon équivalente, quand elles sont satisfaites par les mêmes valuations). Une formule est une **tautologie** si elle est satisfaite par toutes les valuations. Une formule est une **contradiction** (antilogie) ou contradictoire si elle n'est satisfaite par aucune valuation. Une formule F est **satisfiable** s'il existe une valuation v telle que $v \models F$.

L'annexe rappelle certaines règles de la déduction naturelle.

Complexité. Par la complexité d'un algorithme A on entend le nombre d'opérations élémentaires (comparaison, addition, soustraction, multiplication, division, affectation, test, etc.) nécessaires à l'exécution de A **dans le pire cas**. Lorsque la complexité dépend d'un ou plusieurs paramètres $k_0, \dots, k_r$, on dit que A a une complexité en $O(f(k_0, \dots, k_r))$ s'il existe une constante $C > 0$ telle que, pour toutes les valeurs de $k_0, \dots, k_r$ suffisamment grandes (c'est-à-dire plus grandes qu'un certain seuil), pour toute instance du problème de paramètres $k_0, \dots, k_r$ la complexité est au plus $C \times f(k_0, \dots, k_r)$. De même, on dit que A a une complexité en $\Theta(f(k_0, \dots, k_r))$ s'il existe deux constantes $C_1 > 0$ et $C_2 > 0$ telles que, pour toutes les valeurs de $k_0, \dots, k_r$ suffisamment grandes, la complexité de A est au moins $C_1 \times f(k_0, \dots, k_r)$ et au plus $C_2 \times f(k_0, \dots, k_r)$. Dans tout ce sujet, on note « log » le logarithme à base 2, et on utilisera exclusivement ce logarithme.

Programmation. Dans les questions de programmation en C et OCaml, on n'utilise que les fonctions qui sont incluses dans la bibliothèque standard du langage.

Pour le code écrit en C, on suppose que les en-têtes `stdbool.h`, `stdio.h`, `stdlib.h` et `assert.h` ont été inclus.

On rappelle quelques opérations sur les listes en OCaml.

- **List.iter** *f lst* applique la fonction *f*: 'a -> unit à chaque élément de la liste *lst* de type 'a list. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.iter** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.map** *f lst* renvoie la liste de type 'b list obtenue en appliquant la fonction *f*: 'a -> 'b à chaque élément de la liste *lst* de type 'a list. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.map** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.mem** *e lst* renvoie **true** si et seulement si *e* est un élément de *lst*. Si la complexité du test d'égalité est en $O(1)$, la complexité de **List.mem** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.exists** *f lst* renvoie **true** si et seulement s'il existe au moins un élément *e* de *lst* (liste de type 'a list) tel que *f e* renvoie **true** avec *f*: 'a -> bool. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.exists** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.for_all** *f lst* renvoie **true** si et seulement si tous les éléments de *lst* (de type 'a list) satisfont le prédicat *f*: 'a -> bool. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.for_all** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.filter** *f lst* renvoie la liste de tous les éléments de *lst* (de type 'a list) qui satisfont le prédicat *f*: 'a -> bool. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.filter** est en $O(k)$, où *k* est la longueur de *lst*.

On rappelle quelques opérations sur les tableaux en OCaml.

- **Array.length** *tab* renvoie la longueur du tableau *tab* en temps $O(1)$.
- **Array.make** *k v* crée un tableau de *k* éléments, tous initialisés à *v*, en temps $O(k)$.
- La valeur à la case numéro *i* du tableau *tab* est *tab.(i)*. Les cases sont numérotées à partir de zéro et l'accès est fait en temps $O(1)$.
- L'affectation de la case *i* du tableau *tab* avec la valeur *v* s'écrit *tab.(i) <- v*; sa complexité est en $O(1)$.
- **Array.copy** *tab* crée une copie du tableau *tab* en temps $O(k)$ avec *k* la longueur du tableau en argument.

Partie I. Équivalences de littéraux

Dans cette partie, on se place dans le cadre de la logique propositionnelle. On définit L_1 comme la classe des contraintes de la forme $\ell \leftrightarrow \ell'$ avec ℓ et ℓ' des littéraux. Une formule F est dite construite sur L_1 , notation $F \in \mathcal{F}(L_1)$, si elle est une conjonction de contraintes de L_1 . *Dans cette partie, on considère que la variable X_1 est toujours interprétée à 1, donc qu'elle représente la valeur de vérité vrai.* La formule F_0 définie ci-dessous est dans $\mathcal{F}(L_1)$:

$$F_0 \stackrel{\text{def}}{=} (X_3 \leftrightarrow \neg X_2) \wedge (X_4 \leftrightarrow X_3) \wedge (X_3 \leftrightarrow X_4) \wedge (X_1 \leftrightarrow \neg X_5)$$

Question 1. Donner une formule logiquement équivalente à F_0 en forme 2-CNF. ┘

Question 2. Donner une réduction en temps linéaire du problème P_{L_1} au problème de satisfiabilité des formules en 2-CNF. ┘

Mise en œuvre en OCaml. Une contrainte de L_1 est représentée par un couple d'entiers (i, j) avec $i, j \in [1; 2n]$. Pour tout $i \in [1; 2n]$, le littéral représenté par i , noté $\text{lit}(i)$, est X_i si $i \in [1; n]$ et $\neg X_{i-n}$ si $i \in [n+1; 2n]$. Par la convention sur l'interprétation de X_1 , l'entier $i = 1$ (resp. $i = n+1$) représente la valeur de vérité 1 (resp. 0). Enfin, une formule dans $\mathcal{F}(L_1)$ est représentée par la liste des couples correspondant aux contraintes dont la formule est conjonction. Par convention, la liste vide représente $\top$. Le nombre de variables, n , est une variable globale entière. Ce codage est donné par les déclarations OCaml ci-dessous :

```
1 type contr1 = int * int    (* contraintes L1 *)
2 type eform1 = contr1 list  (* formules L1 *)
3 exception Bad_literal of int
4 exception Exn_unsat
```

Les exceptions déclarées ci-dessus seront utilisées dans les fonctions de cette partie et de la partie suivante.

Question 3. Écrire les fonctions suivantes :

- `var_of_lit: int -> int` renvoie l'index de la variable apparaissant dans le littéral représenté par l'argument i . Par exemple, pour $n = 5$, la fonction renvoie 3 si l'argument est 3, et 2 si l'argument est 7. Si $i \notin [1; 2n]$ alors la fonction exécute `raise (Bad_literal i)`.
- `neg_lit: int -> int` renvoie, pour un argument i , l'entier j tel que $\text{lit}(j)$ est logiquement équivalent à $\neg \text{lit}(i)$; si $i \notin [1; 2n]$ alors la fonction exécute `raise (Bad_literal i)`.
- `eform1_is_wf: eform1 -> bool` renvoie `true` si et seulement si l'argument est « bien formé », c'est-à-dire qu'il contient uniquement des entiers qui représentent des littéraux sur l'ensemble de variables $\mathbb{X}$.

Dans la suite de ce sujet, on suppose que tous les entiers utilisés par les valeurs de type `contr1` et `eform1` représentent des littéraux sur l'ensemble de variables $\mathbb{X}$. Par conséquent, l'exception `Bad_literal` ne doit pas être levée. ┘

Graphe d'une formule. Le *graphe associé* à $F \in \mathcal{F}(L_1)$, noté $G_F = (V_F, E_F)$, est un graphe non orienté dont les sommets V_F sont les entiers dans l'intervalle $[1; 2n]$. Les arêtes E_F sont les ensembles $\{i, j\}$ tels que $i, j \in V_F$, $i \neq j$ et F contient soit la contrainte $\text{lit}(i) \leftrightarrow \text{lit}(j)$, soit la contrainte $\text{lit}(i') \leftrightarrow \text{lit}(j')$ avec i' et j' égaux respectivement à $\text{neg_lit}(i)$ et $\text{neg_lit}(j)$. Par exemple, pour $n = 5$, la contrainte $(X_3 \leftrightarrow X_4)$ est représentée par les arêtes $\{3, 4\}$ et $\{8, 9\}$. On note que les contraintes de F de la forme $\ell \leftrightarrow \ell$ ne sont pas représentées par E_F .

Question 4. On suppose $n = 5$. Pour chacune des formules suivantes, donner le graphe associé :

$$\begin{aligned} F_1 &\stackrel{\text{def}}{=} (X_3 \leftrightarrow X_4) \wedge (X_4 \leftrightarrow X_2) \wedge (X_3 \leftrightarrow \neg X_2) \wedge (X_5 \leftrightarrow X_2) \\ F_2 &\stackrel{\text{def}}{=} (\neg X_4 \leftrightarrow X_2) \wedge (X_2 \leftrightarrow \neg X_3) \wedge (X_4 \leftrightarrow \neg X_5) \wedge (X_3 \leftrightarrow X_5) \end{aligned}$$

┘

On suppose disposer d'une bibliothèque munie d'un type **graph** et qui code des graphes non orientés. Les sommets sont représentés par des entiers. Les opérations suivantes sont disponibles pour la construction et l'exploration des graphes :

- **g_create** : **int** -> **graph** renvoie une valeur représentant un graphe sans arêtes et ayant k sommets, où k est donné en argument, supposé strictement positif. Les sommets sont numérotés de 1 à k . La complexité de cette opération est en $O(k)$.
- **g_add_edge** : **graph** -> **int** -> **int** -> **unit** ajoute au graphe donné comme premier argument l'arête entre les sommets donnés en 2ème et 3ème arguments. Si les sommets en argument ne sont pas déclarés dans le graphe ou si l'arête existe déjà, alors cette fonction ne change pas le graphe. Sinon, le graphe est modifié en place. La complexité de cette opération est en $O(k)$, avec k le nombre de sommets du graphe.
- **g_nb_vertex** : **graph** -> **int** renvoie le nombre de sommets du graphe en argument. La complexité de cette opération est en $O(1)$.
- **g_succ** : **graph** -> **int** -> **int list** renvoie la liste des sommets adjacents du sommet en second argument dans le graphe donné en premier argument. Si le sommet en argument n'existe pas, la liste renvoyée est vide. La complexité de cette opération est en $O(1)$.

Question 5. Écrire une fonction **eform1_to_graph**: **eform1** -> **graph** qui renvoie le graphe associé à la formule F représentée par l'argument. Si F contient une contrainte de la forme $\ell \leftrightarrow \neg \ell$, alors la fonction lève l'exception **Exn_unsat**. ┘

Lorsqu'un appel à **eform1_to_graph** **lst** lève l'exception **Exn_unsat**, la formule représentée par **lst** contient toujours une contrainte contradictoire. Cependant, une formule peut être une contradiction sans que l'exception soit levée, comme c'est le cas de la formule $(X_1 \leftrightarrow X_2) \wedge (X_2 \leftrightarrow \neg X_1)$.

Question 6. Donner et justifier soigneusement la complexité de **eform1_to_graph** en fonction de n et de la longueur p de la liste en argument. ┘

Soient $F \in \mathcal{F}(L_1)$ une formule et **g** la représentation de G_F , le graphe associé à F . On note par $CC(G_F)$ l'ensemble des ensembles de sommets des composantes connexes de G_F . Soit m la taille de $CC(G_F)$, c'est-à-dire, le nombre de composantes connexes de G_F . On représente $CC(G_F)$ par un tableau **c** de $2n + 1$ entiers, qui associe à chaque $i \in [1; 2n]$, sommet de **g**, un entier dans l'intervalle $[1; m]$. Ainsi, **c**.(**i**) identifie de manière unique la composante connexe du sommet **i**. La case de **c** à la position 0 ne sera pas utilisée, mais elle permet d'utiliser directement les indices des variables comme positions de **c**. Par convention, **c**.(0) est 0.

Question 7. Écrire une fonction `g_cc_array: graph -> (int array * int)` qui renvoie le couple (c, m) , où c est le tableau des composantes connexes du graphe en argument et où m est le nombre de composantes connexes. ┘

Question 8. Démontrer que si i et j sont des sommets distincts appartenant à la même composante connexe de G_F , alors $F \models \text{lit}(i) \leftrightarrow \text{lit}(j)$. ┘

Question 9. Soit $C = \{i_1, \dots, i_q\} \in CC(G_F)$ l'ensemble des sommets d'une composante connexe de G_F . Soit C' l'ensemble de sommets $\{j_1, \dots, j_q\}$ avec $j_k = \text{neg_lit}(i_k)$ pour tout $k \in [1; q]$. Démontrer que C' est élément de $CC(G_F)$. ┘

Question 10. Démontrer que F est une contradiction si et seulement s'il existe une composante connexe C de G_F et un sommet i de C tels que $\text{neg_lit}(i)$ est un sommet de C . ┘

Question 11. Écrire une fonction `cc_is_sat: int array -> bool` qui a comme argument c , le tableau représentant les composantes connexes de G_F et qui renvoie `true` si et seulement si F est satisfiable. ┘

Représentation d'une valuation. Une valuation v de $\mathbb{X}$ est représentée par un tableau v de $n + 1$ valeurs 0 ou 1 tel que $v.(i)$ est 1 si et seulement si $v(X_i) = 1$. En particulier, $v.(1)=1$ par convention sur X_1 . La case à la position 0 n'est pas utilisée, mais elle permet d'utiliser les indices des variables comme positions du tableau; par convention, $v.(0)$ est 0.

Question 12. Écrire une fonction `eform1_sat: eform1 -> (int array) option` qui a comme argument une liste `lst` représentant une formule $F \in \mathcal{F}(L_1)$. L'appel `eform1_sat lst` renvoie `None` si F est une contradiction. Si F est satisfiable, l'appel renvoie `Some(v)` avec v un tableau qui représente une valuation qui satisfait F . La complexité en temps de l'algorithme doit être en $O(p \times n)$ où p est la longueur de la liste `lst` en argument. Justifier que votre algorithme a cette complexité en temps. ┘

Partie II. Équivalences avec une conjonction de deux littéraux

On reste dans le cadre de la logique propositionnelle, mais on considère une nouvelle classe de contraintes. La classe L_2 contient les contraintes de la forme $\ell \leftrightarrow (\ell' \wedge \ell'')$ où ℓ, ℓ', ℓ'' sont des littéraux. Pour identifier les contraintes des classes L_1 et L_2 , on appelle *i-contrainte* une contrainte de la classe L_i avec $i \in \{1, 2\}$. Une formule F est dite construite sur L_2 , notation $F \in \mathcal{F}(L_2)$, si elle est une conjonction de 2-contraintes. **On garde l'hypothèse que X_1 est toujours interprétée à 1.** Un exemple de formule de $\mathcal{F}(L_2)$ est :

$$F_3 \stackrel{\text{def}}{=} (X_1 \leftrightarrow (X_2 \wedge \neg X_4)) \wedge (\neg X_2 \leftrightarrow (X_2 \wedge X_3)) \wedge (X_4 \leftrightarrow (\neg X_1 \wedge X_5))$$

Question 13. Donner une réduction en temps polynomial du problème P_{L_2} au problème de satisfiabilité des formules en 3-CNF. $\lrcorner$

Question 14. Donner une dérivation formelle (preuve en déduction naturelle) du séquent suivant :

$$X_2 \rightarrow (\neg X_2 \wedge X_3), (\neg X_2 \wedge X_3) \rightarrow X_2 \quad \vdash \quad \neg X_2.$$

Les règles de la déduction naturelle à utiliser dans cette preuve sont rappelées dans l'annexe. $\lrcorner$

En utilisant la convention sur l'interprétation de X_1 , toute 1-contrainte $\ell \leftrightarrow \ell'$ est logiquement équivalente à la 2-contrainte $\ell \leftrightarrow (\ell' \wedge X_1)$. À l'inverse, certaines 2-contraintes sont logiquement équivalentes à des formules de $\mathcal{F}(L_1)$. Par exemple, les équivalences logiques suivantes sont vraies pour tout couple de littéraux ℓ et ℓ' (les preuves ne sont pas demandées) :

- | | |
|--|---|
| $\ell \leftrightarrow (X_1 \wedge \ell')$ équivalent à $\ell \leftrightarrow \ell'$ | par 1 élément neutre de $\wedge$ (1) |
| $\ell \leftrightarrow (\neg X_1 \wedge \ell')$ équivalent à $\ell \leftrightarrow \neg X_1$ | par 0 élément absorbant de $\wedge$ (2) |
| $\ell \leftrightarrow (\ell' \wedge \ell')$ équivalent à $\ell \leftrightarrow \ell'$ | par idempotence de $\wedge$ (3) |
| $\ell \leftrightarrow (\ell' \wedge \neg \ell')$ équivalent à $\ell \leftrightarrow \neg X_1$ | par contradiction (4) |
| $X_1 \leftrightarrow (\ell \wedge \ell')$ équivalent à $(X_1 \leftrightarrow \ell) \wedge (X_1 \leftrightarrow \ell')$ | par définition de $\wedge$ (5) |
| $\ell \leftrightarrow (\neg \ell \wedge \ell')$ équivalent à $(\neg X_1 \leftrightarrow \ell) \wedge (\neg X_1 \leftrightarrow \ell')$ | admis (6) |

Dans la suite de cette partie, on dénote par $\mathbb{E}(\mathbb{X})$ toutes les équivalences définies sur l'ensemble des littéraux par les équivalences de (1) à (6) et par leur variantes obtenues en utilisant la symétrie de la conjonction. Par exemple, « $\ell \leftrightarrow (\ell' \wedge X_1)$ équivalent à $\ell \leftrightarrow \ell'$ » est également une équivalence de $\mathbb{E}(\mathbb{X})$, grâce à la symétrie de la conjonction appliquée à (1).

Mise en œuvre en OCaml. Une 2-contrainte $\ell \leftrightarrow (\ell' \wedge \ell'')$ est représentée par un enregistrement avec trois champs entiers. Les entiers de ces enregistrements prennent des valeurs dans $[1; 2n]$ et représentent des littéraux avec la même convention de codage qu'en première partie. Une formule dans $\mathcal{F}(L_2)$ est représentée par une liste d'enregistrements; la liste vide représente $\top$. Ce codage est donné par les types OCaml suivants :

```

1 type contr2 = { ll: int; lr1: int; lr2: int } (* contraintes L2 *)
2 type eform2 = contr2 list                    (* formules L2 *)
```


Une valeur e de type `contr2` représente la 2-contrainte $e.l1 \leftrightarrow (e.lr1 \wedge e.lr2)$.

Dans la suite de cette partie, on suppose que toutes les valeurs des types `contri` et `eformi` avec $i \in \{1, 2\}$ sont bien formées, c'est-à-dire que les entiers qu'elles contiennent représentent des littéraux sur $\mathbb{X}$.

Question 15. Écrire la fonction `contr2_simplify_eq5: contr2 -> eform1 option` qui applique l'équivalence (5) pour simplifier la 2-contrainte en argument. L'appel `contr2_simplify_eq5 e2` :

- renvoie **None** si la simplification ne peut pas être faite avec (5) ;
- lève l'exception **Exn_unsat** (déclarée en partie I) si la simplification peut être faite mais la 2-contrainte représentée par `e2` est une contradiction ;
- enfin, renvoie **Some** `[(1,e2.lr1);(1,e2.lr2)]` si la simplification peut être effectuée sans détecter de contradiction. ┐

Valuation partielle. Une valuation partielle est une fonction $v : \mathbb{X} \rightarrow \{-1, 0, 1\}$ telle que $v(X_1) = 1$. Pour tout $i \neq 1$, si $v(X_i) = b \in \mathbb{B}$ alors on dit que v assigne X_i à b . Si $v(X_i) = -1$ alors on dit que X_i n'est pas assignée par v . On étend la notion de valuation partielle aux littéraux et aux formules comme suit :

- $v(\neg X_i) = \neg b$ si $v(X_i) = b$ avec $b \in \mathbb{B}$, et $v(\neg X_i) = -1$ si $v(X_i) = -1$.
- Soit la 1-contrainte $\ell \leftrightarrow \ell'$. Si $v(\ell), v(\ell') \in \mathbb{B}$ alors $v(\ell \leftrightarrow \ell') = 1$ si $v(\ell) = v(\ell')$ et 0 sinon. Si au moins une des valeurs $v(\ell)$ ou $v(\ell')$ est -1 , alors $v(\ell \leftrightarrow \ell') = -1$.
- Pour la conjonction de deux littéraux $\ell \wedge \ell'$, $v(\ell \wedge \ell') = -1$ si au moins une des valeurs $v(\ell)$ ou $v(\ell')$ est -1 ; sinon la valeur de $v(\ell \wedge \ell')$ est 1 si et seulement si $v(\ell) = 1$ et $v(\ell') = 1$.
- Soit la 2-contrainte $\ell \leftrightarrow (\ell' \wedge \ell'')$. Si $v(\ell), v(\ell' \wedge \ell'') \in \mathbb{B}$ alors $v(\ell \leftrightarrow (\ell' \wedge \ell'')) = 1$ si $v(\ell) = v(\ell' \wedge \ell'')$ et 0 sinon. Si au moins une des valeurs $v(\ell)$ ou $v(\ell' \wedge \ell'')$ est -1 , alors $v(\ell \leftrightarrow (\ell' \wedge \ell'')) = -1$.

La **valeur d'une formule** $F_i \in \mathcal{F}(L_i)$ où $i \in \{1, 2\}$ pour une valuation partielle v , notation $v(F_i)$, est -1 s'il existe une contrainte e de F_i telle que $v(e) = -1$; sinon, $v(F_i) = 1$ si pour toute contrainte e de F_i on a $v(e) = 1$, et $v(F_i) = 0$ sinon.

Une valuation partielle v est **compatible** avec une valuation v' si pour tout $X_i \in \mathbb{X}$ tel que $v(X_i) \in \mathbb{B}$ on a $v'(X_i) = v(X_i)$.

Une valuation partielle v est représentée en OCaml par un tableau `v` de taille $n + 1$ dont les éléments appartiennent à $\{-1, 0, 1\}$. Comme pour les valuations, la case de position 0 n'est pas utilisée et, par convention, `v.(0)=0`.

Question 16. Écrire la fonction `eform1_of_valp: int array -> eform1` qui renvoie une formule F de $\mathcal{F}(L_1)$ telle que F encode les assignations de la valuation partielle `v` en argument. Ainsi, un appel à `eform1_of_valp v` renvoie la liste de couples (i, j) tels que $i > 0$, `v.(i)  $\neq -1$` et si `v.(i) = 1` alors $j = 1$ et si `v.(i) = 0` alors $j = n + 1$.

Soit F la formule représentée par le résultat de l'appel à `eform1_of_valp v`. Démontrez que, pour toute valuation v' on a $v' \models F$ si et seulement si `v` est compatible avec v' . ┐

Une 2-contrainte peut être simplifiée en présence d'une valuation partielle v en une formule de $\mathcal{F}(L_1)$. Par exemple, si $v(X_2) = 1$ alors la 2-contrainte $X_2 \leftrightarrow (\ell' \wedge \ell'')$ a la même valeur que la 1-contrainte $(X_1 \leftrightarrow \ell') \wedge (X_1 \leftrightarrow \ell'')$ pour la valuation partielle v . On suppose donnée une fonction `contr2_simplify_valp: contr2 -> int array -> eform1 option` qui applique les équivalences de $\mathbb{E}(\mathbb{X})$ pour simplifier la 2-contrainte `e2` en premier argument en tenant compte de

la valuation partielle vp donnée comme second argument. Plus précisément, si $vp(e2)=0$, alors la fonction lève l'exception **Exn_unsat**. Si aucune simplification ne s'applique, alors le résultat est **None**. Si $e2$ est simplifiable, le résultat est **Some(1st)** avec $1st$ une liste représentant une formule F'_1 de $\mathcal{F}(L_1)$ obtenue par simplification de $e2$ telle que $vp(e2) = vp(F'_1)$. La complexité de `contr2_simplify_valp` est en $O(1)$.

Algorithme. Les questions suivantes vous guident pour prouver la correction et la terminaison de l'algorithme de décision pour P_{L_2} qui est mis en œuvre dans la fonction `eform2_sat` de la figure 1.

On cherche à démontrer que la fonction `eform2_sat` est correcte, c'est-à-dire qu'un appel à `eform2_sat 1st0` doit renvoyer **None** si la formule $F_0 \in \mathcal{F}(L_2)$ représentée par `1st0` est une contradiction. Sinon, l'appel doit renvoyer **Some(v)** où v est un tableau qui représente une valuation qui satisfait F_0 . L'algorithme mis en œuvre pour `eform2_sat` est un algorithme de type retour sur trace (*backtracking*) dont le principe est de réduire la formule $F_0 \in \mathcal{F}(L_2)$ en une formule de $\mathcal{F}(L_1)$ afin d'appliquer `eform1_sat`, l'algorithme polynomial de la partie I. Pour effectuer cette réduction, `eform2_sat` appelle à la ligne 5 la fonction récursive `eform2_sat_aux` dont les arguments sont une liste `12` représentant une formule $F_2 \in \mathcal{F}(L_2)$, une liste `11` représentant une formule $F_1 \in \mathcal{F}(L_1)$ et une valuation partielle vp .

La correction de `eform2_sat_aux` est un lemme important de la preuve de correction de `eform2_sat`. Un appel à `eform2_sat_aux 12 11 vp` est correct s'il renvoie un tableau qui représente une valuation v telle que $v \models F_2 \wedge F_1$ et vp est compatible avec v , si une telle valuation existe ; sinon, l'appel doit lever l'exception **Exn_unsat**.

La mise en œuvre de `eform2_sat_aux` cherche à compléter la valuation partielle vp en assignant des valeurs à certaines variables (non assignées) qui apparaissent dans les littéraux de F_2 et qui permettent de simplifier F_2 en une formule de $\mathcal{F}(L_1)$. Pour savoir si une 2-contrainte $e2$ de `12` peut être simplifiée en présence de vp , on appelle `contr2_simplify_valp e2 vp` (ligne 15). Si $e2$ se simplifie, l'argument `11` accumule les formules de $\mathcal{F}(L_1)$ obtenues par la simplification et `eform2_sat_aux` continue la simplification du reste des contraintes de `12` (ligne 17). Si la contrainte $e2$ est une contradiction en présence de vp , l'exception **Exn_unsat** est levée par `contr2_simplify_valp e2 vp`. Si $e2$ ne peut pas être simplifiée (ligne 19), alors `eform2_sat_aux` explore les simplifications de la contrainte $e2$ obtenues en assignant la variable du littéral `e2.l1r1` à chaque valeur de $\mathbb{B}$. L'exploration d'une assignation est faite grâce à un appel récursif à `eform2_sat_aux` avec le paramètre représentant la valuation partielle mis à jour avec l'assignation choisie.

Quand toutes les 2-contraintes de F_2 ont été simplifiées, c'est-à-dire quand la liste `12` est vide (ligne 10), `eform2_sat_aux` appelle `eform1_sat` avec l'argument `(11'@11)` où la liste `11'` est obtenue par l'appel à `eform1_of_valp vp`. Si l'appel à `eform1_sat (11'@11)` renvoie **Some(v)**, alors le résultat de `eform2_sat_aux` est v . Sinon, `eform2_sat_aux` lève l'exception **Exn_unsat**. Ceci a comme effet de revenir dans un appel précédent (sur la pile d'appels) de `eform2_sat_aux` pour explorer une autre assignation (lignes 25–28) ou, si toutes les assignations ont été explorées, de terminer l'appel à `eform2_sat_aux 12 11 vp` par l'exception **Exn_unsat**.

Question 17. Démontrer que tout appel `eform2_sat 1st0` termine. ┘

Question 18. Démontrer que la fonction `eform2_sat` est correcte en supposant que la fonction `eform2_sat_aux` est correcte. ┘

Dans les trois questions suivantes, on cherche à démontrer les principaux cas de la correction de `eform2_sat_aux`.

```

1  let rec eform2_sat (lst:eform2) : (int array) option =
2    try
3      let vinit = Array.make (n+1) (-1) in (
4        vinit.(0) <- 0; vinit.(1) <- 1;
5        Some (eform2_sat_aux lst [] vinit) )
6    with Exn_unsat -> None
7
8  and eform2_sat_aux (l2:eform2) (l1:eform1) (vp:int array) : int array =
9    match l2 with
10   | [] -> let l1' = eform1_of_valp vp in
11           let r = eform1_sat (l1'@l1) in
12           (match r with None -> raise Exn_unsat | Some v -> v)
13
14   | e2::l2' ->
15     match contr2_simplify_valp e2 vp with
16     | Some l1' -> (* simplification *)
17       eform2_sat_aux l2' (l1'@l1) vp
18
19     | None -> (* branchement sur valeur de e2.lr1 *)
20       let xlr1 = var_of_lit e2.lr1 in
21       try
22         let vp1 = Array.copy vp in (
23           vp1.(xlr1) <- 1;
24           eform2_sat_aux l2 l1 vp1 )
25       with Exn_unsat ->
26         let vp0 = Array.copy vp in (
27           vp0.(xlr1) <- 0;
28           eform2_sat_aux l2 l1 vp0 )

```

FIGURE 1 – Mise en œuvre OCaml de l'algorithme de décision pour P_{L_2}

Question 19. Démontrez la correction de `eform2_sat_aux` quand son premier argument est la liste vide, on supposant que les fonctions `eform1_of_valp` et `eform1_sat` sont correctes. ┘

Question 20. Soit un appel `eform2_sat_aux l2 l1 vp` tel que `l2` est `e2::l2'` et `contr2_simplify_vp e2 vp` renvoie `Some l1'`. Démontrez la correction d'un tel appel en supposant que l'appel récursif à `eform2_sat_aux` de la ligne 17 est correct. ┘

Question 21. Soit un appel `eform2_sat_aux l2 l1 vp` tel que `l2` est `e2::l2'` et `contr2_simplify_vp e2 vp` renvoie `None`. Démontrez la correction d'un tel appel en supposant que les appels récursifs aux lignes 24 et 28 sont corrects. ┘

Question 22. Donner la complexité de `eform2_sat` en fonction de la longueur k de la liste en argument et du nombre de variables n . ┘

Partie III. Égalités et différences entre entiers

Dans cette partie, le domaine d'interprétation des variables est $\mathbb{Z}$. La classe L_3 est constituée de contraintes de la forme $X_i = X_j$ et $X_i \neq X_j$, avec $X_i, X_j \in \mathbb{X}$. Une formule F est dite construite sur L_3 , notation $F \in \mathcal{F}(L_3)$, si elle est une conjonction de contraintes de L_3 . Une valuation est une fonction $v : \mathbb{X} \rightarrow \mathbb{Z}$ qui assigne une valeur dans $\mathbb{Z}$ à chaque variable. Une valuation v **satisfait** une contrainte $X_i = X_j$ (resp. $X_i \neq X_j$) si et seulement si $v(X_i) = v(X_j)$ (resp. $v(X_i) \neq v(X_j)$). Une valuation v **satisfait** une formule $F \in \mathcal{F}(L_3)$ si et seulement si v satisfait chaque contrainte de F . Les notions de conséquence logique, équivalence logique, tautologie et antilogie sont définies de manière similaire à la logique propositionnelle.

Le problème P_{L_3} peut être réduit (en temps polynomial en n et en la taille de la formule) au problème de satisfiabilité d'une formule propositionnelle. Dans cette partie, on étudie un algorithme polynomial en n et en la taille de la formule pour P_{L_3} .

Mise en œuvre en langage C. On propose de représenter les formules de $\mathcal{F}(L_3)$ par des listes simplement chaînées acycliques de type `eform3_t*`. Chaque cellule `c` (de type `eform3_t`) d'une telle liste représente une contrainte $e \in L_3$ grâce à trois données : un booléen `c.is_eq` qui est vrai si et seulement si e est une égalité, et deux entiers positifs `c.xi` et `c.xj` qui représentent les indices des variables utilisées par e . La formule $\top$ est représentée par la liste vide de valeur `NULL`. On suppose que le nombre n de variables dans $\mathbb{X}$ est déclaré comme une variable globale entière. Enfin, on suppose que toute structure `eform3_t` est bien formée, c'est-à-dire que tous les entiers qu'elle contient appartiennent à l'intervalle $[1; n]$.

Question 23. Donner la déclaration du type `eform3_t`. ┘

Question 24. Écrire la fonction `void eform3_print(eform3_t *lst)` qui affiche sur la sortie standard chaque contrainte (par exemple, sous la forme `X_1 != X_4`) de la formule représentée par `lst`, une contrainte par ligne ; si `lst` est `NULL`, la fonction affiche `true` sur une ligne. ┘

Classes d'équivalence avec « unir et trouver ». On utilise une structure unir et trouver (*union-find*) pour résoudre le problème P_{L_3} . On suppose disposer d'une bibliothèque C qui fournit une implémentation d'une structure unir et trouver sur des ensembles d'entiers. Cette bibliothèque exporte les déclarations de types et de fonctions suivantes :

```

1  /* Rappel: definition dans stdlib.h
2  typedef unsigned int size_t; */
3
4  struct uf_s {          /* type unir et trouver (union-find) */
5      size_t nelem;      /* nombre d'elements de l'ensemble */
6      int *parent;      /* tableau de taille nelem = relation parent */
7      ...               /* champs utiles pour operations en O(log n) */
8  };
9  typedef struct uf_s uf_t;
10 uf_t *uf_create(size_t sz);
11 int uf_find(uf_t *cs, int i);
12 void uf_union(uf_t *cs, int i1, int i2);

```

Dans ce qui suit, on supposera les propriétés suivantes de cette implémentation :

- `uf_create(s)` renvoie un pointeur `cs` vers une structure unir et trouver allouée dans le tas et qui représente une partition de l'ensemble $\{0, \dots, s-1\}$ où chaque élément de l'ensemble est seul dans sa classe d'équivalence ; `s` donne la valeur du champ `cs->nelem`.
- `uf_find(cs, i)` renvoie la classe de `i`, c'est-à-dire un entier entre 0 et `cs->nelem-1` qui est le représentant de cette classe.
- `uf_union(cs, i1, i2)` fusionne les classes de `i1` et `i2` en modifiant la structure `*cs`.

Si `cs` est un pointeur valide vers une structure `uf_t` avec `cs->nelem==s`, le champ `cs->parent` pointe vers un tableau de `s` entiers ayant des valeurs dans l'intervalle $[0; s-1]$; ce tableau encode la relation représentant d'une classe d'équivalence. Si `cs->parent[i]==j` alors `i` est dans la classe de `j`. Le tableau `cs->parent` est initialisé par `uf_create` de sorte que `cs->parent[i]==i`. Les opérations ci-dessus sur la structure unir et trouver ont une complexité en $\Theta(s)$ pour `uf_create` et en $O(\log s)$ pour `uf_find` et `uf_union`.

Pour notre problème, *l'ensemble de valeurs à partitionner sera* $[0; n]$, c'est-à-dire l'ensemble d'indices de variables dans $\mathbb{X}$ auquel on ajoute la valeur 0. La valeur 0 ne sera pas utilisée, mais elle permet d'utiliser directement les indices des variables comme positions du tableau `cs->parent`.

Question 25. Écrire la fonction `uf_t *eform3_to_uf(eform3_t *lst)` qui renvoie une valeur de type `uf_t*` représentant les classes d'équivalence sur $[0; n]$ définies par les *contraintes d'égalité* de la formule représentée par `lst`. On suppose que `lst` n'est pas vide. Les contraintes de différence de `lst` seront ignorées dans cette fonction. ┘

Algorithme de décision. D'après la question précédente, la structure unir et trouver permet donc de représenter les contraintes d'égalité d'une formule de $\mathcal{F}(L_3)$, mais pas les contraintes de différence.

Question 26. Écrire une fonction `bool eform3_sat_int(eform3_t *lst)` qui renvoie `true` si et seulement si toutes les contraintes de différence de la formule représentée par `lst` peuvent être satisfaites en respectant les contraintes d'égalité de `lst`. Si `lst` est `NULL`, la fonction renvoie `true`. Justifier la correction de la fonction `eform3_sat_int`, c'est-à-dire, que tout appel à cette fonction renvoie `true` si et seulement si la formule représentée par son argument est satisfiable. ┘

Question 27. On suppose que `lst` (de type `eform3_t*`) représente une formule avec un nombre de contraintes d'égalité égal à e et un nombre de contraintes de différence égal à d . Donner la complexité de l'appel à `eform3_sat_int(lst)` en fonction de n , e et d . ┘

Calculer une valuation. Dans la suite de cette partie, on se propose de calculer, si elle existe, une valuation $v : \mathbb{X} \rightarrow \mathbb{Z}$ qui satisfait une formule de $\mathcal{F}(L_3)$.

Question 28. Écrire une fonction `int uf_classes(uf_t *cs)` qui renvoie le nombre de classes d'équivalence de la structure unir et trouver pointée par `cs` ; si `cs` est `NULL` alors la fonction renvoie 0. Cette fonction doit avoir une complexité en $O(n)$. ┘

Question 29. Écrire une fonction `int *uf_valuation(uf_t *cs)` qui renvoie un pointeur `v` vers un tableau de $n + 1$ entiers, alloué dans le tas. Soit k le résultat de l'appel de `uf_classes(cs)`. On suppose $k \geq 2$. Pour tout $i \in [1; n]$, `v[i]` est une valeur entière dans l'intervalle $[1; k]$ qui

est associée *de façon unique* à (chaque élément de) la classe d'équivalence de i . La case à la position 0 de v n'est pas utilisée, mais elle permet d'utiliser les indices des variables comme positions de v ; par convention, $v[0] = 0$. La fonction `uf_valuation` doit avoir une complexité en $O(n \log n)$. ┘

Soit $F \in \mathcal{F}(L_3)$ une formule représentée par une liste `lst` différente de `NULL` telle que `eform3_sat_int(lst)` renvoie `true`. Soient `cs` le résultat de l'appel à `eform3_to_uf(lst)` et `v` le résultat de l'appel à `uf_valuation(cs)`.

Question 30. Démontrer que `v` pointe vers un tableau qui représente une valuation qui satisfait la formule F représentée par `lst`. ┘

Question 31. Si F ne contient pas de contraintes de différence, alors quel est le nombre minimal de valeurs que peut avoir l'image d'une valuation v qui satisfait F ? Donner un exemple d'une telle valuation.

La même question si F contient une seule contrainte de différence. ┘

Question 32. Lorsque cela est possible, on souhaite construire une valuation v telle que v satisfait la formule F et l'image de v est un ensemble de taille 2. Quel algorithme sur les graphes peut être utilisé pour obtenir ce résultat et sur quel graphe? Vous devez définir formellement le graphe en entrée de l'algorithme et donner la complexité de l'algorithme. ┘

Partie IV. Égalités et différences d'expressions

Soit $\mathbb{F}$ un ensemble de m symboles de fonction notés f_i ($1 \leq i \leq m$). Chaque symbole $f_i \in \mathbb{F}$ représente une fonction de $\mathbb{Z} \times \mathbb{Z}$ dans $\mathbb{Z}$. Comme dans la partie III, on interprète les variables X_i de $\mathbb{X}$ dans l'ensemble $\mathbb{Z}$. Les symboles de fonctions de $\mathbb{F}$ **ne sont pas interprétés**, c'est-à-dire qu'ils n'ont pas de définition. Les seules propriétés connues de ces fonctions sont données sous la forme de contraintes d'égalité ou de différence. Ces contraintes sont celles de la classe L_4 définie ci-dessous.

Les contraintes de la classe L_4 utilisent des termes. Un **terme** t est défini de manière inductive : t est

- soit X_i , une variable de $\mathbb{X}$,
- soit $f_i(t_1, t_2)$, l'application d'une fonction $f_i \in \mathbb{F}$ sur deux termes t_1 et t_2 , appelés aussi **sous-termes directs** de t .

Le **symbole racine** du terme X_i (resp. $f_j(t_1, t_2)$) est X_i (resp. f_j). On note $\mathbb{T}(\mathbb{X}, \mathbb{F})$ l'ensemble des termes construits avec les variables de $\mathbb{X}$ et les symboles de fonctions $\mathbb{F}$.

Ces termes peuvent être représentés par des arbres binaires. La figure 2 représente les arbres binaires correspondants aux termes :

$$t_1 \stackrel{\text{def}}{=} f_1(f_2(X_1, X_2), X_2) \quad \text{et} \quad t_2 \stackrel{\text{def}}{=} f_3(X_1, f_2(X_1, X_2)).$$

Chaque noeud de ces arbres est étiqueté en partie haute par un nom unique u_i appelé identifiant, et en partie basse par le symbole racine du terme. L'arc à gauche (resp. droite) d'un noeud correspond au premier (resp. second) sous-terme direct et est dessiné avec un trait en pointillé (resp. plein).

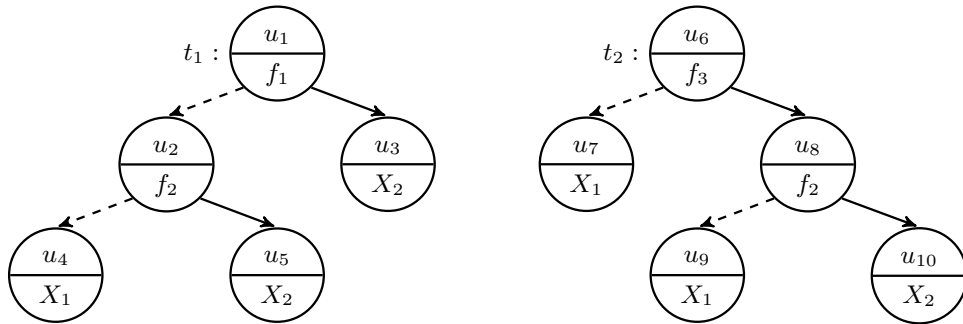


FIGURE 2 – Arbres des termes t_1 (à gauche) et t_2 (à droite)

L'ensemble des sous-termes d'un terme t , noté $\mathcal{T}(t)$, est défini de manière inductive par

$$\mathcal{T}(X_i) \stackrel{\text{def}}{=} \{X_i\} \quad \text{et} \quad \mathcal{T}(f_i(t_1, t_2)) \stackrel{\text{def}}{=} \{f_i(t_1, t_2)\} \cup \mathcal{T}(t_1) \cup \mathcal{T}(t_2).$$

Question 33. Donner $\mathcal{T}(t_1)$ et $\mathcal{T}(t_2)$ pour les termes t_1 et t_2 définis ci-dessus. ┐

La classe L_4 contient les contraintes de la forme $t_1 = t_2$ ou $t_1 \neq t_2$ avec t_1 et t_2 des termes. Une formule F de $\mathcal{F}(L_4)$ est une conjonction de contraintes L_4 . L'ensemble des sous-termes d'une contrainte e de la forme $t_1 = t_2$ ou $t_1 \neq t_2$ est $\mathcal{T}(e) \stackrel{\text{def}}{=} \mathcal{T}(t_1) \cup \mathcal{T}(t_2)$. L'ensemble des sous-termes d'une formule F de $\mathcal{F}(L_4)$, noté $\mathcal{T}(F)$, est défini comme l'union des ensembles des sous-termes de toutes les contraintes de F .

Mise en œuvre en langage C. Afin de concevoir un algorithme de décision pour $P(L_4)$, nous représentons *les contraintes d'égalité* d'une formule de $\mathcal{F}(L_4)$ par une structure de données appelée *e-graphe*. Cette structure combine un graphe orienté acyclique avec une structure unir et trouver. Les définitions de types C suivantes codent une structure de e-graphe :

```

1 struct tnode_s { /* type des noeuds des arbres representant les termes */
2     char *label; /* symbole racine */
3     int li; /* position dans tset du premier sous-terme direct ou 0 */
4     int ri; /* position dans tset du second sous-terme direct ou 0 */
5 };
6 typedef struct tnode_s tnode_t;
7
8 struct egraph_s { /* type e-graphe representant une formule  $L_4$  */
9     size_t tset_sz; /* longueur de tset */
10    tnode_t *tset; /* debut tableau des termes de la formule */
11    int *pre; /* relation sous-terme -- terme */
12    uf_t *uf; /* unir et trouver pour les classes des termes */
13 };
14 typedef struct egraph_s egraph_t;

```

Dans ce qui suit, on supposera les propriétés suivantes de la mise en œuvre de la structure `egraph_t`. Si `g` est une variable de type `egraph_t` représentant les contraintes d'égalité d'une formule $F \in \mathcal{F}(L_4)$, `g.tset` code l'ensemble des sous-termes $\mathcal{T}(F)$ comme suit :

- `g.tset` est un pointeur vers le debut d'un tableau de taille `g.tset_sz`;
- `g.tset[0]` est initialisée à la valeur `{.label=NULL, .li=0, .ri=0}` (premier champ à NULL et les suivants à 0) de `tnode_t`, valeur qu'on appelle aussi *terme vide*;
- un terme est représenté par une valeur `t` de type `tnode_t` tel que `t.label` est une chaîne de caractères qui décrit le symbole racine du terme, et `t.li` (resp. `t.ri`) est la position du premier (resp. second) sous-terme direct dans le tableau `g.tset`.

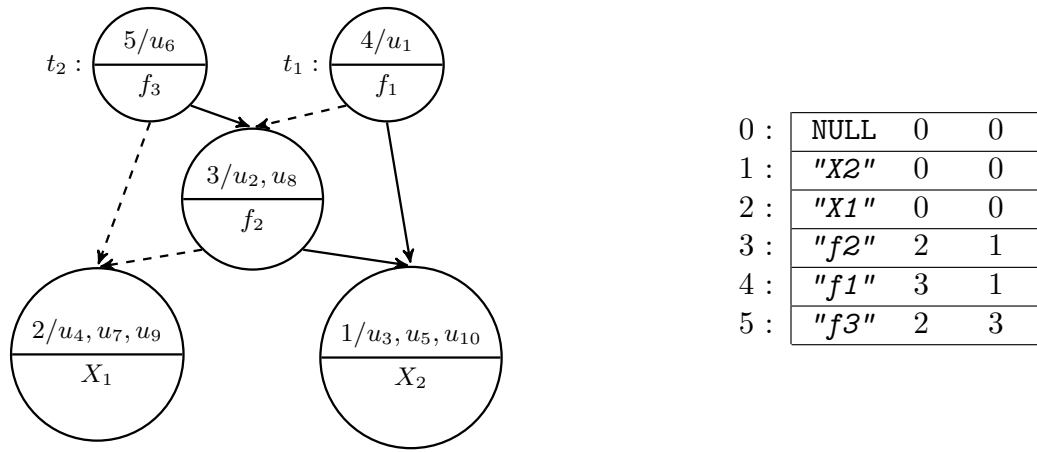
Un invariant du tableau `g.tset` est le suivant :

tout terme de $\mathcal{T}(F)$ est représenté par une unique position du tableau `g.tset`.

Cet invariant correspond à une factorisation des termes communs et donc à la transformation de l'ensemble des arbres correspondant aux termes en un graphe orienté acyclique.

Par exemple, considérons la formule $t_1 = t_2$ utilisant les termes représentés dans la figure 2. La partie droite de la figure 3 représente le tableau `g.tset` de l'ensemble des sous-termes de la formule (une valeur `tnode_t` par ligne) et le terme vide. La partie gauche de la figure 3 représente le tableau `g.tset` sous la forme d'un graphe orienté acyclique : les noeuds contiennent en partie haute la position du terme dans le tableau `g.tset` et, pour la lisibilité de la transformation, l'ensemble des identifiants u_i des noeuds correspondants dans les arbres de la figure 2. La partie basse d'un noeud donne le symbole racine.

La composante `g.pre` de l'e-graphe contient la relation prédécesseur dans le graphe dirigé acyclique représenté par `g.tset`. Cette relation est donnée sous la forme d'une matrice linéarisée en un tableau de `g.tset_sz2` valeurs dans l'ensemble $\{0, 1\}$. Si `g.pre[i*g.tset_sz + j]` vaut 1 alors le terme `g.tset[i]` a comme sous-terme direct le terme `g.tset[j]`. La valeur 0 dénote que `g.tset[j]` n'est pas sous-terme direct de `g.tset[i]`.

FIGURE 3 – Codage de $\mathcal{T}(t_1 = t_2)$ sous forme de graphe (à gauche) et de tableau (à droite)

La composante $\mathbf{g.uf}$ de l'e-graphe $\mathbf{g}$ code les classes d'équivalence définies par les contraintes d'égalités entre les termes. La définition du type `uf_t` est la même que celle donnée dans la partie III. Chaque terme est représenté dans l'ensemble des éléments de $\mathbf{g.uf}$ par la position du noeud racine du terme dans le tableau $\mathbf{g.tset}$. Par conséquent, un autre invariant de la structure d'e-graphe est le suivant :

$\mathbf{g.uf} \rightarrow \mathbf{nelem}$ est égal à $\mathbf{g.tset_sz}$.

Question 34. Écrire la fonction `bool term_is_diff(egraph_t *g, int tid1, int tid2)` qui teste si la contrainte $t_1 \neq t_2$ peut être satisfaite simultanément aux contraintes d'égalité représentées par $\mathbf{g}$. Les paramètres `tid1` et `tid2` sont les positions dans $\mathbf{g.tset}$ des noeuds racine représentant respectivement t_1 et t_2 . ┘

Congruence. Soient $F \in \mathcal{F}(L_4)$ et $\mathbf{g}$ un e-graphe tel que $\mathbf{g.tset}$ représente $\mathcal{T}(F)$, l'ensemble des sous-termes de F . On définit la relation d'équivalence $\equiv_{\mathbf{g}}$ sur $\mathcal{T}(F)$ par : $t \equiv_{\mathbf{g}} t'$ si et seulement si t et t' sont représentés par des positions de $\mathbf{g.tset}$ qui sont dans la même classe d'équivalence de $\mathbf{g.uf}$. On définit la relation $\mathcal{C}_{\mathbf{g}}$ sur $\mathcal{T}(F)$ comme suit :

*pour tout $f_i \in \mathbb{F}$ et pour tout quadruplet de termes $(t_1, t_2, t_3, t_4) \in \mathcal{T}(F)^4$,
si $t_1 \equiv_{\mathbf{g}} t_3$ et $t_2 \equiv_{\mathbf{g}} t_4$, alors $(f_i(t_1, t_2), f_i(t_3, t_4)) \in \mathcal{C}_{\mathbf{g}}$.*

Question 35. Écrire la fonction `bool term_is_congruent(egraph_t *g, int tid1, int tid2)` qui renvoie `true` si et seulement si les termes représentés aux positions `tid1` et `tid2` dans $\mathbf{g.tset}$ sont dans la relation $\mathcal{C}_{\mathbf{g}}$. Donner et justifier soigneusement la complexité de `term_is_congruent` en fonction de $\mathbf{g.tset_sz}$, si on suppose que le test d'égalité d'étiquettes (symboles de $\mathbb{X}$ ou $\mathbb{F}$) se fait en temps constant. ┘

Algorithme de décision. Soient $F \in \mathcal{F}(L_4)$ et $\mathbf{g}$ un e-graphe tel que $\mathbf{g}$ représente les termes et les contraintes d'égalité de F ainsi que deux termes t_1 et t_2 , pas nécessairement dans $\mathcal{T}(F)$. On cherche à modifier $\mathbf{g}$ pour qu'il représente $F \wedge (t_1 = t_2)$. On appelle cette opération `merge(g, t1, t2)` pour faire la différence avec `uf_union`. En effet, cette opération ne peut pas se limiter à appeler `uf_union` pour unir les classes d'équivalence des positions représentant les termes t_1 et t_2 . Elle doit en plus tester si $(t'_1, t'_2) \in \mathcal{C}_{\mathbf{g}}$ pour tous termes t'_1 et t'_2 qui incluent respectivement t_1 et t_2 comme sous-termes directs mais *ne sont pas déjà dans la même*

classe d'équivalence. Si $(t'_1, t'_2) \in \mathcal{C}_g$, alors t'_1 et t'_2 doivent être mis dans la même classe d'équivalence. Cette dernière relation est propagée aux termes qui contiennent t'_1 et t'_2 et ainsi de suite, tant qu'on trouve des couples de termes dans la relation $\mathcal{C}_g$ mais pas dans $\equiv_g$.

Question 36. Écrire une fonction récursive `void merge(egraph_t *g, int tid1, int tid2)` qui ajoute la contrainte d'égalité entre les termes aux positions `tid1` et `tid2` dans `g.tset`, selon le processus décrit ci-dessus. Si `tid1` et `tid2` sont déjà dans la même classe d'équivalence, `merge` termine immédiatement. ┘

Question 37. Donner le nombre maximum d'appels récursifs qu'un appel à `merge` peut engendrer en fonction de `g.tset_sz`. ┘

Question 38. On suppose que le graphe orienté acyclique représenté par `g.tset` contient a arêtes. Donner la complexité en temps de l'opération `merge` en fonction de `g.tset_sz` et a . ┘

_____ *Fin du sujet.* _____

Annexe : Preuves en déduction naturelle

Un séquent $\Gamma \vdash F$ est composé d'un ensemble de formules Γ et d'une formule F .

Le sous-ensemble suivant de règles de la déduction naturelle peut être utilisé dans la preuve demandée à la question 14.

Introduction	Elimination
$\overline{\Gamma, F \vdash F} \text{ ax}$	
$\overline{\Gamma \vdash \top} \top_I$	$\frac{\Gamma \vdash \perp}{\Gamma \vdash F} \perp_E$
$\frac{\Gamma \vdash F'_1 \quad \Gamma \vdash F'_2}{\Gamma \vdash F'_1 \wedge F'_2} \wedge_I$	$\frac{\Gamma \vdash F_1 \wedge F_2}{\Gamma \vdash F_1} \wedge^1_E \quad \frac{\Gamma \vdash F_1 \wedge F_2}{\Gamma \vdash F_2} \wedge^2_E$
$\frac{\Gamma, F' \vdash F}{\Gamma \vdash F' \rightarrow F} \rightarrow_I$	$\frac{\Gamma \vdash F' \rightarrow F \quad \Gamma \vdash F'}{\Gamma \vdash F} \rightarrow_E$
$\frac{\Gamma, F \vdash \perp}{\Gamma \vdash \neg F} \neg_I$	$\frac{\Gamma \vdash \neg F \quad \Gamma \vdash F}{\Gamma \vdash \perp} \neg_E$

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2024

**MARDI 16 AVRIL 2024
14h00 - 18h00**

**FILIERES MP-MPI - Epreuve n° 4
INFORMATIQUE A (XULSR)**

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

*Cette composition ne concerne qu'une partie des candidats de la
filière MP, les autres candidats effectuant simultanément la
composition de Physique et Sciences de l'Ingénieur.
Pour la filière MP, il y a donc deux enveloppes de Sujets pour
cette séance.*

Arbres équilibrés et géométrie algorithmique

Ce sujet traite de l'implémentation d'ensembles finis par des arbres équilibrés dits AVL et de leur application à un problème de géométrie algorithmique classique, la localisation d'un point dans le plan.

Ce sujet est conçu pour être traité linéairement. Les parties I et II introduisent les arbres AVL et leur construction et la partie III les utilise pour la localisation d'un point dans le plan. Pour traiter une question, on peut admettre le résultat de toute question précédente.

Langage OCaml. Ce sujet utilise notamment les tableaux d'OCaml. On peut créer un tableau avec la fonction `Array.make`. L'appel de `Array.make n x` crée un tableau de taille n dont toutes les cases contiennent la valeur x . Les cases d'un tableau sont numérotées à partir de 0. La fonction `Array.length` renvoie la taille d'un tableau. Pour un tableau `tab`, on accède à l'élément d'indice i avec `tab.(i)` et on le modifie avec `tab.(i) <- v`.

Dans le code OCaml fourni dans le sujet, on utilise la construction `assert false` pour signaler un point de code inatteignable, c'est-à-dire un cas de figure qui n'est pas censé se produire si les préconditions de la fonction sont respectées.

Pour les questions de programmation, il n'est pas demandé de justifier la correction du code donné en réponse.

Complexité. Par *complexité en temps* d'un algorithme A on entend le nombre d'opérations élémentaires (comparaison, addition, soustraction, multiplication, division, affectation, test, etc.) nécessaires à l'exécution de A dans le cas le pire. Par *complexité en espace* d'un algorithme A on entend l'espace mémoire minimal nécessaire à l'exécution de A dans le cas le pire. Lorsque la complexité dépend d'un ou plusieurs paramètres $\kappa_0, \dots, \kappa_{r-1}$, on dit que A a une complexité en $O(f(\kappa_0, \dots, \kappa_{r-1}))$ s'il existe une constante $C > 0$ telle que, pour toutes les valeurs de $\kappa_0, \dots, \kappa_{r-1}$ suffisamment grandes (c'est-à-dire plus grandes qu'un certain seuil), pour toute instance du problème de paramètres $\kappa_0, \dots, \kappa_{r-1}$, la complexité est au plus $C \cdot f(\kappa_0, \dots, \kappa_{r-1})$. De même, on dit que A a une complexité en $\Theta(f(\kappa_0, \dots, \kappa_{r-1}))$ s'il existe deux constantes $C_1 > 0$ et $C_2 > 0$ telles que, pour toutes les valeurs de $\kappa_0, \dots, \kappa_{r-1}$ suffisamment grandes, la complexité de A est au moins $C_1 \cdot f(\kappa_0, \dots, \kappa_{r-1})$ et au plus $C_2 \cdot f(\kappa_0, \dots, \kappa_{r-1})$.

Dans tout ce sujet, on note « $\log$ » le logarithme à base 2. On utilisera exclusivement ce logarithme.

Partie I. Préliminaires

On cherche à construire une structure de données pour représenter des sous-ensembles finis d'un ensemble U donné. L'ensemble U est supposé muni d'un ordre strict total noté $<$, c'est-à-dire que, pour tous $x, y \in U$, on a $x < y$ ou $x = y$ ou $y < x$. On va utiliser des arbres binaires de recherche équilibrés pour représenter des sous-ensembles de U .

Arbres binaires. Un *arbre binaire* est défini récursivement de la manière suivante : soit comme l'arbre vide, noté $\langle \rangle$, qui ne contient aucun nœud ; soit comme un *nœud*, noté $\langle \ell, x, r \rangle$ avec un sous-arbre gauche ℓ , un élément $x \in U$ à la racine et un sous-arbre droit r . On dessine un arbre tel que $\langle \langle \rangle, w, \langle \langle \rangle, x, \langle \rangle \rangle \rangle, y, \langle \langle \rangle, z, \langle \rangle \rangle \rangle$ de la manière suivante



avec la racine (ici y) en haut, reliée aux sous-arbres gauche et droit dessinés en-dessous. Les sous-arbres vides ne sont pas dessinés, mais les liaisons vers les sous-arbres vides le sont.

On note $n(t)$ le nombre de nœuds et $h(t)$ la hauteur d'un arbre binaire t , définis par

$$n(\langle \rangle) = h(\langle \rangle) = 0, \quad n(\langle \ell, x, r \rangle) = 1 + n(\ell) + n(r) \quad \text{et} \quad h(\langle \ell, x, r \rangle) = 1 + \max(h(\ell), h(r))$$

On notera ici que, pour des raisons purement techniques, la hauteur de l'arbre vide est fixée à 0 contrairement au programme qui la fixe à -1 . Ainsi, pour l'arbre A pris en exemple ci-dessus, on a $n(A) = 4$ et $h(A) = 3$.

Le *parcours infixe* d'un arbre binaire est une séquence d'éléments de U définie récursivement de la manière suivante :

- le parcours infixe de l'arbre vide $\langle \rangle$ est la séquence vide ;
- le parcours infixe d'un arbre $\langle \ell, x, r \rangle$ est la concaténation, dans cet ordre, du parcours infixe de l'arbre ℓ , de l'élément x et du parcours infixe de l'arbre r .

Ainsi, le parcours infixe de l'arbre précédent est la séquence w, x, y, z .

Un arbre binaire t est un *arbre binaire de recherche* si la séquence de ses éléments donnée par un parcours infixe est triée par ordre strictement croissant. En particulier, un élément n'apparaît qu'au plus une fois dans cette séquence. Tous les arbres binaires manipulés dans ce sujet seront toujours des arbres binaires de recherche.

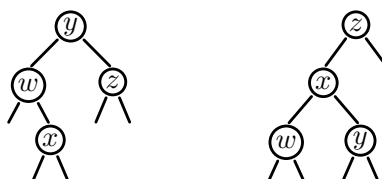
Question 1. Combien y a-t-il d'arbres binaires de recherche contenant exactement les trois éléments x, y et z , avec $x < y < z$? Les dessiner.

Question 2. Montrer qu'un arbre t est un arbre binaire de recherche si et seulement si t est l'arbre vide ou si t est un arbre $\langle \ell, x, r \rangle$ avec ℓ et r des arbres binaires de recherche où tous les éléments de ℓ sont strictement inférieurs à x et tous les éléments de r sont strictement supérieurs à x .

Arbres AVL. Pour équilibrer nos arbres binaires de recherche, on va imposer une condition supplémentaire : pour tout nœud $\langle \ell, x, r \rangle$, on a

$$|h(\ell) - h(r)| \leq 1 \tag{1}$$

De tels arbres sont dits AVL (du nom de leurs auteurs, Adelson-Velsky et Landis). Ainsi, dans les deux arbres ci-dessous,



celui de gauche est un AVL mais celui de droite n'en est pas un (en supposant $w < x < y < z$ par ailleurs).

Question 3. Combien y a-t-il d'arbres AVL contenant exactement les trois éléments x , y et z , avec $x < y < z$? Les dessiner. Même question avec les quatre éléments $w < x < y < z$. Justifier les réponses.

Question 4. Montrer que pour tout AVL t , on a $h(t) \leq 2 \log(n(t) + 1)$. Indication : on pourra considérer N_h , le nombre minimal de nœuds d'un AVL de hauteur h , et minorer cette quantité en fonction de h .

Mise en œuvre en OCaml. On suppose que les éléments de U sont représentés par un type OCaml `elt`, qui n'est pas précisé pour l'instant. Pour deux éléments x et y donnés, on peut tester si $x = y$ avec `eq x y` et tester si $x < y$ avec `lt x y`.

```
val eq: elt -> elt -> bool
val lt: elt -> elt -> bool
```

Un AVL est représenté par une valeur du type OCaml suivant :

```
type tree = E | N of int * tree * elt * tree
```

Le constructeur `E` représente l'arbre binaire vide $\langle \rangle$. Une valeur `N( $z, \ell, x, r$ )` représente l'arbre binaire non vide $\langle \ell, x, r \rangle$, avec l'*invariant* que la valeur z du premier champ est toujours égale à la hauteur de l'arbre, i.e. $z = h(\langle \ell, x, r \rangle)$. On se donne une fonction `height` pour obtenir la hauteur d'un arbre :

```
let height t = match t with E -> 0 | N(h, _, _, _) -> h
```

On note que cette fonction s'évalue en temps constant. Pour maintenir l'invariant, on se donne une fonction `node` pour construire un nouveau nœud :

```
let node l x r = N(1 + max (height l) (height r), l, x, r)
```

On note que cette fonction s'évalue également en temps constant. Dans tout ce qui suit, on utilisera toujours la fonction `node` pour construire un nœud et jamais le constructeur `N` directement. On utilisera le constructeur `N` uniquement dans les motifs de filtrage.

Question 5. Écrire une fonction `mem: elt -> tree -> bool` qui prend en arguments un élément x et un arbre AVL t et qui détermine si x apparaît dans t . La complexité doit être en $O(\log n(t))$ et on demande de la justifier.

Partie II. Construire des arbres AVL

Maintenir la propriété d'arbre AVL pendant la construction des arbres binaires de recherche demande un certain effort. On propose ici de concentrer cet effort dans une unique fonction, `join`, dont le code est donné figure 1. La fonction `join` reçoit en paramètres deux arbres AVL l et r et un élément x . Elle suppose que les éléments de l (resp. r) sont strictement plus petits (resp. plus grands) que x . Elle renvoie un arbre AVL contenant l'union des éléments de l , de r et du singleton $\{x\}$.

Si les hauteurs de l et r satisfont déjà à la propriété d'AVL, alors il suffit d'appeler `node` (ligne 4). Sinon, on appelle `join_right` (resp. `join_left`) selon que le déséquilibre vient de l (ligne 2) ou de r (ligne 3). La fonction `join_right` procède récursivement, en descendant le long de la branche droite de l'arbre l (lignes 8–11) jusqu'à trouver un sous-arbre de hauteur

acceptable (lignes 4–7). Si besoin, des rotations sont effectuées localement avec les deux fonctions `rotate_left` et `rotate_right` (en haut de la figure 1). Ces deux fonctions implémentent la notion usuelle de rotation dans les arbres binaires de recherche, illustrée juste en-dessous de leur code. La fonction `join_left` est symétrique et son code est omis. Dans les raisonnements sur la fonction `join`, on pourra se contenter de considérer seulement le cas où `join_right` est appelée, en considérant l'autre cas symétrique.

On prendra le temps de bien lire et de bien comprendre le code de la fonction `join`. L'objectif des questions suivantes est de se persuader que `join` fonctionne correctement, ce qui est plus subtile qu'il n'y paraît.

Question 6. Montrer que tout appel à `join`, avec deux arguments `l` et `r` qui sont des AVL, termine et ne peut pas « planter », au sens où la ligne 3 de `rotate_left`, la ligne 3 de `rotate_right` et la ligne 12 de `join_right` ne sont jamais atteintes.

Question 7. Montrer que, lorsque la fonction `join_right` est appelée avec un AVL `l` de hauteur h et un AVL `r` de hauteur $\leq h - 2$, alors elle renvoie

- soit un arbre de hauteur h ;
- soit un arbre de hauteur $h + 1$ avec un sous-arbre gauche de hauteur $h - 1$ et un sous-arbre droit de hauteur h .

Indication : pour chaque cas qu'on choisit de distinguer dans la preuve, on pourra se contenter d'un schéma donnant la forme de l'arbre renvoyé annoté avec les hauteurs pertinentes.

Question 8. Montrer que la fonction `join`, appliquée à deux AVL `l` et `r`, renvoie bien un arbre AVL dont la hauteur est au plus $1 + \max(h(l), h(r))$.

Question 9. Montrer que la fonction `join` a une complexité en $O(|h(l) - h(r)|)$.

Insertion d'un élément. Pour insérer un nouvel élément dans un arbre AVL, on se donne la fonction `insert` (voir figure 2). Elle utilise une fonction auxiliaire, `split`, dont le code est également donné.

Question 10. Expliquer ce que fait la fonction `split` appliquée à un AVL `t` et un élément `y`.

Question 11. Illustrer les étapes de l'insertion de l'élément `c` dans un AVL contenant les éléments `a`, `b` et `d`, avec $a < b < c < d$.

Question 12. Montrer que la fonction `split`, lorsqu'elle est appliquée à un AVL `t` et un élément `y`, renvoie deux arbres AVL dont la hauteur n'excède pas celle de `t`.

Question 13. Montrer que la fonction `insert`, lorsqu'elle est appliquée à un élément `x` et un arbre AVL `t`, renvoie bien un arbre AVL et a une complexité en $O(\log n(t))$.

```

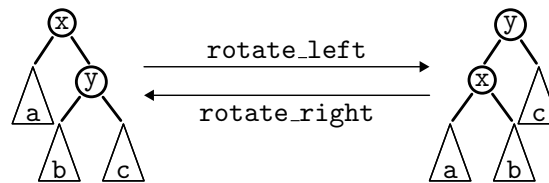
1 let rotate_left t = match t with
2   | N(_, a, x, N(_, b, y, c)) -> node (node a x b) y c
3   | _ -> assert false

```

```

1 let rotate_right t = match t with
2   | N(_, N(_, a, x, b), y, c) -> node a x (node b y c)
3   | _ -> assert false

```



```

1 let rec join_right (l: tree) (x: elt) (r: tree) : tree =
2   match l with
3   | N(_, ll, lx, lr) ->
4     if height lr <= height r + 1 then
5       let t = node lr x r in
6       if height t <= height ll + 1 then node ll lx t
7       else rotate_left (node ll lx (rotate_right t))
8     else
9       let t = join_right lr x r in
10      let t' = node ll lx t in
11      if height t <= height ll + 1 then t' else rotate_left t'
12 | E -> assert false

```

```

1 let rec join_left (l: tree) (x: elt) (r: tree) : tree =
2   ...

```

```

1 let join (l: tree) (x: elt) (r: tree) : tree =
2   if height l > height r + 1 then join_right l x r
3   else if height r > height l + 1 then join_left l x r
4   else node l x r

```

FIGURE 1 – Fonction join sur les AVL.

```

1 let rec split (t: tree) (y: elt) : tree * bool * tree =
2   match t with
3     | E -> E, false, E
4     | N(_, l, x, r) ->
5       if eq y x then l, true, r
6       else if lt y x then let ll, b, lr = split l y in ll, b, join lr x r
7       else let rl, b, rr = split r y in join l x rl, b, rr

1 let insert (x: elt) (t: tree) : tree =
2   let l, _, r = split t x in
3   join l x r

```

FIGURE 2 – Fonctions split et insert sur les AVL.

Question 14. Soit n un entier positif ou nul. On construit un tableau a de n arbres AVL de la manière suivante :

```

let a = Array.make n E
let () = for i = 1 to n - 1 do a.(i) <- insert (i-1) a.(i-1) done

```

Calculer le nombre total $f(n)$ d'entiers (possiblement répétés) stockés dans l'ensemble des arbres du tableau a . Montrer que la complexité en espace de la construction de a est en $O(g(n))$ pour une fonction g telle que $g(n) = o(f(n))$. Expliquer pourquoi il n'y a pas de contradiction.

Supprimer un élément. On souhaite maintenant écrire une fonction `remove` pour supprimer un élément dans un arbre AVL, qui ait une complexité logarithmique comme la fonction `insert`.

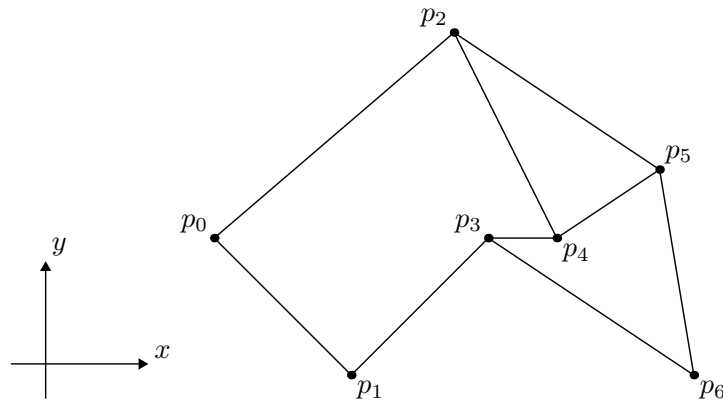
Question 15. Écrire une fonction `split_last: tree -> tree * elt` qui prend en argument un arbre AVL t , supposé non vide, et qui renvoie une paire (u, x) où x est le plus grand élément de t et u un arbre AVL contenant les éléments de t autres que x . La complexité doit être $O(h(t))$. On ne demande pas de la justifier.

Question 16. Écrire une fonction `join2: tree -> tree -> tree` qui prend en arguments deux arbres AVL ℓ et r , en supposant les éléments de ℓ strictement plus petits que ceux de r , et qui renvoie un arbre AVL t contenant l'union des éléments de ℓ et de r . La complexité doit être $O(h(\ell) + h(r))$. On ne demande pas de la justifier.

Question 17. Écrire une fonction `remove: elt -> tree -> tree` qui prend en arguments un élément x et un AVL t , et qui renvoie un arbre AVL contenant les éléments de t privés de x . (Si x n'apparaît pas dans t , le résultat contient les mêmes éléments que t .) La complexité doit être $O(\log n(t))$. On ne demande pas de la justifier.

Partie III. Application : localisation d'un point dans le plan

On considère une subdivision du plan définie par un ensemble fini de polygones. Ces polygones peuvent partager des sommets et des arêtes. Les arêtes ne se croisent pas. Voici un exemple avec 7 points et trois polygones :



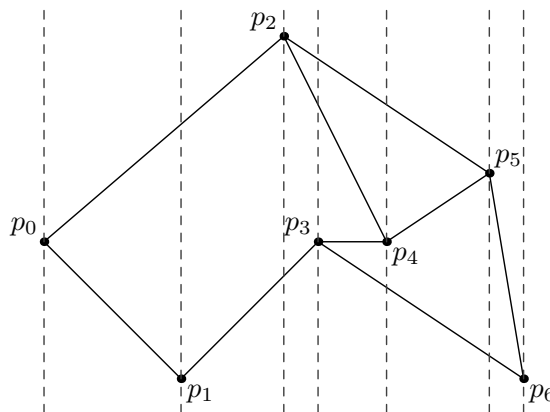
Le plan est ici divisé en quatre zones : l'intérieur de chaque polygone et la zone extérieure. Notre objectif est de construire une structure de données pour représenter cette subdivision, afin de répondre ensuite efficacement à la question « étant donné un point, dans quelle zone se trouve-t-il ? ». On s'intéresse notamment à la complexité de cette structure (temps et espace pour sa construction) et à la complexité de son utilisation ensuite. Ces complexités seront calculées en fonction du nombre total de sommets, noté N dans la suite. On admet que le nombre d'arêtes est en $O(N)$.

Notations. Pour un point p , on note $x(p)$ son abscisse et $y(p)$ son ordonnée. Les N sommets sont notés $p_0, p_1, \dots, p_{N-1}$ et ordonnés de la gauche vers la droite. On suppose qu'il n'y a pas deux sommets sur la même abscisse, c'est-à-dire

$$x(p_0) < x(p_1) < \dots < x(p_{N-1})$$

Une arête e est un couple (p_i, p_j) avec $0 \leq i < j < N$. On note $start(e)$ son extrémité gauche, c'est-à-dire le point p_i , et $end(e)$ son extrémité droite, c'est-à-dire le point p_j . On note $xmin(e)$ l'abscisse de p_i et $xmax(e)$ l'abscisse de p_j . L'arête e s'étend donc de l'abscisse $xmin(e)$ à l'abscisse $xmax(e)$.

Approche. Notre approche consiste à diviser le plan en $N + 1$ tranches verticales, délimitées par les abscisses des N points. Sur notre exemple, on a donc 8 tranches :



La tranche 0 correspond aux abscisses $] -\infty, x(p_0)]$, la tranche 1 aux abscisses $]x(p_0), x(p_1)]$, la tranche 2 aux abscisses $]x(p_1), x(p_2)]$, etc., et la tranche N aux abscisses $]x(p_{N-1}), +\infty[$. Chaque tranche est traversée par un certain nombre d'arêtes. On note que, dans chaque tranche, les segments d'arêtes qui traversent cette tranche peuvent être ordonnés verticalement (car, on le rappelle, les arêtes ne se croisent pas). Ainsi, dans la tranche 3 ci-dessus, on a, de bas en haut, l'arête (p_1, p_3) , l'arête (p_2, p_4) puis enfin l'arête (p_2, p_5) .

L'idée est alors de trier les tranches selon les abscisses, puis, dans chaque tranche, de trier les arêtes la traversant selon les ordonnées. Ainsi, étant un point du plan à localiser, il suffira d'une recherche dichotomique pour déterminer la tranche contenant le point puis d'une seconde recherche dichotomique pour déterminer entre quelles arêtes se trouve le point (et, par conséquent, dans quel polygone se trouve le point, le cas échéant).

Question 18. On note T_i le nombre d'arêtes qui traversent la tranche i . (Dans l'exemple ci-dessus, on a $T_0 = 0$, $T_1 = T_2 = 2$, $T_3 = 3$, etc.) Montrer que la quantité $\sum T_i$ peut être en $\Theta(N^2)$.

Mise en œuvre en OCaml. On utilise des nombres flottants pour les coordonnées des points. On introduit les deux types suivants pour les points et les arêtes, respectivement.

```
type point = float * float
type edge = point * point (* avec x1 < x2 *)
```

On se donne deux fonctions pour récupérer l'abscisse gauche et l'abscisse droite d'une arête :

```
let xmin ((x, _), _) = x
let xmax (_, (x, _)) = x
```

Pour représenter chaque tranche, on va utiliser un arbre AVL dont les éléments sont des arêtes, c'est-à-dire le type `tree` introduit plus haut dans le sujet avec

```
type elt = edge
```

Une tranche est alors représentée par le type suivant,

```
type slab = { xleft: float; xright: float; tree: tree }
```

où `xleft` est l'abscisse du bord gauche de la tranche, `xright` l'abscisse du bord droit et `tree` l'arbre AVL contenant les arêtes qui traversent cette tranche, ordonnées de bas en haut.

Question 19. Écrire une fonction `lt: edge -> edge -> bool` qui prend en arguments deux arêtes e_1 et e_2 , supposées traversant une même tranche, et qui détermine si e_1 est strictement en dessous de e_2 .

Construction des tranches. On représente l'ensemble des tranches par un tableau `slabs`: `slab` array de taille $N + 1$. Il est trié selon les abscisses, c'est-à-dire

```
-∞ = slabs.(0).xleft < slabs.(0).xright = x(p0) = slabs.(1).xleft
                        < slabs.(1).xright = x(p1) = slabs.(2).xleft
                        < ...
                        < slabs.(N).xright = +∞
```

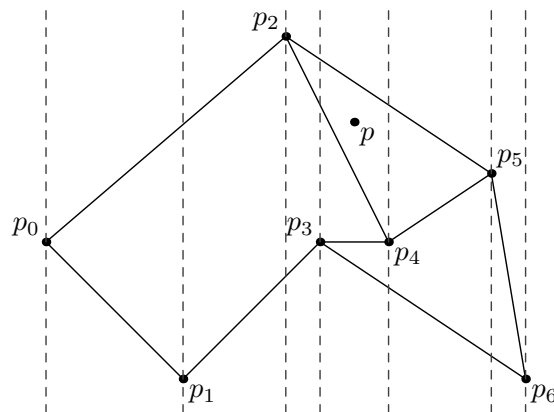
(Le type `float` inclut des valeurs `neg_infinity` pour $-\infty$ et `infinity` pour $+\infty$.) On construit les arbres AVL de chaque tranche de la gauche vers la droite, avec l'algorithme suivant :

- L'arbre de la tranche 0 est vide.
- Pour construire l'arbre de la tranche i , pour $1 \leq i \leq N$,
 - on retire les arêtes qui se terminent au point p_{i-1} ;
 - on ajoute les arêtes qui démarrent au point p_{i-1} .

En particulier, l'arbre de la dernière tranche est vide.

Question 20. Justifier que l'on peut construire, à partir de la liste des arêtes, le tableau `slabs` en temps et en espace $O(N \log N)$. On ne demande pas d'écrire le code, mais on demande d'être précis quant aux structures de données et algorithmes utilisés et on demande d'être convaincant quant à la complexité. On ne demande pas de vérifier que tous les sommets ont des abscisses différentes.

Localisation d'un point. Enfin, on peut utiliser le tableau `slabs` pour localiser un point p donné. On fait l'hypothèse que ce point n'est sur aucune arête, et est donc en particulier différent des N points p_i . Si on prend l'exemple suivant



alors le point p est d'abord localisé dans la tranche 4 puis, dans cette tranche, localisé entre les arêtes (p_2, p_4) et (p_2, p_5) .

Question 21. Écrire une fonction `find_slab: slab array -> float -> slab` qui prend en arguments les tranches et l'abscisse du point p recherché et renvoie la tranche dans laquelle se trouve le point p . La complexité doit être $O(\log N)$. On ne demande pas de la justifier.

Question 22. Écrire une fonction `find: slab array -> point -> answer` qui prend en arguments les tranches et un point p et renvoie la zone dans laquelle le point p se trouve, sous la forme d'une valeur du type

type `answer` = Outside | Between of edge * edge

où `Outside` signifie que le point p est à l'extérieur de tout polygone et `Between( $e_1, e_2$ )` signifie que le point p se trouve à l'intérieur d'un polygone, entre les arêtes e_1 et e_2 . La complexité doit être $O(\log N)$ et on demande de la justifier.

* *
*

ECOLES NORMALES SUPERIEURES

CONCOURS D'ADMISSION 2024

**VENDREDI 19 AVRIL 2024
14h00 - 18h00**

FILIERES MP et MPI

Epreuve n° 10

INFO-FONDAMENTALE (ULSR)

Durée : 4 heures

***L'utilisation des calculatrices n'est pas
autorisée pour cette épreuve***

Couverture minimale d'un graphe

Le sujet comporte 12 pages, numérotées de 1 à 12.

Début de l'épreuve.

Ce sujet est consacré au calcul d'un paramètre de graphe appelé ***couverture minimale***, défini comme étant le plus petit nombre de sommets d'un ***ensemble couvrant***. On étudiera plusieurs façon de le calculer, ce qui nous permettra de faire un tour d'horizon de quelques techniques algorithmiques modernes.

Ce sujet est constitué de 6 parties qui peuvent être traitées relativement indépendamment ; cependant, les concepts introduits dans une partie peuvent être réutilisés dans des parties ultérieures.

- Dans la première partie (page 3) on découvre progressivement certaines propriétés des couvertures minimales, leurs valeurs sur quelques familles de graphes simples et leurs liens avec d'autres paramètres classiques de la théorie des graphes.
 - Dans la deuxième partie (page 4) on développe deux algorithmes efficaces qui calculent un ensemble couvrant minimum d'un arbre.
 - Dans la troisième partie (page 5) on étudie deux heuristiques naturelles calculant un ensemble couvrant s'avérant loin d'être minimum, pour finalement découvrir un algorithme simple calculant une approximation de la couverture minimale.
 - Dans la quatrième partie (page 7) on découvre la complexité paramétrée.
 - Dans la cinquième partie (page 9) on continue notre exploration de la complexité paramétrée en introduisant la notion de noyau.
 - Dans la sixième partie (page 10) on découvre la puissance de l'optimisation linéaire, permettant une autre approximation de la couverture minimale ainsi qu'un noyau plus petit que celui de la partie V.
-

Notations et définitions

- Un **graphe** (non-orienté) G est la donnée d'un ensemble fini $V(G)$ de **sommets** et d'un ensemble $E(G)$ dont les éléments, appelés **arêtes**, sont des paires de sommets de V . On notera le fait que deux sommets u et v sont reliés par une arête par $\{u, v\} \in E(G)$. On dit que u et v sont les **extrémités** de l'arête $\{u, v\}$ et que u et v sont **adjacents** dans G .
- Une conséquence de cette définition est que tous les graphes considérés sont **simples**, c'est à dire sans boucle ($\{u, u\} = \{u\}$ n'est pas une arête), et avec au plus une arête entre deux sommets.
- Soit v un sommet d'un graphe G . Le **voisinage** $N(v)$ de v est l'ensemble des sommets adjacents à v , c'est-à-dire $\{u \in V(G) \mid \{u, v\} \in E(G)\}$. Le **degré** $d(v)$ est le cardinal de $N(v)$.
- Un **chemin** $P = (v_1, \dots, v_n)$ dans un graphe G est une liste de sommets deux à deux distincts tels que pour tout $1 \leq i < n$, v_i et v_{i+1} sont adjacents dans G .
- Un **ensemble couvrant** d'un graphe G est un ensemble de sommets X tel que toute arête de G a au moins une extrémité dans X . Le nombre de sommets d'un **ensemble couvrant minimum** (c'est-à-dire, un ensemble couvrant minimisant le nombre de sommets) de G est appelé **couverture minimale** de G et noté $\tau(G)$.
- Soit G un graphe et X un ensemble de sommets de G . On dénote par $G - X$ le graphe sur les sommets $V(G) \setminus X$ avec comme arêtes $\{\{u, v\} \in E(G) \mid u \notin X \text{ et } v \notin X\}$.
- Par **complexité** (en temps) d'un algorithme ALG , on entend le nombre d'opérations élémentaires nécessaires à l'exécution de ALG dans le pire des cas.
- Lorsque la complexité en temps dépend d'un ou plusieurs paramètres $x_1, \dots, x_p$, on dit qu'elle est en $O(f(x_1, \dots, x_p))$ s'il existe une constante $C > 0$ telle que, pour toutes les valeurs de $x_1, \dots, x_p$, suffisamment grandes (c'est-à-dire, plus grandes qu'un certain seuil), la complexité est au plus $C \times f(x_1, \dots, x_p)$. La complexité est dite **linéaire** si $f(x_1, \dots, x_p) = x_1 + \dots + x_p$.
- Dans tout l'énoncé (sauf dans la partie II où on travaille sur des arbres et dans la partie III où on ne s'intéresse pas à la complexité précise des algorithmes), on fera l'hypothèse que les graphes sont encodés par leur matrice d'adjacence.

Partie I

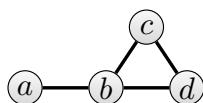
Un graphe est **complet** si tous les sommets sont deux à deux adjacents. On dénote par K_n le graphe complet sur n sommets $\{1, \dots, n\}$.

Un **ensemble indépendant** d'un graphe est un ensemble de sommets du graphe dont les sommets sont deux à deux non-adjacents. On note $\alpha(G)$ le nombre maximum de sommets d'un ensemble indépendant de G . Un **couplage** est un ensemble d'arêtes deux à deux disjointes, c'est à dire ne partageant aucun sommet. Le nombre maximum d'arêtes dans un couplage de G est noté $\mu(G)$.

Un graphe est **biparti** si ses sommets peuvent être partitionnés en deux ensembles indépendants. Il est **biparti complet** s'il admet une partition en deux ensembles indépendants telle que chaque sommet d'une partie est adjacent à chaque sommet de l'autre partie. Étant donné deux entiers naturels a et b , on dénote par $K_{a,b}$ le graphe biparti complet dont les parties de la partition en deux ensembles indépendants ont respectivement a et b sommets (disons, les sommets étant numérotés de 1 à a dans la première partie et de $a+1$ à $a+b$ dans la deuxième).

On dénote par P_n le **graphe chemin** sur $n \geq 1$ sommets, c'est-à-dire le graphe dont les sommets sont $\{1, \dots, n\}$ et les arêtes $\{\{1, 2\}, \{2, 3\}, \dots, \{n-1, n\}\}$. Un **graphe cycle** est le graphe obtenu en ajoutant une arête entre les deux extrémités d'un graphe chemin. On dénote par C_n le graphe cycle sur $n \geq 2$ sommets.

Question I.1. Donner les valeurs de $\tau(G)$, $\alpha(G)$ et $\mu(G)$ pour le graphe G représenté ci-dessous. Justifier.



Question I.2. Donner la valeur de $\tau(K_n)$, $\tau(P_n)$, $\tau(C_n)$ et $\tau(K_{a,b})$ pour tous entiers naturels non nuls a, b, n . Justifier soigneusement les réponses.

Question I.3. Montrer que pour tout graphe G , $\tau(G) = |V(G)| - \alpha(G)$.

Question I.4. Montrer que pour tout graphe G , $\tau(G) \geq \mu(G)$.

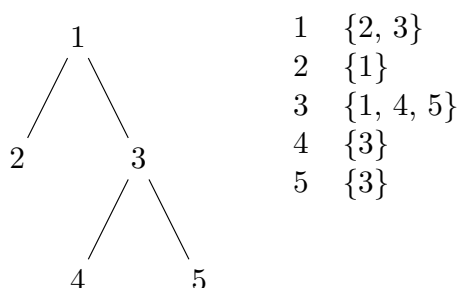
Aucun algorithme polynomial n'est connu pour calculer un ensemble couvrant minimum d'un graphe. Dans la question suivante, on demande un algorithme exponentiel.

Question I.5. Donner une description à haut niveau (sans détailler les lignes de pseudo-code) d'un algorithme calculant un ensemble couvrant minimum d'un graphe G en temps $O(2^{|V(G)|} \cdot |V(G)|^2)$.

Partie II

Un **arbre** est un graphe connexe sans cycle. Un sommet de degré ≤ 1 d'un arbre est appelé une **feuille**. Dans cette partie, on va développer deux algorithmes efficaces pour calculer un ensemble couvrant minimum d'un arbre.

Dans cette partie, contrairement au reste du sujet, les arbres ne sont pas représentés par leur matrice d'adjacence mais par un tableau T dont les éléments sont des ensembles de sommets : pour chaque sommet $1 \leq i \leq n$ (où n est le nombre de sommets de l'arbre), $T[i]$ est une représentation de l'ensemble des sommets adjacents à i . On supposera que la structure de données utilisée pour représenter l'ensemble $T[i]$ permet en temps $O(1)$ les opérations suivantes : renvoyer le nombre d'éléments de l'ensemble ; rechercher si un élément donné appartient à l'ensemble ; ajouter un nouvel élément à l'ensemble ; retirer un élément existant de l'ensemble. Par exemple, voici un arbre sur les sommets $\{1, \dots, 5\}$ et sa représentation sous forme de tableau dont les éléments sont les ensembles de sommets adjacents :



Question II.1. Soient T un arbre et $\{u, v\}$ une arête de T telle que v est une feuille de T . Montrer qu'il existe un ensemble couvrant minimum de T qui contient le sommet u , mais pas v .

Question II.2. En utilisant la question précédente, proposer un algorithme (en pseudo-code) qui reçoit un arbre T en entrée, et renvoie un ensemble couvrant minimum de T , en enlevant progressivement des arêtes à l'arbre. La complexité de cet algorithme doit être linéaire en le nombre n de sommets de l'arbre. Justifier de la complexité de l'algorithme.

Question II.3. Proposer un autre algorithme (en pseudo-code) qui renvoie le nombre de sommets d'un ensemble couvrant minimum d'un arbre, cette fois basé sur la programmation dynamique. On pourra commencer par **enraciner** l'arbre, c'est-à-dire choisir un sommet arbitraire qui jouera le rôle de racine de l'arbre. Pour chaque sommet $v \in V(G)$, on note T_v le sous-arbre de T enraciné en v ; pour calculer le résultat final, l'algorithme devra d'abord calculer en programmation dynamique pour chaque $v \in V(G)$ les trois valeurs suivantes :

- $D_{\in}(v)$ = nombre de sommets d'un ensemble couvrant minimum de T_v contenant v ;
- $D_{\notin}(v)$ = nombre de sommets d'un ensemble couvrant minimum de T_v ne contenant pas v ;
- $D(v)$ = nombre de sommets d'un ensemble couvrant minimum de T_v .

On ne demande pas la preuve de la correction, mais il faut donner et prouver sa complexité en fonction du nombre n de sommets de l'arbre.

Partie III

Dans cette partie, plutôt que d'essayer de calculer un ensemble couvrant minimum, on va chercher à en calculer un dont le cardinal est proche du minimum.

Formalisons cette idée. Soit ALG un algorithme prenant un graphe en entrée et renvoyant un ensemble couvrant (pas forcément minimum). On note $ALG(G)$ le nombre de sommets de l'ensemble couvrant renvoyé par ALG appliqué à G . On note $OPT(G)$ le nombre de sommets d'un ensemble couvrant minimum de G . On dit que ALG est une *c -approximation* (pour un nombre réel $c \geq 1$) si pour tout graphe G , on a :

$$\frac{ALG(G)}{OPT(G)} \leq c.$$

Noter que les algorithmes que nous allons considérer n'ont pas une façon unique de s'exécuter sur un graphe donné. Typiquement, à la ligne 3 de l'algorithme HEURISTIQUE ci-dessous, il peut y avoir beaucoup de façon différentes de choisir le sommet v . Dans ce cas, on considère que $HEURISTIQUE(G)$ est le résultat de la pire exécution possible, c'est à dire celle renvoyant le plus grand ensemble couvrant parmi tous les ensembles couvrants qu'elle peut potentiellement renvoyer.

On considère un premier algorithme calculant un ensemble couvrant :

Algorithme 1 HEURISTIQUE(G)

```

1:  $S \leftarrow \emptyset$ 
2: while  $G$  a des arêtes do
3:   Choisir un sommet  $v$  de  $G$  tel que  $d(v) \geq 1$ 
4:    $S \leftarrow S \cup \{v\}$ 
5:    $G \leftarrow G - \{v\}$ 
6: return  $S$ 
```

Question III.1. Montrer que l'ensemble S calculé par HEURISTIQUE est bien un ensemble couvrant.

Question III.2. Montrer que, pour tout entier $n \geq 2$, il existe un graphe G à n sommets tel que

$$\frac{HEURISTIQUE(G)}{OPT(G)} \geq n - 1.$$

L'heuristique ci-dessus pouvant donner des résultats catastrophiques, on propose de l'améliorer en choisissant le sommet de degré maximum, donc celui couvrant le plus d'arêtes.

Algorithme 2 GLOUTON(G)

```

1:  $S \leftarrow \emptyset$ 
2: while  $G$  a des arêtes do
3:   Choisir un sommet  $v$  de degré maximum dans  $G$ 
4:    $S \leftarrow S \cup \{v\}$ 
5:    $G \leftarrow G - \{v\}$ 
6: return  $S$ 
```

GLOUTON renvoie un ensemble couvrant pour les mêmes raisons que HEURISTIQUE.

On va définir une série de graphes bipartis G_n avec bipartition A_n et B_n comme suit. A_n est un ensemble de n sommets. B_n est partitionné en n sous-ensembles $B_n^1, \dots, B_n^n$ ayant les propriétés suivantes pour $k = 1, \dots, n$:

- (i) $|B_n^k| = \lfloor \frac{n}{k} \rfloor$,
- (ii) chaque sommet de B_n^k a exactement k voisins dans A_n ,
- (iii) deux sommets de B_n^k n'ont pas de voisins communs dans A_n .

Question III.3. Dessiner (une réalisation possible de) G_4 et donner les valeurs de $\text{OPT}(G_4)$ et de $\text{GLOUTON}(G_4)$. On ne demande pas de justifier.

Question III.4. En utilisant la construction ci-dessus, montrer que pour toute constante c , GLOUTON n'est pas une c -approximation.

Ainsi, GLOUTON ne donne pas non plus une bonne approximation. On va maintenant considérer l'algorithme suivant.

Algorithme 3 APPROX(G)

```

1:  $S \leftarrow \emptyset$ 
2: while  $G$  a des arêtes do
3:   Choisir une arête  $\{u, v\}$  de  $G$ 
4:    $S \leftarrow (S \cup \{u\}) \cup \{v\}$ 
5:    $G \leftarrow (G - \{u\}) - \{v\}$ 
6: return  $S$ 
```

Question III.5. Montrer que APPROX renvoie un ensemble couvrant, en s'appuyant sur la question I.3.

Question III.6. Montrer que APPROX est une 2-approximation.

Question III.7. Donner une suite infinie de graphes justifiant que, pour tout $c < 2$, APPROX n'est pas une c -approximation.

Partie IV

Dans cette partie, on cherche à répondre à la question suivante : *Étant donné un graphe G et un entier k , est-ce que $\tau(G) \leq k$?* On rappelle que G est représenté par sa matrice d'adjacence.

Le but est de trouver un algorithme qui, étant donné un graphe G et un entier k , décide si $\tau(G) \leq k$ en temps $O(f(k) \times |V(G)|^c)$ où f est une fonction (arbitraire, potentiellement exponentielle en son argument) et $c \geq 0$ une constante. Un algorithme avec une telle complexité est dit **FPT** pour *fixed-parameter tractable* ou *tractable à paramètre fixé*.

On va commencer modestement. Noter que l'algorithme demandé à la question ci-dessous n'est pas FPT.

Question IV.1. Donner une description à haut niveau (sans détailler les lignes de pseudo-code) d'un algorithme qui, étant donné un graphe G et un entier k , décide si $\tau(G) \leq k$ en temps $O(|V(G)|^{k+2})$.

Le reste de cette partie est dédié à la conception de notre premier algorithme FPT.

Question IV.2. Soit G un graphe et $\{u, v\} \in E(G)$. Montrer l'équivalence suivante :

$$\tau(G) \leq k \iff \tau(G - \{u\}) \leq k - 1 \text{ ou } \tau(G - \{v\}) \leq k - 1.$$

Question IV.3. En s'aidant de la question précédente, donner un algorithme récursif (en pseudo-code) qui, étant donné un graphe G et un entier k , décide si $\tau(G) \leq k$ en temps $O(2^k \times |V(G)|^2)$. Justifier la correction et la complexité de l'algorithme. On rappelle qu'avec la représentation en matrice d'adjacence, il est possible de supprimer l'ensemble des arêtes dont un sommet u est une extrémité en $O(|V(G)|)$.

On va maintenant légèrement modifier l'algorithme de la question précédente pour améliorer sa complexité.

Question IV.4. Donner une description précise des graphes de degré maximum 2. En déduire une description à haut niveau (sans détailler les lignes de pseudo-code) d'un algorithme en $O(|V(G)|)$ pour décider si un graphe de degré maximum 2 a un ensemble couvrant avec au plus k sommets.

On rappelle que, étant donné un sommet v , $N(v)$ est l'ensemble des voisins de v et on définit $N[v] = N(v) \cup \{v\}$, le *voisinage fermé* de v .

Question IV.5. Soit $v \in V$. Montrer que si $d(v) \geq 3$, alors

$$\tau(G) \leq k \iff \tau(G - \{v\}) \leq k - 1 \text{ ou } \tau(G - N[v]) \leq k - d(v).$$

Question IV.6. En s'aidant de la question précédente, donner un algorithme récursif (en pseudo-code) qui décide si $\tau(G) \leq k$ en temps $O(1.4656^k \times |V(G)|^2)$. Justifier la correction et la complexité de l'algorithme. On utilisera le fait que la fonction $f(k)$ définie récursivement comme suit :

$$f(i) = \begin{cases} f(i-1) + f(i-3) & \text{si } i \geq 3 \\ 1 & \text{sinon.} \end{cases}$$

satisfait $f(k) \leq 1.4656^k$.

Partie V

Comme dans la partie précédente, on cherche à répondre à la question suivante : *Étant donné un graphe G et un entier k , est-ce que $\tau(G) \leq k$?*

On va s'intéresser à la notion de noyau. On dit que le problème de l'ensemble couvrant a un **noyau de taille** $f(k)$ (pour une certaine fonction croissante f) s'il existe un algorithme qui étant donné une instance (G, k) renvoie en temps polynomial (en $|V(G)|$) une instance (G', k') tel que :

- (i) $\tau(G) \leq k$ si et seulement si $\tau(G') \leq k'$,
- (ii) si $\tau(G) \leq k$, alors $|V(G')| \leq f(k)$ et
- (iii) $k' \leq k$.

Question V.1. Montrer que si le problème de l'ensemble couvrant a un noyau de taille $f(k)$, alors il existe un polynôme P tel qu'on peut décider si $\tau(G) \leq k$ en temps $O\left(P(|V(G)|) + f(k)^{k+2}\right)$.

Le reste de la partie est dédié à la conception d'un algorithme montrant que le problème de l'ensemble couvrant a un noyau de taille $k^2 + k$.

Question V.2. Montrer que si un sommet v de G a degré 0, alors $\tau(G) \leq k$ si et seulement si $\tau(G - \{v\}) \leq k$.

Question V.3. Montrer que si un sommet v de G a degré au moins $k + 1$, alors $\tau(G) \leq k$ si et seulement si $\tau(G - \{v\}) \leq k - 1$.

On considère maintenant l'algorithme qui, étant donné (G, k) , applique les deux règles suivantes (dans n'importe quel ordre) tant que c'est possible. On note (G', k') le couple obtenu.

Règle 1 : si un sommet v a degré 0, supprimer v .

Règle 2 : si un sommet v a degré au moins $k + 1$, supprimer v , et diminuer k de 1.

On remarque à l'aide des deux questions précédentes que $\tau(G) \leq k$ si et seulement si $\tau(G') \leq k'$.

Question V.4. Donner la complexité de cet algorithme en fonction de $|V(G)|$ (on rappelle que G est représenté par sa matrice d'adjacence).

Question V.5. Montrer que pour tout sommet v de G' , on a dans G' : $1 \leq d(v) \leq k'$. En déduire que si $\tau(G') \leq k'$, alors $|V(G')| \leq k' + k'^2$ et $E(G') \leq k'^2$.

Question V.6. Conclure que le problème de l'ensemble couvrant a un noyau de taille $k^2 + k$.

Partie VI

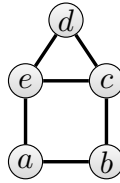
Soit $G = (V, E)$ un graphe avec $V = \{v_1, \dots, v_n\}$. Un ensemble de sommets S peut être représenté par un vecteur $x = (x_{v_1}, x_{v_2}, \dots, x_{v_n}) \in \{0, 1\}^n$ tel que, pour $i = 1, \dots, n$, $x_{v_i} = 1$ si $v_i \in S$ et $x_{v_i} = 0$ si $v_i \notin S$.

On introduit une variable x_v pour chaque sommet de G . Observer que les ensembles couvrant de G correspondent précisément aux solutions du système d'équations suivant :

$$\begin{cases} x_u + x_v \geq 1 \text{ pour tout } \{u, v\} \in E(G); \\ x_v \in \{0, 1\} \text{ pour tout } v \in V(G). \end{cases} \quad (1)$$

Ainsi, une solution minimisant $\sum_{v \in V(G)} x_v$ correspond à un ensemble couvrant minimum. Pour un graphe G donné, on nomme $\text{OLNEEC}(G)$ (pour **optimisation linéaire en nombre entier pour ensemble couvrant**) le système d'équations (1) associé à G .

Question VI.1. Écrire le système d'équations $\text{OLNEEC}(G)$ pour le graphe ci-dessous.



En relaxant la contrainte $x_v \in \{0, 1\}$ par $0 \leq x_v \leq 1$, on obtient le système d'équations suivant nommé $\text{OLEC}(G)$ (pour **optimisation linéaire pour ensemble couvrant**) :

$$\begin{cases} x_u + x_v \geq 1 \text{ pour tout } \{u, v\} \in E(G); \\ 0 \leq x_v \leq 1 \text{ pour tout } v \in V(G). \end{cases} \quad (2)$$

On peut considérer que $x_u = \frac{1}{3}$ (par exemple) correspond à mettre $\frac{1}{3}$ du sommet u dans la solution. Une solution à ce système d'équations s'appelle un **ensemble couvrant fractionnaire** de G et sa **taille** vaut $\sum_{v \in V(G)} x_v$. Une solution minimisant $\sum_{v \in V(G)} x_v$ est un **ensemble couvrant fractionnaire minimum** de G , et sa taille est noté $\tau_f(G)$. Il est clair que pour tout graphe G ,

$$\tau_f(G) \leq \tau(G)$$

Question VI.2. Parmi les vecteurs $(x_a, x_b, x_c, x_d, x_e)$ suivants, dites lesquels sont des ensembles couvrants fractionnaires du graphe de la question VI.1 et lesquels ne le sont pas. Donner leur taille quand ils le sont, et expliquer précisément la raison quand ils ne le sont pas.

$$\left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right), \quad \left(\frac{1}{3}, \frac{1}{3}, \frac{2}{3}, \frac{2}{3}, \frac{1}{3}\right), \quad (1, 1, 0, 1, 0), \quad (1, 0, 1, 1, 0).$$

Question VI.3. Montrer que pour tout graphe G , on a $\tau_f(G) \leq \frac{|V(G)|}{2}$. En déduire qu'il n'existe pas de constante A telle que $\tau(G) - \tau_f(G) < A$ pour tout graphe G .

Soit G un graphe. Soit $(x_v)_{v \in V(G)} \in [0, 1]^n$ un ensemble couvrant fractionnaire minimum de G . On partitionne les sommets de G de la façon suivante :

$$V_0 = \left\{v : x_v < \frac{1}{2}\right\} \quad V_{\frac{1}{2}} = \left\{v : x_v = \frac{1}{2}\right\} \quad V_1 = \left\{v : x_v > \frac{1}{2}\right\}$$

Question VI.4. Montrer que V_0 est un ensemble indépendant (c'est à dire qu'aucune arête n'a ses deux extrémités dans V_0), et qu'il n'y a pas d'arête ayant une extrémité dans V_0 et l'autre dans $V_{\frac{1}{2}}$.

Question VI.5. Montrer que $V_{\frac{1}{2}} \cup V_1$ est un ensemble couvrant de G et que $|V_{\frac{1}{2}} \cup V_1| \leq 2\tau(G)$. On admettra qu'il existe un algorithme en temps polynomial permettant de trouver un ensemble couvrant fractionnaire minimum. En déduire un algorithme polynomial qui donne une 2-approximation pour le problème de l'ensemble couvrant minimum.

Le but des 3 prochaines questions est de montrer qu'il existe un ensemble couvrant minimum inclus dans $V_{\frac{1}{2}} \cup V_1$. Soit S^* un ensemble couvrant minimum de G , et soit $S = (V_{\frac{1}{2}} \cap S^*) \cup V_1$.

Question VI.6. Montrer que S est un ensemble couvrant de G .

On va perturber la solution $(x_v)_{v \in V(G)}$ comme suit. On pose

$$\varepsilon = 0 \text{ si } V_0 \cup V_1 = \emptyset \quad \text{et} \quad \varepsilon = \min \left\{ \left| x_v - \frac{1}{2} \right| : v \in V_0 \cup V_1 \right\} \text{ sinon}$$

et pour tout $v \in V(G)$, on définit :

$$y_v = \begin{cases} x_v - \varepsilon & \text{si } v \in V_1 \setminus S^*; \\ x_v + \varepsilon & \text{si } v \in V_0 \cap S^*; \\ x_v & \text{dans tous les autres cas.} \end{cases}$$

Question VI.7. Montrer que $(y_v)_{v \in V(G)}$ est un ensemble couvrant fractionnaire de G .

Question VI.8. Montrer que S est un ensemble couvrant minimum de G .

On va maintenant étudier l'algorithme suivant pour déterminer, étant donné un graphe G et un entier k , si $\tau(G) \leq k$.

Algorithme 4 OLNOYAU(G, k)

- 1: Calculer un ensemble couvrant fractionnaire minimum $(x_v)_{v \in V(G)}$ à l'aide d'un algorithme en temps polynomial.
 - 2: Définir les ensembles V_0 , $V_{\frac{1}{2}}$ et V_1 comme précédemment.
 - 3: **if** $\sum_{v \in V(G)} x_v > k$ **then return** NON
 - 4: **if** $|V_{\frac{1}{2}}| > 2k$ **then return** NON
 - 5: Résoudre l'instance $(G - V_0 - V_1, k - |V_1|)$ en utilisant un algorithme par force brute.
-

Question VI.9. Montrer que si l'algorithme renvoie NON aux lignes 3 ou 4, alors $\tau(G) > k$.

Question VI.10. Montrer que $\tau(G) \leq k$ si et seulement si $\tau(G - V_0 - V_1) \leq k - |V_1|$. Conclure que le problème de l'ensemble couvrant a un noyau de taille $2k$.

Fin du sujet.

A2024 – INFO I MPI

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

**Concours Mines-Télécom,
Concours Centrale-Supélec (Cycle International).**

CONCOURS 2024**PREMIÈRE ÉPREUVE D'INFORMATIQUE**

Durée de l'épreuve : 3 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

Cette épreuve concerne uniquement les candidats de la filière MPI.

L'énoncé de cette épreuve comporte 9 pages de texte.

Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.

Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

Présentation du sujet

L'épreuve est composée d'un problème unique comportant 33 questions divisées en trois sections. L'objectif du problème est d'extraire un *sous-graphe le plus dense* d'un graphe non orienté quelconque, c'est-à-dire repérer un sous-ensemble de sommets riche en arêtes. Le calcul d'un sous-graphe dense possède des applications considérables dans le domaine de la fouille des données : qu'il s'agisse de détection de communautés dans des réseaux sociaux, de repérage d'attaques par pollupostage (ou *spamming*) ou d'identification de complexes protéiques au sein de données biologiques massives pour ne citer que quelques exemples.

Dans la première section (page 1), nous résolvons un problème de minimisation de bordure dans un multigraphe orienté à l'aide d'une technique de *chemins augmentants*. Dans la deuxième section (page 5), nous étudions une technique de *réduction du problème* de construction d'un sous-graphe le plus dense au problème de la première section. Dans la troisième section (page 8), nous étudions un *algorithme d'approximation* plus rapide.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractères différentes désignera la même entité, mais du point de vue mathématique avec la police en italique (par exemple n , $n_{\max}$ ou n_1) et du point de vue informatique avec celle en romain avec espacement fixe (par exemple `n`, `NMAX` ou `n1`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat.

Selon les consignes, il faudra coder des fonctions à l'aide du langage de programmation C exclusivement, en reprenant le prototype de fonction fourni par le sujet, ou en pseudo-code (c-à-d. dans une syntaxe souple mais conforme aux possibilités offertes par le langage C). Inclure les entêtes tels que `<assert.h>`, `<stdbool.h>`, etc. n'est pas demandé.

Quand l'énoncé demande de coder une fonction, sauf indication explicite de l'énoncé, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté et de la concision des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle. Lorsqu'une réponse en pseudo-code est permise, seule la logique de programmation est évaluée, même dans le cas où un code en C a été fourni en guise de réponse.

1 Entourage de plus petite bordure

Dans cette section, nous travaillons dans un *multigraphe orienté*, c'est-à-dire un couple (W, F, μ) où W est un ensemble fini, appelé ensemble de sommets, F est une partie de $W \times W$, appelée ensemble d'arcs, et $\mu : F \rightarrow \mathbb{N}^*$ est une application, appelée multiplicité des arcs.

Les démonstrations ou justifications pourront utiliser la notion de *multi-ensemble* sans formalité excessive. L'intersection, notée $\cap$, se calcule en considérant le minimum des multiplicités des éléments; l'union disjointe, notée $\sqcup$, se calcule en considérant la somme des multiplicités des éléments; le cardinal se calcule en sommant toutes les multiplicités.

1.1 Réseaux

Définitions : Nous appelons *réseau* tout quintuplet $H = (W, F, \mu, s, t)$ tel que

- W est un ensemble fini, appelé ensemble de *sommets*,
- F est un sous-ensemble de couples de $W \times W$, appelé ensemble d'*arcs*,
- μ est une application $F \rightarrow \mathbb{N}^*$, l'entier naturel $\mu(e)$ étant appelé la *multiplicité* de l'arc e ,
- s est un sommet particulier de W , appelé la *source*,
- t est un sommet particulier de W , appelé le *puits*.

Indication C : Par convention, nous numérotons les sommets d'un réseau $H = (W, F, \mu, s, t)$ par les entiers entre 0 et $n + 1$, où $n + 2$ est le cardinal de W . La source s porte le numéro 0 ; le puits t porte le numéro $n + 1$.

Nous préparons deux fichiers, `network.h` et `network.c`. Le premier fichier contient les déclarations suivantes.

```

1.  /* Debut du fichier network.h */
2.
3.  #ifndef NETWORK_H
4.  #define NETWORK_H
5.
6.  const unsigned int NMAX = 102;
7.
8.  struct network_s {
9.      unsigned int n;
10.     unsigned mu[NMAX][NMAX];
11. };
12. typedef struct network_s network;
13.
14. void nw_init(unsigned int n);
15. void nw_add(int u, int v);
16. void nw_remove(int u, int v);
17. void nw_disconnect(void);
18.
19. void _arr_init(int *T);
20. bool _bfs_tree(void);
21. void _residue(void);
22.
23. #endif
24.
25. /* Fin du fichier network.h */
```

□ 1 – Expliquer le rôle des lignes 3, 4 et 23 dans le fichier `network.h`.

Le second fichier contient initialement les définitions suivantes et sera complété par les fonctions de la section 1.

Première épreuve d'informatique MPI 2024

```

26.  /* Debut du fichier network.c */
27.
28.  network H;
29.  int A[NMAX];
30.
31.  /* Fin du fichier network.c */

```

Pour tout couple $(u, v) \in W \times W$, le champ $\mu[u][v]$ de la variable globale H vaut 0 si (u, v) n'est pas un arc du réseau et vaut $\mu((u, v))$ si (u, v) est un arc.

☐ 2 – Expliquer comment faire interagir les fichiers `network.h` et `network.c`.

☐ 3 – Écrire une fonction C `void nw_init(unsigned int n)` ainsi spécifiée :

Précondition : L'entier naturel n est strictement inférieur à la constante globale $n_{\max}$.

Effet : La précondition est vérifiée par une assertion.

Postcondition : La variable globale H contient le réseau vide avec $n + 2$ sommets et aucun arc.

☐ 4 – Écrire une fonction C `void nw_add(int u, int v)` qui ajoute une occurrence de l'arc (u, v) au réseau (W, F, μ, s, t) stocké dans la variable globale H , autrement dit, qui incrémente la multiplicité $\mu[u][v]$ d'une unité.

☐ 5 – Écrire une fonction C `void nw_remove(int u, int v)` qui retire une occurrence de l'arc (u, v) dans le réseau stocké dans la variable globale H et qui se défend, par le truchement d'une assertion, du cas où l'arc à supprimer n'existe pas.

☐ 6 – Écrire une fonction C `void _arr_init(int *T)` ainsi spécifiée :

Précondition : Le pointeur T désigne un tableau d'entiers formé de $n + 2$ cases, où $n + 2$ est le nombre de sommets du réseau stocké dans la variable globale H .

Postcondition : Toutes les cases du tableau désigné par le pointeur T valent -1 .

Nous utilisons le terme *chemin* pour signifier systématiquement un chemin simple dans le graphe orienté (W, F) , c'est-à-dire une suite d'arcs $(e_1, \dots, e_t)$ de F distincts qui se succèdent. Le sommet initial d'un chemin s'appelle la *source*, le sommet final s'appelle le *puits*.

☐ 7 – Écrire une fonction C `bool _bfs_tree(void)` ainsi spécifiée :

Précondition : La variable globale H contient un réseau (W, F, μ, s, t) à $n + 2$ sommets.

Effet : La fonction calcule une arborescence $\mathcal{A}$ de plus courts chemins selon un parcours en largeur dans le graphe (W, F) et l'écrit dans la variable globale A .

Postcondition : On a $A[s] = s$ et, pour tout autre sommet $v \in W$ accessible depuis s , la case $A[v]$ contient le prédécesseur du sommet v dans l'arborescence $\mathcal{A}$. Enfin, toute autre case de A vaut -1 .

Valeur de retour : Booléen `true` si le puits t est accessible depuis la source s dans l'arborescence $\mathcal{A}$. Booléen `false` sinon.

1.2 Déconnexion de la source et du puits

Définition : Nous disons qu'un ensemble de chemins $\Pi = \{\pi_1, \dots, \pi_k\}$ de source commune s et de puits commun t est *réalisable* dans le réseau $H = (W, F, \mu, s, t)$ si, pour tout arc $e \in F$, les chemins de Π utilisent moins de $\mu(e)$ fois l'arc e , autrement dit

$$\forall e \in F, \quad \text{Card}(\{j \in \llbracket 1, k \rrbracket \text{ tel que } e \in \pi_j\}) \leq \mu(e).$$

Définition : Étant donné un ensemble réalisable de chemins $\Pi = \{\pi_1, \dots, \pi_k\}$ de source commune s et puits commun t dans un réseau $H = (W, F, \mu, s, t)$, nous appelons *réseau résiduel de H par rapport à Π* , et nous notons $\text{Res}(H, \Pi)$, le réseau obtenu à partir de H en effectuant, pour tout indice j compris entre 1 et k et pour tout arc $e = (u, v) \in \pi_j$, la suppression d'une occurrence de l'arc e et l'ajout d'une occurrence de l'arc opposé $\bar{e} = (v, u)$.

□ 8 – Écrire une fonction C `void _residue(void)` ainsi spécifiée :

Précondition : La variable globale H contient un réseau (W, F, μ, s, t) . L'ensemble des arcs $\{(A[v], v); 0 < v \leq n + 1 \text{ avec } A[v] > 0\}$ tiré de la variable globale A forme une arborescence $\mathcal{A}$ de racine $s = 0$ atteignant le puits t dans le graphe (W, F) .

Postcondition : La variable H contient le réseau résiduel $\text{Res}((W, F, \mu, s, t), \{\alpha\})$ où α est l'unique chemin de source s et de puits t dans l'arborescence $\mathcal{A}$.

□ 9 – Nous appliquons k fois la fonction `_residue` à un réseau H en mettant à jour la variable globale A et notons $\alpha_1, \dots, \alpha_k$ les chemins utilisés entre la source s et le puits t . Montrer qu'il est toujours possible de trouver un ensemble réalisable Π de k chemins dans H tels que le réseau obtenu des applications de `_residue` coïncide avec le réseau résiduel par rapport à Π , autrement dit tel que l'on a

$$\text{Res}(\dots \text{Res}(\text{Res}(H, \{\alpha_1\}), \{\alpha_2\}), \dots, \{\alpha_k\}) = \text{Res}(H, \Pi).$$

□ 10 – Écrire une fonction C `void nw_disconnect(void)` ainsi spécifiée :

Précondition : La variable globale H contient un réseau (W, F, μ, s, t) .

Effet : Tant qu'il existe un chemin de source s et de puits t , le réseau H est transformé en le réseau résiduel $\text{Res}((W, F, \mu, s, t), \{\alpha\})$, où α est un plus court chemin de s vers t .

Postcondition : Si la fonction se termine, il n'existe pas de chemin de la source vers le puits dans le réseau H . De plus, l'ensemble des arcs $\{(A[v], v); 0 \leq v \leq n + 1 \text{ avec } A[v] > 0\}$ tiré de la variable globale A forme une arborescence $\mathcal{A}$ dans le réseau résiduel dans laquelle la racine est s et le sommet t n'est pas accessible.

□ 11 – Démontrer que la fonction `nw_disconnect` se termine en introduisant un variant de boucle inspiré par la question 9.

1.3 Optimalité

Définition : Soit $H = (W, F, \mu, s, t)$ un réseau et S un sous-ensemble de sommets de W . Nous appelons *bordure*, et notons ∂S , l'ensemble d'arcs allant de S vers son complémentaire :

$$\partial S = \{(u, v) \in F \text{ avec } u \in S \text{ et } v \notin S\}.$$

Nous étendons la notion de multiplicité à des ensembles d'arcs en posant :

$$\mu(\partial S) = \sum_{e \in \partial S} \mu(e).$$

Première épreuve d'informatique MPI 2024

Définition : Nous appelons *entourage* dans le réseau $H = (W, F, \mu, s, t)$ tout sous-ensemble de sommets $S \subseteq W$ contenant la source s mais pas le puits t .

□ 12 – Soient Π un ensemble réalisable de k chemins dans un réseau $H = (W, F, \mu, s, t)$ et $S \subseteq W$ un entourage dans H . Établir la relation

$$k \leq \mu(\partial S).$$

Nous fixons pour les quatre prochaines questions un ensemble réalisable Π_0 de k_0 chemins dans un réseau H tel que, dans le réseau résiduel $\text{Res}(H, \Pi_0)$, il n'existe plus aucun chemin entre la source s et le puits t . Nous notons S_0 l'entourage dans H composé des sommets accessibles dans $\text{Res}(H, \Pi_0)$ depuis la source s .

□ 13 – Montrer que, pour tout arc $e = (u, v) \in F$ tel que $u \in S_0$ et $v \notin S_0$, toutes les $\mu(e)$ occurrences de l'arc e sont utilisées par un chemin de l'ensemble Π_0 .

□ 14 – Montrer qu'aucune occurrence d'un arc $(v, u) \in F$ tel que $v \notin S_0$ et $u \in S_0$ n'est utilisée par un chemin de l'ensemble Π_0 .

□ 15 – Démontrer l'égalité

$$k_0 = \mu(\partial S_0).$$

Définition : Nous disons qu'un entourage S est *de plus petite bordure* s'il minimise la multiplicité $\mu(\partial S)$. Nous notons $\sigma(H)$ la multiplicité de la bordure d'un tel entourage.

□ 16 – Démontrer la relation

$$k_0 = \sigma(H)$$

et en déduire que, si la variable globale H contient un réseau, après exécution de la fonction `nw_disconnect`, l'ensemble $\{v \in W ; A[v] \geq 0\}$ est un entourage de plus petite bordure de H , à savoir un entourage de multiplicité $\sigma(H)$.

2 Algorithme de Goldberg

2.1 Densité d'un graphe

Pour tout ensemble fini X , la notation $\binom{X}{2}$ désigne l'ensemble des paires (non ordonnées) à valeur dans X , c'est-à-dire l'ensemble

$$\binom{X}{2} = \{\{u, v\} \text{ avec } (u, v) \in X^2 \text{ tel que } u \neq v\}.$$

Définition : Un *graphe non orienté* est un couple $G = (V, E)$ où V est un ensemble fini, appelé ensemble de sommets, et $E \subseteq \binom{V}{2}$ est un ensemble de paires, appelé ensemble d'arêtes. Pour tout ensemble de sommets $X \subseteq V$, nous notons $G_X = (X, E_X)$ le graphe $G_X = (X, E \cap \binom{X}{2})$, appelé *graphe induit par X sur G* .

Première épreuve d'informatique MPI 2024

Indication C : Par convention, nous numérotons les sommets de tout graphe non orienté $G = (V, E)$ par les entiers entre 1 et n , où n est le cardinal de V . Nous représentons un graphe par un tableau de listes d'adjacence doublement chaînées en adoptant les déclarations de type suivantes.

```

32.  struct link_s {
33.      int node;
34.      struct link_s *prev;
35.      struct link_s *next;
36.  };
37.  typedef struct link_s link;
38.
39.  struct graph_s {
40.      unsigned int n;
41.      link **neighbours;
42.  };
43.  typedef struct graph_s graph;

```

Le champ `neighbours` est un tableau alloué dynamiquement de longueur $n + 1$ et dont la première case n'est pas utilisée. Chaque arête est stockée sous forme de deux arcs.

□ 17 – Écrire une fonction C `graph *gr_create(unsigned int n)` dont la valeur de retour est un graphe non orienté vide à n sommets.

□ 18 – Écrire une fonction C `void gr_add(graph *G, int u, int v)` ayant pour effet d'insérer une arête $\{u, v\}$ dans le graphe non orienté G .

Définition : Nous notons $\deg_G(v)$ le *degré* d'un sommet v dans le graphe non orienté $G = (V, E)$:

$$\deg_G(v) = |\{u \in V \text{ tel que } \{u, v\} \in E\}| = \sum_{\substack{u \in V \text{ t.q.} \\ \{v, u\} \in E}} 1.$$

□ 19 – Écrire une fonction C `int degree(graph *G, int v)` dont la valeur de retour est le degré $\deg_G(v)$ du sommet v dans le graphe non orienté G .

□ 20 – Écrire une fonction C `int edges(graph *G)` dont la valeur de retour est le nombre d'arêtes dans le graphe non orienté G .

Définition : Nous appelons densité du graphe $G = (V, E)$ le rapport

$$\delta(G) = \frac{m}{n}$$

où n est le cardinal de V et m est le cardinal de E .

□ 21 – Écrire une fonction C `double density(graph *G)` dont la valeur de retour est la densité $\delta(G)$ du graphe non orienté G .

Définition : Le problème d'optimisation du *sous-graphe le plus dense d'un graphe non orienté* G consiste à trouver un ensemble de sommets non vide X tels que la densité $\delta(G_X)$ du sous-graphe induit $G_X = (X, E_X)$ est maximale et calculer cette densité maximale

$$\rho(G) = \max_{\substack{X \subseteq V; \\ X \neq \emptyset}} \delta(G_X).$$

2.2 Oracle pour le problème de décision

Nous nous intéressons au problème de décision suivant :

Étant donné un graphe non orienté G à n sommets et un entier naturel r , existe-t-il un sous-graphe induit $G_X = (X, E_X)$ de densité $\delta(G_X)$ strictement supérieure au seuil $\frac{r}{n^2}$? Autrement dit, a-t-on $\rho(G) > \frac{r}{n^2}$?

Définition : Étant donné un graphe non orienté $G = (V, E)$ avec n sommets et m arêtes ainsi qu'un entier naturel r , nous appelons *réseau associé au graphe G et au seuil r* le réseau $H = (W, E, \mu, s, t)$ où

(α) L'ensemble W est la réunion $W = V \cup \{s, t\}$, les sommets source s et puits t étant de nouveaux sommets n'appartenant pas à V .

(β) L'ensemble d'arcs F comprend :

- pour tout sommet $v \in V$, un arc (s, v) , de multiplicité mn^2 .
- pour toute arête $\{u, v\} \in E$, un arc (u, v) et un arc (v, u) , de multiplicité n^2 .
- pour tout sommet $u \in V$, un arc (u, t) , de multiplicité $mn^2 - n^2 \deg_G(u) + 2r$.

□ 22 – Écrire une fonction C `void reduce(graph *G, int r)` ayant pour effet d'initialiser la constante globale H , de type `network`, par le réseau associé au graphe G et au seuil r .

□ 23 – Soient $G = (V, E)$ un graphe non orienté et $X \subseteq V$ une partie non vide des sommets. Montrer que la densité du graphe induit $G_X = (X, E_X)$ satisfait la relation

$$\delta(G_X) = \frac{\sum_{v \in X} \deg_G(v) - \sum_{\substack{u \in X, v \in V \setminus X \\ \text{t.q. } \{u, v\} \in E}} 1}{2|X|}.$$

□ 24 – Soient $G = (V, E)$ un graphe non orienté avec n sommets et m arêtes, $X \subseteq V$ une partie non vide des sommets, H le réseau associé au graphe G et à un seuil r et S l'entourage $S = \{s\} \cup X$ dans H . Montrer que la multiplicité de la bordure de S satisfait l'égalité

$$\mu(\partial S) = mn^3 + 2|X| \left(r - n^2 \delta(G_X) \right)$$

et que

$$\mu(\partial\{s\}) = mn^3.$$

□ 25 – Soit H le réseau associé au graphe G et au seuil r et soit $\sigma(H)$ la multiplicité de la bordure d'un entourage de plus petite bordure. Montrer les équivalences suivantes :

$$\begin{cases} \sigma(H) = mn^3 & \Leftrightarrow \rho(G) \leq \frac{r}{n^2}, \\ \sigma(H) < mn^3 & \Leftrightarrow \rho(G) > \frac{r}{n^2}. \end{cases}$$

□ 26 – Écrire une fonction C `bool oracle(graph *G, int r)` dont la valeur de retour est le booléen `true` si $\rho(G) > \frac{r}{n^2}$ et `false` si $\rho(G) \leq \frac{r}{n^2}$. En justifier le principe. On pourra s'intéresser au contenu du tableau global `A` après exécution de la fonction `nw_disconnect`.

2.3 Recherche dichotomique

Nous considérons l'ensemble de couples d'entiers

$$\Delta = \left\{ (m', n') \in \mathbb{N}^2 \text{ avec } 0 \leq m' \leq m \text{ et } 1 \leq n' \leq n \right\}.$$

□ 27 – Soient deux couples (m_1, n_1) et (m_2, n_2) appartenant à Δ . Montrer que,

$$\text{si } \frac{m_1}{n_1} \neq \frac{m_2}{n_2}, \quad \text{alors} \quad \left| \frac{m_1}{n_1} - \frac{m_2}{n_2} \right| \geq \frac{1}{n^2}.$$

□ 28 – Soient $G = (V, E)$ un graphe non orienté à n sommets et X une partie non vide de V . Montrer que s'il n'existe aucun sous-graphe induit de densité supérieure ou égale à $\delta(G_X) + \frac{1}{n^2}$, alors le sous-graphe induit G_X est un sous-graphe de densité maximale $\rho(G)$.

□ 29 – Écrire une fonction C efficace `void binary_search(graph *G)` qui, pour tout graphe $G = (V, E)$, modifie le tableau global `A` de sorte que l'ensemble de sommets $X = \{v \in V ; A[v] \geq 0\}$ vérifie

$$\delta(G_X) = \rho(G).$$

Justifier brièvement le principe de fonctionnement de la fonction.

□ 30 – Nous supposons avoir écrit l'ensemble des fonctions de la section 2 dans un fichier `densest_subgraph.c`. Expliquer comment faire interagir le fichier `densest_subgraph.c` avec les fichiers `network.h` et `network.c` de la section 1.

3 Algorithme de Charikar

Dans le cadre d'un compromis entre temps d'exécution et justesse du calcul d'optimisation, nous nous intéressons à l'algorithme suivant.

Algorithme 1 : Recherche gloutonne d'une densité élevée

Entrées : Graphe $G = (V, E)$.

Sorties : Sous-graphe induit à grande densité

```

1  $\Gamma \leftarrow G$  ; /* Plus dense graphe rencontré */
2  $G_n \leftarrow G$  où  $n = |V|$ 
3 pour  $i$  de  $n$  à 2 (cas inclus) faire
4   si  $\delta(G_i) > \delta(\Gamma)$  alors
5      $\Gamma \leftarrow G_i$ 
6   Identifier le sommet  $v_i$  qui minimise le degré  $\deg_{G_i}$ .
7   Former le graphe  $G_{i-1}$  en supprimant de  $G_i$  le sommet  $v_i$  et les
   arêtes incidentes à  $v_i$ .
8 retourner  $\Gamma$ 
```

3.1 Analyse d'un facteur d'approximation

Définition : Nous appelons *orientation* d'un graphe non orienté $G = (V, E)$ tout graphe orienté $\hat{G} = (V, \hat{E})$ tel qu'il existe une bijection $\omega : E \rightarrow \hat{E}$ avec $\omega(\{u, v\}) = (u, v)$ pour tout arc $(u, v) \in \hat{E}$. Le *degré entrant* d'un sommet v dans un graphe orienté $\hat{G} = (V, \hat{E})$ est

$$\deg_{\hat{G}}^{-}(v) = \text{Card} \left(\left\{ u \in V \text{ tel que } (u, v) \in \hat{E} \right\} \right) = \sum_{\substack{u \in V \text{ t.q.} \\ (u, v) \in \hat{E}}} 1.$$

□ 31 – Établir la majoration suivante entre la densité d'un graphe non orienté quelconque G et le degré entrant maximum d'une orientation quelconque $\hat{G}$ de G

$$\delta(G) \leq \max_{v \in V} \deg_{\hat{G}}^{-}(v).$$

Nous exécutons l'algorithme 1 sur un certain graphe non orienté G . Dans les questions qui suivent, nous numérotions les sommets de G selon la même numérotation que celle de l'algorithme. Nous notons ω_0 la bijection qui oriente l'arête $\{v_i, v_{i'}\}$ dans le sens $(v_{\min(i, i')}, v_{\max(i, i')})$ et $\hat{G}_0$ l'orientation associée. Autrement dit, nous orientons toute arête, au moment de sa suppression, vers le sommet incident qui a conduit à sa suppression.

□ 32 – Établir, pour tout i compris entre 1 et n , l'inégalité

$$\deg_{\hat{G}_0}^{-}(v_i) \leq 2\delta(G_i)$$

et en déduire que l'algorithme 1 est un algorithme d'approximation dont on précisera le facteur d'approximation.

3.2 Implémentation optimale

□ 33 – Détailler une structure de données qui permette d'exécuter l'algorithme 1 en temps $O(n + m)$ où $n = |V|$ et $m = |E|$. Il est suggéré de nantir la structure de données **graph** d'une table qui maintient, pour tout entier d compris entre 0 et n , la collection des sommets de degré d ainsi que d'autres éléments que l'on jugera opportuns.

Une réponse décrite en pseudo-code ou en langage naturel est admise mais doit être suffisamment précise pour s'assurer que la complexité est belle et bien linéaire.

FIN DE L'ÉPREUVE

A2024 – INFO II MPI

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

**Concours Mines-Télécom,
Concours Centrale-Supélec (Cycle International).**

CONCOURS 2024**DEUXIÈME ÉPREUVE D'INFORMATIQUE**

Durée de l'épreuve : 4 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

Cette épreuve concerne uniquement les candidats de la filière MPI.

L'énoncé de cette épreuve comporte 12 pages de texte.

Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.

Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

L'épreuve est composée d'un problème unique, comportant 36 questions. Le problème est divisé en quatre sections. Dans la première section (page 1), nous introduisons une forme paresseuse de listes, les *flots de données*, et construisons quelques fonctions de base utiles tout au long du sujet. Dans la deuxième section (page 4), nous nous intéressons à un *algorithme de détermination à la volée du cardinal* d'un flot, dont l'inspiration est probabiliste. Dans la troisième section (page 7), nous nous appuyons sur des raisonnements de logique pour analyser le *chiffrement par flot*. Dans la quatrième section (page 9), nous traitons quelques questions de *combinatoire énumérative* à l'aide d'une grammaire ou simplement d'un procédé manuel de dérécursification.

Les sections 2, 3 et 4 peuvent être vues comme des exercices indépendants qui ne dépendent que de la première section.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractères différentes désigne la même entité, mais du point de vue mathématique pour la police en italique (par exemple n) et du point de vue informatique pour celle en romain avec espacement fixe (par exemple `n`).

Des rappels portant sur des extraits du manuel de documentation de OCaml et sur les règles de la déduction naturelle sont fournis en annexe.

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat.

Il faudra coder des fonctions à l'aide du langage de programmation OCaml exclusivement, en reprenant l'en-tête de fonctions fourni par le sujet, sans s'obliger à recopier la déclaration des types. Il est permis d'utiliser la totalité du langage OCaml mais il est recommandé de s'en tenir aux fonctions les plus courantes afin de rester compréhensible. Quand l'énoncé demande de coder une fonction, sauf demande explicite, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté et de la concision des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle.

1. Flots de données

1.1. Motivation

En langage OCaml, le type natif `'a list` est défini comme point fixe de l'équation

```
1. type 'a list = [] | (::) of 'a * 'a list
```

Ce type permet de définir des listes finies tout comme certaines listes infinies, ainsi que l'expose l'exemple suivant, qui est parfaitement licite :

```
2. let list_finite = [ 1; 2; 3 ]          (* liste finie a 3 elements *)
3. let rec list_ones = 1 :: list_ones    (* liste infinie toute a 1 *)
```


Seconde épreuve d'informatique MPI 2024

□ 1 – Rendre compte, sous la forme d'un croquis constitué de maillons chaînés entre eux, de l'organisation en mémoire des variables `list_finite` et `list_ones`. Dire quelle région de la mémoire d'un programme est utilisée pour ce stockage.

Les termes de la suite de Fibonacci $(F_n)_{n \in \mathbb{N}}$ obéissent aux relations :

$$F_0 = 0, \quad F_1 = 1, \quad \text{et} \quad \forall n \in \mathbb{N}, F_{n+2} = F_n + F_{n+1}.$$

Nous nous enhardissons à définir la liste des termes de la suite de Fibonacci $(F_n)_{n \in \mathbb{N}}$ en écrivant la déclaration récursive suivante :

```
4. let list_fibo : int list =
5.   let rec list_fibo_with_internal_state (a:int) (b:int) : int list =
6.     a :: (list_fibo_with_internal_state b (a+b))
7.   in
8.     list_fibo_with_internal_state 0 1
```

Cependant, à l'évaluation de cette expression par un interpréteur (ou *toplevel* ou encore *REPL*), le système renvoie le message d'erreur suivant :

```
9. Stack overflow during evaluation (looping recursion?).
```

□ 2 – Traduire en français le mot *stack*. Expliquer le message d'erreur en présentant le déroulement des premières étapes de création de la variable `list_fibo` par l'interpréteur OCaml.

Nous recopions le code fautif dans un fichier `fibo.ml` et — *horresco referens* — nous nous apprêtons à appeler successivement depuis un terminal les commandes `ocamlopt fibo.ml -o fibo` pour la compilation de la source vers un exécutable puis `./fibo` pour l'exécution.

□ 3 – Dire si l'une des deux étapes, compilation ou exécution, renvoie une erreur et, le cas échéant, préciser laquelle des étapes.

1.2. Définition des flots de données

Définition : Un *flot de données* est une suite, finie ou infinie, d'éléments de même type.

Indication OCaml : Afin de représenter les flots de données, nous fixons le type suivant :

```
10. type 'a stream = Nil | Cons of 'a * (unit -> 'a stream)
```

Le constructeur `Nil` qualifie le flot vide. Définir la suite infinie constante égale à 1 en tant que `int stream` demeure possible. Nous parvenons de surcroît à déclarer la suite de Fibonacci en tant que `int stream` sans encombre. En voici le code :

```
11. let rec (ones : int stream) = Cons (1, fun () -> ones)
12.
13. let fibo : int stream =
14.   let rec fibo_with_internal_state a b =
15.     Cons (a, fun () -> fibo_with_internal_state b (a+b))
16.   in
17.     fibo_with_internal_state 0 1
```

 Seconde épreuve d'informatique MPI 2024

- 4 – Écrire une constante OCaml `integers : int stream` dont la valeur de retour est le flot des entiers naturels énumérés par ordre croissant.
- 5 – Écrire une fonction OCaml `range (a:int) (b:int) : int stream` dont la valeur de retour est le flot des entiers compris entre l'entier a inclus et l'entier b exclu énumérés par ordre croissant.
- 6 – Soit $(\sigma_n)_{n \in \mathbb{N}}$ une suite quelconque de chaînes de caractères. Dire s'il existe forcément un flot de donnée, de type `string stream`, qui encode la suite $(\sigma_n)_{n \in \mathbb{N}}$ et, le cas échéant, en préciser la construction en langage OCaml ou en démontrer l'inexistence.

1.3. Quelques manipulations de base des flots de données

- 7 – Écrire une fonction OCaml `hd (u : 'a stream) : 'a` dont la valeur de retour est le premier élément du flot de données u et qui lève une exception `Failure "hd"` si le flot u est vide.
- 8 – Écrire une fonction OCaml `tl (u : 'a stream) : 'a stream` dont la valeur de retour est le flot de données u privé de l'élément de tête et qui lève une exception `Failure "tl"` si le flot u est vide.
- 9 – Écrire une fonction OCaml `of_list (l : 'a list) : 'a stream` dont la valeur de retour est le flot des éléments de la liste ℓ .
- 10 – Écrire une fonction OCaml `iter (f : 'a -> unit) (t : int) (u : 'a stream) : unit` qui applique la fonction f aux t premières valeurs existantes du flot de données u si l'entier t est positif et ne fait rien sinon.
- 11 – Écrire une fonction OCaml `map (f : 'a -> 'b) (u : 'a stream) : 'b stream` dont la valeur de retour est le flot constitué de l'application de la fonction f aux éléments de u .
- 12 – Écrire une fonction OCaml `zip (w : 'a stream array) : 'a array stream` qui transforme un tableau de flots de données $w = [(u_n)_{n \in \mathbb{N}}, (v_n)_{n \in \mathbb{N}}, \dots]$ en un flot de données tabulaire $([u_n, v_n, \dots])_{n \in \mathbb{N}}$. Si l'un des flots de w est vide à partir d'un certain rang, la valeur de retour est aussi vide à partir de ce rang.
 Par exemple, `zip [|ones; range 0 100 ; fibo|]` désigne le flot fini des tableaux d'entiers de longueur trois $([1, k, F_k])_{0 \leq k < 100}$.
- 13 – Écrire une fonction OCaml `intertwin (u : 'a stream) (v : 'a stream) : 'a stream` dont la valeur de retour est le flot obtenu en prenant alternativement des éléments du flot u et du flot v , en cessant de prendre des éléments d'un flot une fois que celui-ci est vide. Par exemple, `intertwin ones fibo` désigne le flot d'entiers $1, F_0, 1, F_1, 1, F_2, 1, F_3, \dots$, tandis que `intertwin (range 0 2) fibo` désigne le flot d'entiers $0, F_0, 1, F_1, F_2, F_3, F_4, \dots$.

Seconde épreuve d'informatique MPI 2024

Nous nous proposons d'écrire une fonction `product` (`u1 : 'a stream`) (`u2 : 'b stream`) : (`'a * 'b`) `stream` dont la valeur de retour est un flot qui énumère tous les couples tirés du flot u_1 et du flot u_2 dans un ordre non spécifié. Nous écrivons le code suivant :

```
18. let rec product_wrong u1 u2 =
19.     match u1,u2 with
20.     | _, Nil | Nil, _ -> Nil
21.     | Cons(hd1, tl_begetter1), Cons(hd2, tl_begetter2) ->
22.         let part1 = product_wrong (tl_begetter1 ()) u2 in
23.         let part2 = map (fun y -> (hd1, y)) (tl_begetter2 ()) in
24.         Cons((hd1, hd2), fun () -> intertwin part1 part2)
```

□ 14 – Montrer que l'appel `product_wrong u1 u2` ne se termine pas toujours mais qu'il suffit de déplacer quelques mots au sein du code pour le rendre correct.

1.4. Filtre d'un flot

□ 15 – À titre préliminaire, rappeler brièvement la signification de l'entier 0 dans l'instruction `return 0` qui termine usuellement la fonction `main` en langage C ou dans l'instruction `exit 0` du langage OCaml.

□ 16 – Énoncer le théorème de Turing relatif au problème de l'arrêt.

Pour répondre à la question 17, on pourra admettre sans justification l'existence d'une *machine universelle à chronomètre*, c'est-à-dire une fonction OCaml `run : string -> int -> int` qui se termine toujours et telle que :

- si la chaîne de caractères x est le code source d'une expression OCaml et si l'exécution du code x se termine sans erreur en moins de n étapes élémentaires de calcul, alors `run x n` a pour valeur de retour l'entier 0 ;
- sinon, `run x n` a pour valeur de retour l'entier 1.

□ 17 – Décrire la construction ou bien réfuter l'existence d'une fonction OCaml `filter (f : 'a -> bool) (u : 'a stream) -> 'a stream` dont la valeur de retour est formé des éléments du flot u satisfaisant le prédicat f , l'ordre des éléments de u étant préservé.

Par exemple, `let is_negative x = x < 0 in filter is_negative ones` vaut `Nil`.

2. Estimation du cardinal d'un flot de données

Définition : Nous appelons *cardinal* d'un flot de données le cardinal de l'ensemble des valeurs distinctes appartenant à ce flot. Ce cardinal peut être *fini* même si le flot est infini.

Dans cette section, nous cherchons à estimer le cardinal s d'un flot de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$ de cardinal fini avec peu de mémoire et en supposant que $0 \leq s < 2^{62}$.

Seconde épreuve d'informatique MPI 2024

Définition : Nous appelons *premier bit activé* d'un entier $h \in \llbracket 0, 2^{62} - 1 \rrbracket$, et notons $\varrho(h)$, le plus grand entier i tel que les $i - 1$ bits les plus à droite dans la représentation binaire de h sont tous nuls. Par convention, nous fixons $\varrho(0) = 63$.

Par exemple, nous avons $\varrho(1) = \varrho(\overline{00001}_{(2)}) = 1$, $\varrho(6) = \varrho(\overline{00110}_{(2)}) = 2$, ou encore $\varrho(8) = \varrho(\overline{01000}_{(2)}) = 4$.

□ 18 – Écrire une fonction OCaml `ffs (h:int) : int` dont la valeur de retour est le premier bit activé $\varrho(h)$ de l'entier h . En anglais, `ffs` désigne *find first set*.

Indication OCaml : Nous disposons d'une fonction de hachage $\mathcal{H}$ à valeur dans l'intervalle entier $\llbracket 0, 2^{62} - 1 \rrbracket$ que nous réalisons à l'aide de `String.hash : string -> int`.

Soit s un entier naturel non nul. Nous tirons s chaînes de caractères aléatoirement et notons $(H_j)_{1 \leq j \leq s}$ leur haché par la fonction de hachage $\mathcal{H}$. Pour tout indice j compris entre 1 et s , nous supposons que chaque variable aléatoire H_j est distribuée selon une loi uniforme dans l'intervalle entier $\llbracket 0, 2^{62} - 1 \rrbracket$. Nous posons

$$R = \max_{1 \leq j \leq s} \varrho(H_j).$$

Une analyse mathématique, que nous admettons, montre alors que les variables aléatoires $\varrho(H_j)$ sont tirées selon des lois géométriques et que la variable R est proche de $\log_2 s$, si bien que la quantité 2^R est une bonne approximation de s .

Soit $(u_n)_{n \in \mathbb{N}}$ un flot de chaînes de caractères de cardinal s , l'entier s étant inconnu. Nous proposons deux algorithmes qui estiment le cardinal s par l'observation des t premiers termes du flot, en supposant que les s valeurs distinctes apparaissent parmi les t premiers termes.

— Algorithme naïf : garnir à la volée une table de hachage avec les chaînes $(u_n)_{n \leq t}$, puis renvoyer le compte des éléments présents dans la table. En voici le code :

```

25.   let cardinality_nf (t:int) (u: string stream) : float =
26.     let htb = Hashtbl.create 1 in
27.     iter (fun x-> Hashtbl.add htb x ()) t u;
28.     float_of_int (Hashtbl.length htb)

```

où l'on réutilise la fonction `iter` de la question 10 et des fonctions du module `Hashtbl` rappelées en annexe.

— Algorithme de Flajolet-Martin, inspiré par les observations ci-dessus. En voici le code :

```

29.   let cardinality_fm (t:int) (u: string stream) : float =
30.     let r = ref 0 in
31.     let update x = r := max !r (ffs (String.hash x)) in
32.     iter update t u;
33.     float_of_int (1 lsl !r) (* calcule 2^r en flottant *)

```

□ 19 – Comparer la complexité en espace des algorithmes `cardinality_nf` et `cardinality_fm`.

Nous rappelons que la moyenne harmonique de m réels $z_1, \dots, z_m$ strictement positifs se calcule par la formule

$$\text{Harm}(z_1, \dots, z_m) = m \cdot \left(\sum_{j=1}^m \frac{1}{z_j} \right)^{-1}.$$

Seconde épreuve d'informatique MPI 2024

□ 20 – Écrire une fonction OCaml `harmonic_mean` (`z : float array`) : `float` dont la valeur de retour est la moyenne harmonique des termes du tableau `z`.

La suite de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$ et l'entier m étant fixés, pour tout entier i compris entre 0 et $m - 1$ et pour tout entier naturel t , nous notons U_t^i l'ensemble de chaînes de caractères

$$U_t^i = \{u_n \text{ où } n \leq t \text{ et } \mathcal{H}(u_n) \equiv i \pmod{m}\}.$$

Afin d'obtenir une estimation de cardinal du flot $(u_n)_{n \in \mathbb{N}}$ plus robuste et de diminuer la variabilité associée à l'unicité du point de mesure, nous proposons une amélioration de la fonction `cardinality_fm` qui utilise un effet de moyenne.

— Algorithme HyperLogLog : Nous divisons les premiers termes du flot $(u_n)_{n \in \mathbb{N}}$ en m sous-flots disjoints $U_t^0, \dots, U_t^{m-1}$. Nous estimons le cardinal de chaque sous-flot comme le fait l'algorithme de Flajolet-Martin et obtenons des valeurs $(2^{r_i})_{0 \leq i \leq m-1}$, où

$$r_i = \max_{u \in U_t^i} \varrho(\mathcal{H}(u)/m),$$

qui, intuitivement, forment chacune une estimation grossière du quotient s/m . Nous calculons la moyenne harmonique $\text{Harm}(2^{r_0}, \dots, 2^{r_{m-1}})$, qui, intuitivement, constitue une estimation robuste du quotient s/m .

Une analyse mathématique poussée confirme notre intuition. Il s'avère que le cardinal s est très bien approché par la quantité

$$s \simeq \alpha_m \cdot m \cdot \text{Harm}(2^{r_0}, \dots, 2^{r_{m-1}}) \quad (\clubsuit)$$

où α_m est une constante, valant ici $\alpha_m = \left(m \int_0^\infty \left(\log_2 \left(\frac{2+u}{1+u} \right) \right)^m du \right)^{-1}$. On utilisera l'équation $(\clubsuit)$ sans la démontrer durant l'épreuve.

Indication OCaml : Nous fixons les constantes globales pour le reste de la section 2.

```
34. let m = 16
35. let alpha16 = 0.6731 (* calcul numerique admis *)
```

L'algorithme HyperLogLog se présente alors sous la forme

```
1. let cardinality_hll (t:int) (u : string stream) : float =
2.   let r = Array.make m 0 in
3.   iter (hll_update r) t u;
4.   mean_estimate r
```

où les fonctions `hll_update` et `mean_estimate` seront écrites aux questions 21 et 22.

□ 21 – Écrire une fonction OCaml `hll_update` (`r : int array`) (`x : string`) : `unit` ainsi spécifiée :

Précondition : Il existe un flot de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$, des entiers m et t tels que $x = u_t$ et le tableau r contient $\left(\max_{u \in U_{t-1}^i} \varrho(\mathcal{H}(u)/m) \right)_{0 \leq i \leq m-1}$.

Postcondition : Le tableau r contient $\left(\max_{u \in U_t^i} \varrho(\mathcal{H}(u)/m) \right)_{0 \leq i \leq m-1}$.

Seconde épreuve d'informatique MPI 2024

- 22 – Écrire une fonction OCaml `mean_estimate (r : int array) : float` ainsi spécifiée
Précondition : Le tableau r contient les valeurs $(r_i)_{0 \leq i \leq m-1}$ avec

$$r_i = \max_{u \in U_t^i} \varrho(\mathcal{H}(u)/m)$$

pour un certain flot de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$ et un certain entier m .

Valeur de retour : Estimation du cardinal du flot $(u_n)_{n \in \mathbb{N}}$ d'après l'équation ($\clubsuit$).

Nous nous plaçons finalement dans la situation où deux flots de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$ et $(v_n)_{n \in \mathbb{N}}$ nous parviennent simultanément. Nous chargeons deux fils d'exécution auxiliaires distincts de construire indéfiniment leurs tableaux respectifs de maxima de bit activés tandis que le fil d'exécution principal déclenche toutes les cinq secondes l'affichage du cardinal. Voici une ébauche de code :

```

36. let cardinality_threadedhll u v =
37.   let r_u = Array.make m 0 in
38.   let f1 () = iter (hll_update r_u) Int.max_int u in
39.   let _ = Thread.create f1 () in
40.
41.   let r_v = Array.make m 0 in
42.   let f2 () = iter (hll_update r_v) Int.max_int v in
43.   let _ = Thread.create f2 () in
44.
45.   while true do
46.     let r = Array.init m (fun i -> max r_u.(i) r_v.(i)) in
47.     print_float (mean_estimate r);
48.     Thread.delay 5. (* mise en pause de 5 secondes *)
49.   done

```

dans laquelle `Int.max_int` représente le plus grand entier représentable.

- 23 – Identifier les portions de code sujettes à des courses critiques. Indiquer un mécanisme qui permet de corriger l'ébauche de code.

3. Chiffrement par flot de booléens

Dans cette partie, nous nous appuyons sur des variables booléennes $(x_j)_{j \leq k}$ et sur les constantes $\top$ (tautologie) et $\perp$ (antilogie) pour former des formules, à l'aide des trois connecteurs $\neg$ (négation), $\wedge$ (conjonction) et $\vee$ (disjonction). Pour toutes formules de logique ϕ et ψ , nous notons $X(\phi, \psi)$ la nouvelle formule

$$X(\phi, \psi) = (\phi \vee \psi) \wedge (\neg\phi \vee \neg\psi).$$

Les règles d'inférence de la déduction naturelle sont rappelées dans l'annexe B.

- 24 – Soit ϕ une formule. Construire des arbres de preuve qui démontrent les séquents

$$X(\phi, \phi) \vdash \perp \quad \text{et} \quad \perp \vdash X(\phi, \phi)$$

à partir des règles d'inférence de la déduction naturelle.

Seconde épreuve d'informatique MPI 2024

□ 25 – Soit ψ une formule. Construire des arbres de preuve qui démontrent les séquents

$$X(\perp, \psi) \vdash \psi \quad \text{et} \quad \psi \vdash X(\perp, \psi).$$

Nous admettons que, pour toutes formules de logique ϕ et ψ , il existe des arbres de preuve qui démontrent les séquents $X(X(\phi, \phi), \psi) \vdash \psi$ et $\psi \vdash X(X(\phi, \phi), \psi)$.

Définitions : Nous notons $\mathbb{B} = \{V, F\}$ l'ensemble des valeurs de vérité. Nous introduisons la fonction booléenne $\oplus : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ telle que $\oplus(x, y)$ est l'évaluation de la formule $X(x, y)$.

□ 26 – Justifier que les fonctions booléennes $\mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ suivantes $(x, y) \mapsto \oplus(\oplus(x, x), y)$ et $(x, y) \mapsto y$ sont égales.

Nous admettons, plus généralement, que la fonction booléenne $\oplus : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ est commutative et associative. Pour tout entier $m \geq 1$ et pour tous booléens $(b_i)_{1 \leq i \leq m}$, la somme $\bigoplus_{i=1}^m b_i$ est définie comme $\oplus(\dots \oplus (\oplus(b_1, b_2), b_3), \dots, b_m)$ mais peut être calculée dans n'importe quel ordre. La somme vide vaut, par convention, la valeur de vérité F .

Définition : Nous appelons *registre à décalage à rétroaction linéaire* (ou encore *LFSR* d'après l'anglais *linear feedback shift register*) de longueur m , de germe $(b_i)_{0 \leq i < m} \in \mathbb{B}^m$ et de connexion $C \subseteq \llbracket 0, m-1 \rrbracket$, le flot de booléens $(b_n)_{n \in \mathbb{N}}$ défini par la relation de récurrence

$$\forall n \in \mathbb{N}, \quad b_{n+m} = \bigoplus_{i \in C} b_{n+i}$$

Indication OCaml : Nous utilisons le type `bool array` pour représenter le germe et le type `int list` pour représenter la connexion d'un LFSR. La longueur s'obtient avec `Array.length`.

□ 27 – Écrire une fonction OCaml `lfsr (g:bool array) (c:int list) : bool stream` dont la valeur de retour est le flot de données constitué par le LFSR de germe g et de connexion c .

Indication OCaml : Nous représentons les formules de logique à l'aide du type

```
50.   type formula = | Var of int
51.   | Not of formula
52.   | And of formula * formula
53.   | Or  of formula * formula
```

Pour toute formule de logique ϕ définie à partir des variables booléennes $(x_j)_{1 \leq j \leq k}$ et pour tout k -upplet de valeurs de vérité $b = (b^1, \dots, b^k) \in \mathbb{B}^k$, nous notons $\phi(b^1, \dots, b^k)$ l'évaluation de la formule ϕ en b .

□ 28 – Écrire une fonction OCaml `logical_map (phi : formula) (w : bool stream array) : bool stream` ainsi spécifiée :

Précondition : La formule ϕ ne dépend que des variables $x_1, \dots, x_k$. Le tableau w est de longueur k . Nous notons $w = \left[(b_n^{(1)})_{n \in \mathbb{N}}, \dots, (b_n^{(k)})_{n \in \mathbb{N}} \right]$ son contenu.

Valeur de retour : Flot des évaluations booléennes $\left(\phi(b_n^{(1)}, \dots, b_n^{(k)}) \right)_{n \in \mathbb{N}}$.

Dans la question 29, deux personnages, Alice et Bob, se sont accordés, lors de leur tout premier tête-à-tête et à l'abri de toute écoute par un tiers, au sujet de k germes et de k connexions de LFSR ainsi qu'au sujet d'une formule de logique $\psi_0(x_1, \dots, x_k)$. Nous notons $(b_n^{(1)})_{n \in \mathbb{N}}, \dots, (b_n^{(k)})_{n \in \mathbb{N}}$ les k flots de données booléens associés aux LFRS. À présent, Alice souhaite communiquer à Bob la suite de bits $c = (c_n)_{n \in \mathbb{N}}$ via un canal public. Elle transmet à Bob la suite des évaluations $(X(\psi_0(b_n^{(1)}, \dots, b_n^{(k)}), c_n))_{n \in \mathbb{N}}$.

□ 29 – Dire si Bob peut toujours parvenir à reconstituer le flot $c = (c_n)_{n \in \mathbb{N}}$ et, le cas échéant, expliquer comment.

4. Énumération d'objets combinatoires

4.1. Flot des mots de Dyck

Soit Σ l'alphabet binaire $\Sigma = \{a, b\}$. Nous nous intéressons à la constante OCaml dyck définie par le code suivant

```
54. let rec dyck =
55.   let f (x,y) = "a" ^ x ^ "b" ^ y in
56.   Cons("", fun () -> map f (product dyck dyck))
```

où la fonction product a été introduite à la question 14. Nous notons $\mathcal{D} \subseteq \Sigma^*$ le langage constitué des mots appartenant au flot de données dyck.

□ 30 – Exhiber une grammaire non contextuelle $\mathcal{G}$ telle que le langage $L(\mathcal{G})$ engendré par la grammaire $\mathcal{G}$ coïncide avec le langage $\mathcal{D}$.

□ 31 – Énoncer une propriété de la grammaire $\mathcal{G}$ de la question 30 qui permet d'établir que le flot de chaînes de caractères dyck n'énumère jamais deux fois le même mot. La démontrer.

4.2. Flot des permutations d'un tableau

Nous partageons une variante d'un algorithme dû à B. R. Heap sous la forme du pseudo-code suivant. Dans ce pseudo-code, les positions de tableau sont numérotées à partir de l'indice 0.

Seconde épreuve d'informatique MPI 2024

Algorithme 1 : Algorithme de B. R. Heap**Entrées** : Entier k , tableau t de longueur $\geq k$.**Effet** : Effectue des échanges en place dans le tableau t et des affichages de t .**Sortie** : Aucune valeur de retour.

```

1 si  $k = 1$  alors
2   | Afficher le tableau  $t$ .
3 sinon
4   | pour  $i = 0$  à  $k - 1$  (inclus) faire
5     | Exécuter  $\text{Heap}((k - 1), t)$ .
6     | si  $k$  est pair alors
7       | Échanger  $t[i]$  et  $t[k - 1]$ 
8     | sinon
9       | Échanger  $t[0]$  et  $t[k - 1]$ 

```

□ 32 – Soient k un entier et t un tableau de longueur supérieure à k . Dénombrer le nombre d'appels $A(k)$ à la fonction « afficher » lorsque l'on exécute $\text{Heap}(k, t)$.

□ 33 – Soient k un entier et t un tableau de longueur supérieure à k . Démontrer, pour tout entier naturel non nul k , la proposition $\mathcal{P}(k)$ suivante :

Après l'exécution de $\text{Heap}(k, t)$, si k est un entier impair, alors le tableau t est inchangé ; en revanche, si k est un entier pair, alors les k premiers coefficients du tableau t ont subi une permutation circulaire vers la droite : autrement dit, le nouvel état du tableau est

$$(t[k - 1] \ t[0] \ t[1] \ \dots \ t[k - 3] \ t[k - 2] \ t[k] \ \dots \ t[n - 1])$$

□ 34 – Soient k un entier et t un tableau de longueur supérieure à k . Démontrer que l'exécution de $\text{Heap}(k, t)$ affiche, pour toute permutation des k premiers alvéoles du tableau t , le tableau dans lequel les k premiers alvéoles ont été permutés et dans lequel les derniers alvéoles ont été laissés inchangés.

□ 35 – Écrire une fonction OCaml `pivot (k:int) (i:int) (t:'a array) : unit` dont l'effet est d'appliquer au tableau t les opérations des lignes 6 à 9 de l'algorithme 1.

□ 36 – Écrire une fonction OCaml `permstream_of_array (t:'a array) : 'a array stream` dont la valeur de retour est un flot de données fini constitué de toutes les permutations du tableau t énumérées dans l'ordre d'affichage de l'algorithme de Heap et qui utilise efficacement la mémoire.

A. Annexe : rappels de programmation

(D'après <https://v2.ocaml.org/api/index.html>)

Opérations générales : Le module Stdlib, chargé par défaut, offre les fonctions suivantes :

- `max : 'a -> 'a -> 'a`
Return the greater of the two arguments. The result is unspecified if one of the arguments contains the float value nan.
- `float_of_int : int -> float`
Convert an integer to floating-point.
- `(lsl) : int -> int -> int`
`n lsl m` shifts `n` to the left by `m` bits.

Opérations sur les tableaux : Le module Array offre les fonctions suivantes :

- `make : int -> 'a -> 'a array`
`make n x` returns a fresh array of length `n`, initialized with `x`.
- `init : int -> (int -> 'a) -> 'a array`
`init n f` returns a fresh array of length `n`, with element number `i` initialized to the result of `f(i)`.
- `exists : ('a -> bool) -> 'a array -> bool`
`exists f [|a1; ...; an|]` checks if at least one element of the array satisfies the predicate. That is, it returns `(f a1) || (f a2) || ... || (f an)`.

Opérations sur les tables de hachage : Le module Hashtbl offre les fonctions suivantes :

- `create : int -> ('a, 'b) t`
`Hashtbl.create n` creates a new, empty hash table, with initial size `n`.
- `add : ('a, 'b) t -> 'a -> 'b -> unit`
`Hashtbl.add t k d` adds a binding of key `k` to data `d` in table `t`.
- `length : ('a, 'b) t -> int`
`Hashtbl.length tt` returns the number of bindings in `t`. It takes constant time. Multiple bindings are counted once each.

Opérations sur les fils d'exécution : Le module Thread offre les fonctions suivantes :

- `create : ('a -> 'b) -> 'a -> t`
`Thread.create f x` creates a new thread of control, in which the function application `f(x)` is executed concurrently with the other threads of the domain. The application of `Thread.create` returns the handle of the newly created thread.
- `delay : float -> unit`
`delay d` suspends the execution of the calling thread for `d` seconds. The other program threads continue to run during this time.

Opérations sur les verrous : Le module Mutex offre les fonctions suivantes :

- `create : unit -> t`
Return a new mutex.
- `lock : t -> unit`
Lock the given mutex. Only one thread can have the mutex locked at any time. A thread that attempts to lock a mutex already locked by another thread will suspend until the other thread unlocks the mutex.
- `unlock : t -> unit`
Unlock the given mutex. Other threads suspended trying to lock the mutex will restart. The mutex must have been previously locked by the thread that calls `Mutex.unlock`.

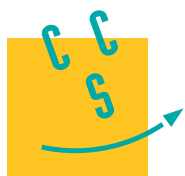
B. Annexe : règles de la déduction naturelle

Dans les tableaux suivants, la lettre Δ désigne un ensemble de formules de logique ; les lettres A , B et C désignent des formules de logique.

Axiome
$\frac{}{\Delta, A \vdash A} \text{ (ax)}$

	Introduction	Élimination
$\top$	$\frac{}{\Delta \vdash \top} \text{ (}\top\text{i)}$	
$\perp$		$\frac{\Delta \vdash \perp}{\Delta \vdash A} \text{ (}\perp\text{e)}$
$\wedge$	$\frac{\Delta \vdash A \quad \Delta \vdash B}{\Delta \vdash A \wedge B} \text{ (}\wedge\text{i)}$	$\frac{\Delta \vdash A \wedge B}{\Delta \vdash A} \text{ (}\wedge\text{e)}$ $\frac{\Delta \vdash A \wedge B}{\Delta \vdash B} \text{ (}\wedge\text{e)}$
$\vee$	$\frac{\Delta \vdash A}{\Delta \vdash A \vee B} \text{ (}\vee\text{i)}$ $\frac{\Delta \vdash B}{\Delta \vdash A \vee B} \text{ (}\vee\text{i)}$	$\frac{\Delta \vdash A \vee B \quad \Delta, A \vdash C \quad \Delta, B \vdash C}{\Delta \vdash C} \text{ (}\vee\text{e)}$
$\neg$	$\frac{\Delta, A \vdash \perp}{\Delta \vdash \neg A} \text{ (}\neg\text{i)}$	$\frac{\Delta \vdash A \quad \Delta \vdash \neg A}{\Delta \vdash \perp} \text{ (}\neg\text{e)}$

FIN DE L'ÉPREUVE



CONCOURS CENTRALE•SUPÉLEC

Informatique

MPI

2024

4 heures

Calculatrice autorisée

*Le jeu de go***I Introduction au jeu de go****I.A – Présentation du sujet**

Ce sujet s'intéresse au jeu de go et à différentes facettes de la construction d'un programme jouant au jeu. La partie 1 présente des règles simplifiées du jeu de go qui seront utilisées tout au long de ce sujet, et qui seront particulièrement utiles aux parties 2 et 5. Plusieurs fonctions permettant de mener une partie de go seront à implémenter en langage **C** dans la partie 2. Les parties suivantes se penchent davantage sur la dimension stratégique du jeu. La partie 3 s'intéresse à l'estimation d'un coup pertinent à partir d'une base de données, en utilisant l'algorithme des **k plus proches voisins**, puis en mettant en place une **table de hachage** contenant des coups localement similaires, en langage **C**. Cette recherche de coup se poursuit en partie 4 à travers une **Recherche Arborescente de Monte-Carlo** utilisant des **fils d'exécutions concurrents**, en **OCaml**. Enfin, la partie 5 conclut ce sujet en montrant que décider si une position est gagnante ou non dans un jeu de go généralisé est un problème **NP-dur**.

I.B – Présentation et règles du jeu de go

Le jeu de *go* est un jeu de stratégie opposant deux adversaires, qui placent à tour de rôle des pierres sur un plateau quadrillé appelé *goban*. Ce plateau est modélisé par une grille de d lignes par d colonnes, avec d un entier appelé la *dimension* du goban. Dans ce sujet nous nous intéresserons principalement à des gobans standards de dimension $d = 19$. Chacune des d^2 intersections du goban est identifiée par sa *position*, qui est un couple (*ligne, colonne*) représentant son emplacement sur le plateau.

Les deux joueurs sont nommés d'après la couleur des pierres qu'ils placent sur le goban : Noir, qui joue en premier et place des pierres noires, et Blanc qui place des pierres blanches. À chaque tour, un joueur place une pierre de sa couleur sur une intersection *libre* du goban, c'est-à-dire sans pierre. Un coup correspond donc simplement à l'ajout d'une pierre sur une intersection libre, les joueurs ne peuvent pas déplacer une pierre déjà posée.

Nous appellerons *configuration* du goban, ou simplement *goban*, l'état du plateau à un moment de la partie, c'est-à-dire la donnée des positions et couleurs de toutes les pierres placées sur le goban. La figure 1 illustre différentes configurations progressives au cours d'une partie possible de go.

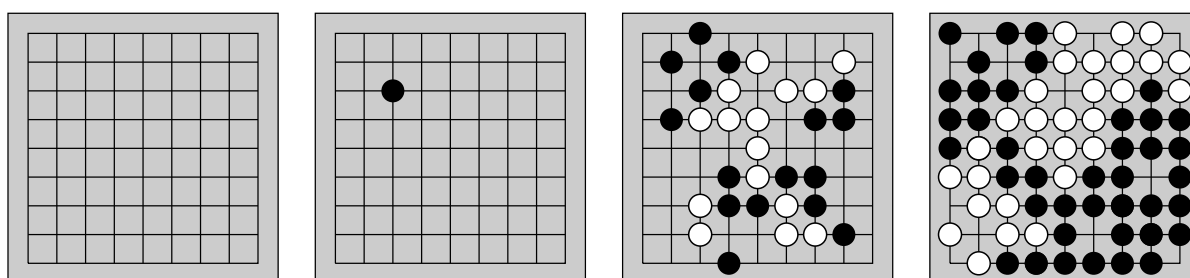


Figure 1 Un goban vide de dimension 9, un premier coup de Noir, une configuration du jeu à un moment intermédiaire, et une configuration à la fin de la partie dans laquelle chaque intersection est contrôlée par un des deux joueurs.

Chaque intersection possède plusieurs *intersections voisines* qui sont les intersections orthogonalement adjacentes à cette dernière (et non diagonalement adjacentes). Une intersection a donc au plus 4 intersections voisines, celles sur les extrémités du goban pouvant en avoir moins. Par extension, les intersections voisines d'une pierre sont celles de l'intersection où elle est placée.

Deux pierres d'une même couleur sont dites *connectées* si elles se trouvent sur des intersections voisines. Nous appellerons *groupe* de pierres une composante connexe pour cette relation de connexion. Chaque groupe possède un nombre de *libertés* qui est défini comme le nombre d'intersections libres qui sont voisines d'une pierre du groupe. Une intersection voisine de plusieurs pierres du groupe n'est comptée qu'une seule fois.

Un groupe de pierres auquel on vient de supprimer la dernière liberté est *capturé*, et les pierres de ce groupe sont alors immédiatement retirées du plateau. Ces notions de libertés et de capture sont illustrées dans la figure 2.

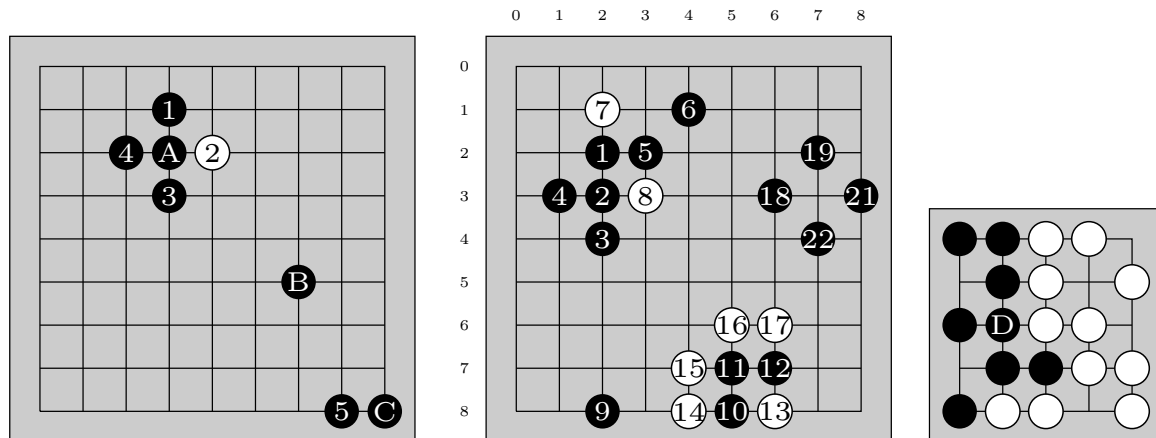


Figure 2 À gauche : la pierre A a une pierre sur chacune de ses 4 *intersections voisines*. La pierre B n'a pas de pierres sur ses 4 intersections voisines. La pierre C a une pierre sur une de ses deux intersections voisines. Au milieu : les pierres 1, 2, 3, 4 et 5 forment un *groupe* de pierres noires, la pierre 6 ne fait pas partie de ce groupe, ni les pierres blanches 7 et 8. Ce groupe a 7 *libertés*. La pierre 9 a 3 libertés. Le groupe de pierre 10, 11, 12 a une seule liberté, si Blanc joue en (7, 7) alors il *capture* immédiatement ce groupe, et les pierres 10, 11, 12 sont retirées du plateau. L'intersection (3, 7) est *contrôlée* par noir. À droite : un exemple de goban de dimension 5 en fin de partie.

Cette règle de capture permet d'aboutir à des configurations avec moins de pierres sur le goban. Afin de s'assurer que la partie termine, une règle supplémentaire interdit de jouer un coup qui fasse revenir à une configuration du goban déjà vue durant la partie.

Lorsqu'une pierre est placée, on commence par regarder si elle permet de capturer des pierres adverses, et dans ce cas on retire immédiatement ces pierres capturées du goban. On regarde ensuite si le groupe de la pierre placée possède des libertés. Si elle ne possède aucune liberté, le coup est interdit. Cependant, si elle a permis de capturer des pierres, son groupe a nécessairement au moins une liberté. Il est donc possible de jouer sur une position sans libertés, mais uniquement dans le cas où cela permet de capturer des pierres à l'adversaire.

Une intersection est dite *contrôlée* par un joueur si une de ses pierres est placée dessus, ou si l'intersection est vide mais que toutes les intersections voisines contiennent des pierres de sa couleur.

La partie s'arrête lorsque toutes les intersections sont contrôlées par un des deux joueurs. Une telle configuration du goban est alors dite *finale*. Le *score* de chaque joueur est défini comme le nombre d'intersections contrôlées par ce dernier. Le joueur avec le score le plus élevé remporte la partie.

I.C – Généralités sur le jeu de go

Q 1. Compter le nombre de libertés du groupe auquel appartient la pierre D dans le goban de la figure 2 Droite. Donner le score de Blanc, celui de Noir et le joueur gagnant de cette partie.

Q 2. Montrer qu'il y a toujours un gagnant dans une configuration finale sur un goban standard de dimension 19.

II Implémentation du jeu de go

L'objectif de cette première partie est d'implémenter diverses fonctions permettant de manipuler le goban pour mener une partie de go. Ces fonctions sont à écrire en langage C. Dans tout le sujet, il sera supposé que les entêtes suivantes ont été incluses :

```
#include <assert.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
```

Afin de représenter et manipuler un goban, nous utiliserons la structure de données suivante pour représenter une configuration.

```
1 struct goban_s {
2     int d;
3     int* m;
4 };
5 typedef struct goban_s goban;
```

Le champ `d` est ici la dimension du goban, et `m` est un pointeur vers un tableau unidimensionnel d'entiers de taille $d \times d$ représentant les pierres placées. Une intersection libre est représentée par l'entier 0, une pierre noire par l'entier 1 et une pierre blanche par l'entier 2. Initialement, le tableau est donc rempli de 0.

Q 3. Écrire une fonction `goban_initialisation(int d)` prenant en argument un entier `d` et qui renvoie un goban de dimension `d` avec un tableau `m` alloué dynamiquement, représentant un goban vide.

La position d'une intersection du goban sera représentée par une structure de couple d'entiers, contenant la ligne i et la colonne j de la position, avec $0 \leq i, j < d$.

```
1 struct couple_int {
2     int i;
3     int j;
4 };
5 typedef struct couple_int position;
```

Une position pourra être initialisée avec la fonction suivante :

```
1 position pos(int i, int j) {
2     position p = {.i = i, .j = j};
3     return p;
4 }
```

Un goban pourra être lu et modifié grâce aux deux fonctions ci-dessous, en supposant que la position correspond à une intersection valide du goban.

```
1 int lire(goban g, position p) {
2     return g.m[p.i * g.d + p.j];
3 }
4
5 void ecrire(goban g, position p, int couleur) {
6     g.m[p.i * g.d + p.j] = couleur;
7 }
```

Les questions suivantes vont permettre d'isoler un groupe de pierres. Pour cela, nous représentons les intersections voisines d'une intersection donnée grâce à la structure suivante :

```
1 struct voisins_s {
2     int nb;
3     position t[4];
4 };
5 typedef struct voisins_s voisins;
```

Le nombre de voisins de l'intersection est donné par le champ `nb`, et `t` est un tableau de positions de taille 4, dont les `nb` premières cases contiennent les positions des intersections voisines de l'intersection. La fonction incomplète suivante permet ce calcul des voisins :

```
1 voisins pos_voisins(goban g, position p) {
2     int i = p.i;
3     int j = p.j;
4     voisins v;
5     ...
6     if (...) {v.t[k] = pos(i - 1, j); k = k + 1;}
7     if (...) {v.t[k] = pos(i + 1, j); k = k + 1;}
8     if (...) {v.t[k] = pos(i, j - 1); k = k + 1;}
9     if (...) {v.t[k] = pos(i, j + 1); k = k + 1;}
10    v.nb = k;
11    return v;
12 }
```

Q 4. Compléter cette fonction `pos_voisins` en donnant l'instruction à écrire en ligne 5 ainsi que les conditions à écrire pour chacune des lignes 6, 7, 8 et 9 pour qu'elle renvoie, dans une structure `voisins` les positions des intersections voisines de la position donnée en entrée.

Q 5. Écrire une fonction `goban groupe(goban g, position p)` qui renvoie un nouveau goban dans lequel seules les pierres du groupe de la pierre en position `p` sont présentes. Expliquer par une phrase, avant le code de votre fonction, l'approche choisie. Il pourra être utile de commencer par implémenter une fonction auxiliaire récursive, dont la spécification sera clairement explicitée. Donner la complexité obtenue.

Une première tentative pour calculer le nombre de libertés du groupe de la pierre en position `p` a abouti sur la fonction suivante :

```

1  int liberte(goban g, position p) {
2      goban g1 = groupe(g, p);
3      int cpt = 0;
4      int couleur = lire(g, p);
5      for (int i = 0; i < g.d; i = i + 1) {
6          for (int j = 0; j < g.d; j = j + 1) {
7              position p2 = pos(i, j);
8              if (lire(g1, p2) == couleur) {
9                  voisins v = pos_voisins(g1, p2);
10                 for (int k = 0; k < v.nb; k = k + 1) {
11                     if (lire(g, v.t[k]) == 0) {
12                         cpt = cpt + 1;
13                     }
14                 }
15             }
16         }
17     }
18     return cpt;
19 }
```

Q 6. Proposer un jeu de quatre tests permettant de tester cette fonction `liberte`. Deux tests seront passés par la fonction, et deux montreront qu'elle n'est pas correcte. Les gobans en entrées pourront être représentés par des schémas. On prendra soin de justifier l'intérêt des tests proposés et d'indiquer les valeurs attendues.

Q 7. Proposer des modifications à apporter au code de la fonction `liberte` pour qu'elle renvoie bien le nombre de libertés, et qu'elle libère les structures créées pendant son appel.

Q 8. Écrire une fonction `void retire_groupe(goban g, position p)` prenant en argument un goban, une position et qui retire toutes les pierres du groupe de la pierre à cette position. Une complexité linéaire en la taille du groupe à retirer est attendue.

Q 9. Écrire une fonction `void joue(goban g, position p, int couleur)` prenant en argument un goban, une position et une couleur et qui modifie le goban en ajoutant une pierre de la couleur donnée à la position donnée. On prendra en compte les possibles pierres à retirer et on utilisera des assertions pour vérifier que le coup est valable. On ne vérifiera cependant pas si la configuration du goban a déjà été rencontrée au cours de la partie. Donner la complexité de cette fonction.

III Prédiction et évaluation d'un prochain coup

Dans cette partie, nous envisageons de réaliser un programme qui renvoie ou évalue un prochain coup pertinent à jouer depuis une configuration donnée du goban, afin de simuler un adversaire. La dimension stratégique du jeu de go est réputée pour être complexe, c'est pourquoi une première approche simple peut être de s'inspirer de coups joués par des joueurs humains professionnels.

Pour cela nous nous baserons sur une base de données de 160 000 parties de maîtres, jouées sur des gobans standards de dimension 19. Nous avons commencé par décomposer ces parties en 29.4 millions de triplets (configuration du goban, joueur dont c'est le tour, coup suivant choisi), que nous appellerons *observations*, et qui peuvent être redondantes.

Q 10. À partir des informations sur la base de données, donner une estimation de la profondeur du jeu de go, c'est-à-dire du nombre de coups moyen d'une partie de go. En utilisant la description du jeu ainsi que la profondeur, donner une estimation de la largeur du jeu de go, c'est-à-dire le nombre moyen de coups valables lors d'une partie.

Est-il envisageable de mettre en place un algorithme *min-max* pour trouver une stratégie gagnante à ce jeu ? Justifier.

III.A – *K plus proches voisins*

Afin de jouer en un temps raisonnable, dans cette sous-partie nous allons implémenter l'algorithme des *k plus proches voisins* pour prédire un prochain coup pertinent de manière supervisée. Pour cela, nous allons confronter la configuration d'entrée sur laquelle nous désirons trouver le prochain coup à toutes les configurations compatibles de la base de données, c'est-à-dire les configurations dont le coup suivant qui a été joué peut effectivement être réalisé. Le calcul d'une distance entre deux configurations du goban ne sera pas étudié ici, nous supposons que nous disposons d'un tableau non trié dans lequel ces n distances ont été préalablement calculées. Dans cette sous-partie, pour simplifier les algorithmes et les programmes nous nous intéresserons seulement à déterminer les k plus petites valeurs de ce tableau de distances non trié, sans se préoccuper de garder en mémoire les configurations auxquelles correspondent ces distances.

Q 11. Proposer une fonction `double* k_plus_petits(double* distances, int n, int k)` qui prend en entrée un tableau de n distances et un entier k et renvoie un tableau de k distances correspondant aux k plus petites valeurs du tableau en entrée. Il est attendu ici une complexité en $O(nk)$ dans le pire cas. La fonction pourra, si besoin, modifier le tableau donné en entrée. Justifier la complexité obtenue et prouver la correction de cet algorithme à l'aide d'un invariant de boucle.

Q 12. Proposer une structure de données abstraite qui permette d'améliorer la complexité asymptotique de la fonction `k_plus_petits`. Rappeler en une phrase une façon d'implémenter cette structure de données ainsi que la complexité obtenue.

Dans le cadre d'un adversaire jouant en temps réel, il est préférable d'avoir une borne précise sur le nombre de comparaisons qui seront réalisées. Nous supposons désormais que n est une puissance de 2 avec $n \geq 2$. Nous allons construire un algorithme permettant de trouver les indices des deux plus petits éléments d'un tableau de n distances. Il sera possible de supposer que les distances de ce tableau sont toutes différentes.

Nous appellerons comparatif un algorithme prenant en entrée un tableau de distances pour lequel la seule opération autorisée sur les distances du tableau est la comparaison entre deux distances du tableau.

Q 13. Montrer que tout algorithme comparatif renvoyant l'indice du plus petit élément d'un tableau devra réaliser la comparaison du plus petit élément et du second plus petit élément du tableau.

Q 14. Proposer un algorithme qui renvoie les indices des deux plus petits éléments d'un tableau de n distances en réalisant au plus $n + \log_2(n) - 2$ comparaisons. On pourra s'inspirer d'une approche de type diviser pour régner et commencer par comparer chaque élément d'indice $2i$ avec l'élément d'indice $2i + 1$, pour $0 \leq i < n/2$. Il est ensuite possible d'adapter cette approche à la sélection des k plus petits éléments.

III.B – *Hachage de Zobrist*

Le but de cette sous-partie est de présenter une autre approche utilisant la base de données des parties de maîtres, cette fois pour apprendre une fonction d'évaluation des prochains coups qui s'intéresse uniquement à l'état local du goban autour d'une position envisagée. Nous chercherons, dans la base de données, si des configurations localement identiques ont mené à un coup à cette position. Cette recherche sera implémentée de manière efficace dans cette partie grâce à un prétraitement de la base de données utilisant des tables de hachage et une fonction de hachage adaptée, nommée *hachage de Zobrist*.

Un état local sera modélisé par différents *zooms* autour d'une position, qui considèrent la disposition du plateau autour d'une intersection en ignorant le reste du goban, appelée *motif*. Nous noterons $Z_i(g, (x_0, y_0))$ le motif obtenu à partir du zoom i sur la position (x_0, y_0) du goban g comme étant les couleurs des pierres aux positions (x, y) telles que $|x - x_0| + |y - y_0| \leq i$, et avec $(x - x_0, y - y_0) \neq (0, 0)$. Par exemple, $Z_1(g, p)$ correspond à la configuration des intersections voisines de l'intersection de position p . Nous dénoterons par $(x - x_0, y - y_0)$ la position du goban (x, y) dans ce motif $Z_i(g, (x_0, y_0))$. Pour avoir toujours le même nombre d'éléments dans les motifs, une couleur supplémentaire 3 sera assignée aux positions qui dépassent les bords du goban. Les intersections libres seront représentées par 0, celles avec une pierre noire par 1 et celles avec une pierre blanche par 2.

Le but d'un zoom sur une position est d'évaluer un coup potentiel à cette position. Étant donné un motif centré sur une intersection libre sur laquelle il est possible de jouer, nous allons chercher dans la base de données si des configurations possédant ce même motif ont mené à un coup à cette position. Nous considérerons toujours que le centre du motif est une intersection vide.

Q 15. Donner une borne supérieure sur le nombre de motifs distincts pour un zoom i . Justifier qu'il est raisonnable de supposer que tout motif de zoom 1 a été rencontré au moins une fois dans la base de données. On pourra faire cette supposition dans le reste du sujet.

Les fonctions de cette partie sont à écrire en langage C, les configurations seront manipulées avec les mêmes structures que celles de la partie 2, et les fonctions qui y sont définies peuvent être librement réutilisées.

Pour savoir combien de fois un motif a conduit à un coup en son centre, nous allons construire et remplir une table de hachage, pour chaque niveau de zoom. La fonction de hachage associée assignera à chaque motif un entier non signé de 64 bits, de type `uint64_t`. Cette fonction de hachage particulière se base sur une table d'entiers `uint64_t` fixés, ayant autant de colonnes que de positions dans le motif, et quatre lignes pour les quatre

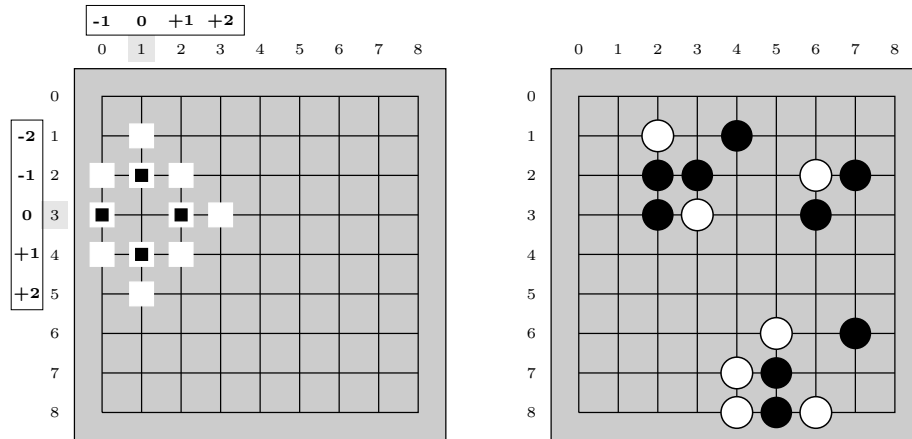


Figure 3 Gauche : l'étendue du zoom 1 (les 4 intersections avec un carré noir), et du zoom 2 (les 11 intersections avec des carrés blancs, et une intersection extérieure à la grille non représentée) sur la position (3,1). La case (2,1) est en position $(-1,0)$ dans les zooms sur la position (3,1).
Droite : un exemple de goban. Les motifs correspondant au zoom 1 en position (2,4) et au zoom 1 en position (3,7) sont identiques, ce motif est ici observé deux fois dans cette configuration. Si Noir choisit de jouer en (2,4) on dira alors que ce motif a été choisi une fois.

couleurs possibles (libre, noir, blanc et hors du goban). Afin de manipuler à la main un nombre raisonnable de bits, sur la table donnée en exemple en figure 4, nous utilisons des entiers codés sur 8 bits et non sur 64 bits. La valeur de hachage est alors calculée comme le *ou exclusif* (XOR) bit à bit, noté $\oplus$, des entiers des cases de toutes les combinaisons position-couleur se trouvant dans le motif. La table de vérité du XOR pour un bit est rappelée en figure 4.

Formellement, en notant A la fonction qui associe à un couple de couleur et de position dans le motif l'entier fixé associé dans la table, nous définissons la fonction de hachage d'un motif h par :

$$h(Z_i(g, (x_0, y_0))) = \bigoplus_{\substack{(x_u, y_u) \in \mathbb{Z}^2 \setminus \{(0,0)\} \\ \text{tels que } |x_u| + |y_u| \leq i}} A(g(x_0 + x_u, y_0 + y_u), (x_u, y_u))$$

a	b	a XOR b
0	0	0
1	0	1
0	1	1
1	1	0

Couleur et position	(0, +1)	(+1, 0)	(0, -1)	(-1, 0)
0 (vide)	00001111	11100111	00011000	01111011
1 (noir)	11110110	11111110	01101001	10011111
2 (blanc)	10100001	10001101	00110111	00110100
3 (au delà du bord)	10100000	01101010	10111011	11000010

Figure 4 Table de vérité du XOR, et une table d'entiers fixés sur 8 bits associés à des couples couleur et position dans le motif. Par exemple une pierre noire en position (+1,0) dans un motif est associée à 11111110.

Q 16. Dans le goban de la figure 3 Droite, et avec le tableau de valeurs sur 8 bits donné dans la figure 4, calculer la valeur de hachage du motif en position (7,6) pour le zoom 1. Expliquer comment il est possible de mettre à jour cette valeur en calculant seulement 2 XOR supplémentaires lorsque l'on joue une pierre blanche en (6,6).

Nous disposons désormais d'une fonction `uint64_t hachage(position p, goban g, int zoom)` qui calcule la valeur de hachage $h(Z_i(g, p))$ étant donnée une position p , un goban g et un zoom $zoom$.

De plus, en connaissant la valeur de hachage h_1 du motif $Z_i(g_1, p)$, nous disposons d'une fonction `uint64_t maj_hachage(position p, goban g1, goban g2, int zoom, uint64_t h1)` qui permet de calculer efficacement, en terme de nombre de XOR effectués, la valeur $h(Z_i(g_2, p))$ en mettant à jour $h(Z_i(g_1, p))$ à partir des différences entre les gobans g_1 et g_2 .

Nous nous intéressons maintenant uniquement aux choix des coups de Noir. Il nous faut dénombrer les motifs observés et les motifs choisis dans la base de données de parties. Pour un niveau de zoom fixé, nous stockerons ces valeurs dans une table de hachage, associant un motif à son nombre d'*observations* et de *choix*.

Pour un niveau de zoom fixé, dans une configuration du goban où c'est au tour de Noir de jouer, nous *observons* un motif par position libre de cette configuration. Noir va alors jouer dans une de ces positions libres, et on dit que le motif associé à la position libre où Noir joue est *choisi* (voir la figure 3).

Pour chaque niveau de zoom, les valeurs stockées dans sa table de hachage associée seront représentées par la structure suivante :

```

1  struct observation_s {
2      int n; //nombre d'observations du motif au total
3      int v; //nombre de fois ou ce motif a ete choisi
4  };
5  typedef struct observation_s observation;
```

Le champ `n` correspond au nombre d'observations du motif dans des gobans où c'était à Noir de jouer. Le champ `v` correspond au nombre de fois où le motif était observé au moment où c'était à Noir de jouer et où le coup de Noir a été de placer une pierre au centre de ce motif.

Nous définissons alors une structure de table de hachage composée d'un tableau d'observations, et d'un entier `nt` correspondant à sa taille.

```

1  struct table_de_hachage_s {
2      int nt;
3      observation* t;
4  };
5  typedef struct table_de_hachage_s t_hachage;
```

La table de hachage est initialisée avec $n = 0$ et $v = 0$, pour chaque case. Pour accéder à la valeur de la table associée à une clé, nous prendrons la clé modulo `nt`. Ce sujet ne s'intéresse pas aux collisions, et nous pourrions supposer qu'il n'y en a aucune.

Pour remplir la table de hachage, nous allons simuler la partie de maître configuration après configuration, en utilisant la fonction `maj_hachage` pour mettre à jour les valeurs de hachages. Au départ, nous partons donc de la configuration initiale du goban, et nous allons garder en mémoire les valeurs de hachage pour chaque position du goban, et les faire évoluer au fur et à mesure de la partie.

Q 17. Écrire une fonction `uint64_t* init_t_hash(int d, int zoom)` qui prend en entrée un entier `d` correspondant à la dimension d'un goban, et un entier `zoom` correspondant au niveau de zoom d'intérêt, et qui renvoie un tableau d'entiers non signés sur 64 bits alloué dynamiquement, de taille $d \times d$ où chaque case correspond à la valeur de hachage initiale sur cette position dans un goban vide.

On considère un goban `gN` où Noir doit jouer, le goban suivant `gB` après le coup de Noir, et le goban après le coup de Blanc, quand Noir doit jouer à nouveau `gNs`. La fonction `maj_coup`, donnée ci-dessous, met à jour dans toutes les positions possibles du goban, si nécessaire, le nombre d'observations des motifs en prenant en compte le goban `gN`, avec la fonction `maj_n`, le nombre de choix des motifs avec la fonction `maj_v`, en s'intéressant au coup choisi par Noir entre `gN` et `gB`. Elle prend notamment en entrée `tableau_hache` qui contient les valeurs de hachage des positions du goban `gN`. La fonction `maj_t_hache`, donnée ci-dessous, met à jour les valeurs de hachage associées aux positions avec la fonction `maj_h`, le tableau en entrée `tableau_hache` contient les valeurs de hachage des positions du goban `gN` et doit être à jour sur le goban `gNs` à l'issue de l'exécution de la fonction.

```

1  void maj_coup(t_hachage table, uint64_t* tableau_hache, goban gN, goban gB, int zoom) {
2      for (int i = 0; i < gN.d; i = i + 1) {
3          for (int j = 0; j < gN.d; j = j + 1) {
4              position p = pos(i, j);
5              maj_n(table, tableau_hache, gN, p);
6              maj_v(table, tableau_hache, gN, gB, p);
7          }
8      }
9  }
10
11 void maj_t_hache(uint64_t* tableau_hache, goban gN, goban gNs, int zoom) {
12     for (int i = 0; i < gN.d; i = i + 1) {
13         for (int j = 0; j < gN.d; j = j + 1) {
14             position p = pos(i, j);
15             maj_h(tableau_hache, gN, gNs, zoom, p);
16         }
17     }
18 }
```

Q 18. Écrire les fonctions

```
void maj_n(t_hachage table, uint64_t* tableau_hache, goban gN, position p),
void maj_v(t_hachage table, uint64_t* tableau_hache, goban gN, goban gB, position p),
void maj_h(uint64_t* tableau_hache, goban gN, goban gNs, int zoom, position p),
```

pour que les fonctions `maj_coup` et `maj_t_hache` mettent correctement à jour la table de hachage et le tableau des valeurs de hachage.

Une partie de go sera représentée par une structure de liste simplement chaînée, chaque coup étant représenté par le goban actuel, le joueur qui doit jouer, et un pointeur vers le reste de la partie. Nous supposons que les parties commencent par le goban vide associé au joueur d'indice 1 (Noir).

```
1 struct partie_s {
2     goban g;
3     int joueur;
4     struct partie_s* suivant;
5 };
6 typedef struct partie_s partie;
```

Q 19. En s'intéressant uniquement aux coups de Noir, écrire une fonction `t_hachage lit_partie(partie* lp, int zoom, t_hachage table)` qui prend en entrée une partie et met à jour pour le zoom donné et dans toutes les positions libres du goban le nombre d'occurrences des motifs, et le nombre de choix des motifs choisis, dans la table de hachage donnée en entrée (qui a déjà été initialisée et mise à jour sur d'autres parties). On pourra supposer que la partie contient au moins 3 coups.

Q 20. Étant donné un tableau des valeurs de hachage initialisé avec le goban initial, montrer qu'au plus $4c$ XOR sont nécessaires pour calculer toutes les valeurs de hachage d'un motif à cette position au cours d'une partie de c coups.

Nous allons représenter toutes les tables de hachages pour les différents niveaux de zoom allant de 1 à $n_z - 1$, inclus, dans une table de tables de hachage, représentée par la structure suivante :

```
1 struct table_table_de_hachage {
2     int nz;
3     t_hachage* t;
4 };
5 typedef struct table_table_de_hachage tt_hachage;
```

Nous construisons cette table de tables de hachages `tt` telle que `tt[i]` contienne la table de hachage associée au zoom i . La première case de ce tableau, `tt[0]`, ne sera ainsi pas utilisé.

Pour évaluer la pertinence d'un coup dans une configuration, nous allons chercher le plus grand zoom pour lequel le motif a été observé au moins une fois dans la base de données. La qualité estimée de ce coup est alors le nombre de fois où ce motif a été choisi sur le nombre de fois où il a été vu.

Q 21. Écrire une fonction booléenne `bool appartient(position p, goban g, t_hachage table, int zoom)` qui renvoie vrai si le motif en position p et avec le zoom `zoom` sur le goban g est présent dans la table de hachage, c'est-à-dire a été observé au moins une fois dans la base de données, et faux sinon.

Q 22. Écrire une fonction `float evaluate_zoom(position p, goban g, t_hachage table, int zoom)` qui renvoie la valeur associée au motif, c'est-à-dire le rapport entre le nombre d'observations et le nombre de choix.

Q 23. Écrire une fonction `float evaluation(position p, goban g, tt_hachage tt)` qui renvoie l'évaluation du coup en position p sur le goban g . Une fonction de complexité logarithmique en le nombre de zooms n_z est attendue. Donner une preuve de la terminaison et de la correction partielle de ce programme, et justifier sa complexité.

IV Recherche arborescente de Monte-Carlo

Dans cette partie nous nous intéressons à la construction d'un programme en OCaml jouant au go, en utilisant l'approche de Recherche Arborescente de Monte-Carlo. C'est notamment sur cette méthode que s'est basé AlphaGo pour vaincre le champion du monde de go en 2017. Nous supposons dans cette section que les parties de go finissent toujours dans une configuration finale, c'est-à-dire complètement contrôlée, sans égalité et que le goban est de dimension standard $d = 19$.

Les fonctions de cette partie sont à écrire en langage OCaml. Une annexe rappelant des éléments de langage OCaml, modules Thread et Mutex, module List, type enregistrement et fonction de conversion de type, est proposée en fin de partie.

Commençons par décrire quatre types pour représenter les joueurs, les gobans et les positions :

```

1  type joueur = Noir | Blanc
2  type intersection = Libre | Pierre_noire | Pierre_blanche
3  type goban = intersection array array
4  type position = int * int

```

Nous supposons que les fonctions suivantes ont été déjà implémentées et peuvent être librement utilisées.

- `liste_coup_prior : goban -> joueur -> (position * float) list` : prend un goban et un joueur et renvoie une liste, pour toutes les positions jouables dans le goban pour le joueur, de couples (`position`, `prior`) où `prior` est un flottant entre 0 et 1 correspondant à une estimation de la probabilité de victoire de Noir lorsque le joueur joue en position `position`.
- `strategie_default : goban -> joueur -> position` : prend un goban et un joueur et renvoie une position où jouer, en suivant une stratégie par défaut définie *a priori*, et qui peut contenir une part d'aléatoire. Cette fonction renvoie l'exception `Pas_de_coup_valide` lorsque le plateau est complètement contrôlé et qu'il n'y a plus de coup valide.
- `gagnant : goban -> joueur` : prend un goban complètement contrôlé et renvoie le joueur gagnant.
- `joue : goban -> position -> joueur -> goban` : prend un goban, une position et un joueur et renvoie le goban obtenu après que le joueur a joué en cette position. Le goban en entrée n'est pas modifié.

Q 24. Écrire une fonction de signature `js : joueur -> joueur` qui prend un joueur et renvoie le joueur suivant.

IV.A – Simulation de partie pour évaluer une configuration

Pour trouver un prochain coup pertinent, il est important de pouvoir évaluer si une configuration du goban est prometteuse ou non, ce qui est une tâche complexe. L'évaluation de Monte-Carlo permet d'estimer cette évaluation de goban, en jouant une partie depuis la configuration actuelle jusqu'au bout, et en choisissant les coups successifs grâce à une stratégie définie *a priori*. Celle-ci peut-être complètement aléatoire, ou basée sur des données d'apprentissage par exemple.

Q 25. Écrire une fonction de signature `sim_default : goban -> joueur -> joueur` qui prend un goban et un joueur, qui utilise la fonction `strategie_default` pour choisir des coups jusqu'à atteindre une configuration finale, et qui renvoie alors le joueur gagnant.

IV.B – Valeur d'action

Dans la recherche arborescente de Monte-Carlo, nous explorons l'arbre de toutes les parties possibles depuis le goban actuel, mais sans stocker explicitement cet arbre tout entier. De multiples explorations sont réalisées partant chacune de la racine de l'arbre et descendant jusqu'à une configuration finale, sans représentation explicite de l'arbre, en suivant des stratégies définies *a priori*. L'idée principale de la recherche arborescente de Monte-Carlo est d'ajouter à cette approche sans mémoire la construction d'un arbre partiel de jeu, dont les nœuds vont correspondre aux configurations souvent explorées, et pour lesquelles il sera possible d'enregistrer différentes informations :

- `n` : le nombre de visites du nœud, entier mutable
- `v` : le nombre de visites du nœud ayant mené à une victoire de Noir, entier mutable
- `prior` : la probabilité *a priori* de victoire de Noir donnée par la fonction `liste_coups_prior`, flottant
- `pos` : la position du coup associé au nœud, position
- `t` : un verrou, de type `t.Mutex`, qui sera utilisé pour mettre en place des parcours et mises à jour concurrentes de l'arbre.

Afin de stocker ces données, les nœuds de l'arbre seront étiquetés par le type enregistrement `etiquette`.

```

1  type etiquette =
2      {mutable n:int; mutable v:int; prior:float; pos:position; t:Mutex.t}

```

Enfin, nous définissons le type suivant pour l'arbre partiel :

```

1  type arbre = N of etiquette * arbre list

```

La première étape de la recherche arborescente de Monte-Carlo est une descente qui consiste à suivre un chemin dans l'arbre partiel qui semble prometteur jusqu'à atteindre une feuille. Pour un nœud a de l'arbre, nous notons n_a , v_a , et $prior_a$ les nombres de visites, de victoires et la prior, c'est-à-dire l'estimation à priori de victoire de Noir, du nœud a . Dans un nœud de l'arbre partiel n'étant pas une feuille, pour savoir quel enfant choisir nous définissons la quantité V_a^N appelée *valeur d'action* du nœud a pour Noir :

$$V_a^N = \begin{cases} \frac{v_a}{n_a} + \frac{\text{prior}_a}{1+n_a} & \text{si } n_a > 0 \\ \text{prior}_a & \text{sinon} \end{cases}$$

Cette valeur est un compromis entre la prior apprise par apprentissage et le taux de victoires obtenu lors des descentes dans l'arbre, l'importance de la prior décroissant avec l'augmentation du nombre de descentes dans l'arbre passant par le nœud. Une valeur d'action analogue peut être définie pour Blanc. En partant de la racine de l'arbre partiel, la feuille à explorer est atteinte en descendant le long d'une branche, en choisissant itérativement comme nœud suivant le fils de valeur d'action maximale pour le joueur qui doit jouer.

Q 26. Écrire une fonction de signature `etiq : arbre -> etiquette` qui prend un arbre et renvoie l'étiquette de cette arbre.

Q 27. Écrire une fonction de signature `valeurActionNoir : arbre -> float` qui prend un arbre et renvoie la valeur d'action pour Noir de cet arbre basée sur son étiquette. Les champs mutables de l'étiquette étant modifiables par d'autres fils d'exécution concurrents, une attention particulière sera donc accordée à l'utilisation du verrou du nœud lors des accès à ces champs.

Q 28. Donner la valeur à maximiser au tour de Blanc en suivant le même principe que pour la valeur d'action de Noir. Nous supposons par la suite que la fonction `valeurActionBlanc : arbre -> float` a été également implémentée.

Q 29. Écrire une fonction de signature `popargmax : 'a list -> ('a -> 'b) -> 'a * 'a list` qui prend en entrée une liste d'éléments de type 'a, 1, et une fonction f qui associe à un type 'a un type 'b tel que les éléments de type 'b soient comparables avec les opérateurs polymorphes de comparaison de OCaml (comme >), et qui renvoie le premier élément de 1 maximum pour f ainsi qu'une liste des autres éléments de 1. Informellement, nous sortons l'argument maximum de 1 pour f pour renvoyer cet argument maximum ainsi que la liste privée de ce dernier.

IV.C – Descente et remontée dans l'arbre

Pour affiner les évaluations des configurations stockées dans l'arbre partiel, la recherche arborescente de Monte-Carlo procède en trois étapes clés :

- une descente dans l'arbre partiel en choisissant à chaque nœud le fils de valeur d'action maximale pour le joueur associé, afin de sélectionner une feuille qui semble prometteuse ;
- une simulation de partie depuis cette feuille comme implémentée en partie IV.A, donnant un joueur vainqueur ;
- une remontée qui met à jour le nombre d'observations et de victoires pour Noir dans chaque nœud rencontré, le long de la branche reliant la feuille explorée à la racine de l'arbre.

Ces étapes pourront être exécutées en parallèles par plusieurs fils d'exécution. Il est donc nécessaire de faire attention à ce que deux fils ne rentrent pas en conflit d'écriture et de lecture pour un champ mutable d'une étiquette de l'arbre, ce qui justifie l'intérêt des verrous associés à chaque nœud.

De plus, pour décourager deux fils d'exécution de suivre la même branche lors de la descente, lorsque l'enfant d'un nœud est sélectionné par un fil d'exécution, ce fil va modifier l'étiquette de cet enfant pour lui ajouter des *défaites virtuelles*, ce qui baissera sa valeur d'action et donc son attractivité du point de vue des autres fils. Nous ajouterons 5 défaites virtuelles à un nœud s'il est sélectionné, en prenant soin d'enlever ces défaites virtuelles lors de la remontée.

Lors de la descente, nous empêcherons ainsi un fil d'exécution de commencer à traiter un nœud, (c'est-à-dire de chercher l'enfant de ce nœud de valeur d'action maximale) si un autre fil d'exécution est déjà en train de traiter ce nœud. Nous attendrons que le fil d'exécution en cours ait ajouté des défaites virtuelles à la branche qu'il a choisi de suivre avant de commencer le traitement.

Q 30. Écrire la fonction `descente_remontee_arbre : goban -> joueur -> arbre -> joueur` qui prend un goban, le joueur dont c'est le tour et un arbre de jeu, et qui réalise une descente, une simulation et une remontée de l'arbre. Cette fonction renverra le joueur gagnant, et lors de la remontée mettra à jour les champs mutables des nœuds rencontrés. Une attention particulière sera portée sur le fait que cette fonction puisse être exécutée par plusieurs fils d'exécution concurrents.

IV.D – Expansion de l'arbre

L'idée de la recherche arborescente de Monte-Carlo est d'alterner entre des étapes de descente, simulation et remontée de l'arbre pour améliorer les évaluations des nœuds, et des étapes d'expansion, où nous ajoutons de nouveaux nœuds à l'arbre partiel de jeu.

Q 31. Écrire une fonction de signature `expansion_feuille : goban -> joueur -> arbre list` qui prend un goban et un joueur, et qui renvoie la liste des enfants du nœud correspondant à ce goban et ce joueur. Il est attendu d'utiliser la fonction `liste_coup_prior` et d'initialiser les étiquettes des enfants avec 0 observation, 0 victoire et un nouveau verrou `Mutex.t`.

Une étape d'expansion consiste à étendre une feuille, qui sera sélectionnée en partant de la racine et en choisissant à chaque nœud le fils ayant été le plus visité.

Q 32. Écrire une fonction de signature `expansion : goban -> joueur -> arbre -> arbre` qui réalise une étape complète d'expansion, c'est-à-dire part de l'arbre et descend jusqu'à une feuille en choisissant à chaque étape le fils le plus visité, étend la feuille rencontrée en ajoutant tous ses fils, puis qui renvoie l'arbre partiel de jeu étendu.

IV.E – Algorithme complet

Pour pouvoir commencer à réaliser l'étape d'expansion, il est nécessaire d'attendre que tous les fils d'exécution réalisant des descentes et remontées aient terminé.

Q 33. Écrire une fonction de signature `attendre : Thread.t list -> unit` permettant d'attendre que tous les fils d'exécutions de la liste donnée en argument aient terminé leur exécution.

Q 34. Écrire une fonction de signature `etape_complete : int -> goban -> joueur -> arbre -> arbre` qui réalise une étape complète de l'algorithme, telle que `etape_complete nt g j a` lance `nt` fils d'exécution qui réalisent chacun une descente, simulation puis remontée de l'arbre `a` où la racine correspond au goban `g` et au joueur `j`, et une fois que tous les fils ont terminé, réalise une étape d'expansion avant de renvoyer l'arbre obtenu après cette expansion.

Q 35. Montrer que cette fonction termine. Il est notamment attendu de montrer qu'il n'y a pas d'interblocages dans les exécutions de la fonction `descente_remontee_arbre` en se référant au code proposé en question 30.

Pour estimer le coup à jouer, il reste alors à réaliser plusieurs étapes complètes, par exemple un nombre `ns` fixé à l'avance. Puis, à la fin de ces étapes, l'algorithme renvoie la position du coup correspondant au nœud le plus visité parmi les fils de la racine.

Le sous-arbre associé à ce coup sera ensuite utilisé comme base pour la suite de la partie, ce qui permet de ne pas repartir de zéro à chaque coup.

Q 36. Écrire une fonction de signature `recherche_arborescente : int -> int -> goban -> joueur -> arbre -> arbre * position` telle que `recherche_arborescente ns nt g j a` effectue `ns` étapes de simulations sur l'arbre `a` en partant du goban `g` avec le joueur `j`, chaque étape de simulations utilisant `nt` fils d'exécutions, avant de renvoyer le coup choisi, ainsi que l'arbre à conserver pour le coup suivant.

Éléments du langage OCaml

Module Thread

- `Thread.t` : Le type des fils d'exécution.
- `Thread.create : ('a -> 'b) -> 'a -> Thread.t` : `Thread.create funct arg` crée un nouveau fil d'exécution dans lequel l'application de `funct arg` est exécuté en même temps que les autres fils d'exécutions. L'application de `Thread.create` renvoie le fil d'exécution créée. Le nouveau fil termine quand l'application `funct arg` termine, soit normalement soit en remontant une exception `Thread.Exit` ou en remontant d'autres exceptions non rattrapées. Dans ce dernier cas, l'exception non rattrapée est affichée sur la sortie d'erreur standard, mais pas propagée au fil d'exécution parent. De la même manière le résultat de l'application `funct arg` est perdu et n'est pas directement accessible au fil d'exécution parent.
- `Thread.join : t -> unit` : `join th` suspend l'exécution du fil qui appelle cette fonction jusqu'à ce que le fil d'exécution `th` ait terminé.

Module Mutex

- `Mutex.t` : Le type des mutex.
- `Mutex.create : unit -> Mutex.t` : Renvoie un nouveau mutex.
- `Mutex.lock : Mutex.t -> unit` : Verrouille le mutex donné. Un seul fil d'exécution peut verrouiller le mutex à un moment donné. Un fil d'exécution tentant de verrouiller un mutex déjà verrouillé attendra jusqu'à ce que l'autre fil d'exécution déverrouille le mutex.
- `Mutex.unlock : Mutex.t -> unit` : Déverrouille le mutex donné. Les autres fils d'exécution en pause tentant de verrouiller le mutex reprennent. Le mutex doit avoir été verrouillé par le fil d'exécution qui appelle `Mutex.unlock`.

Module List

- `List.iter : ('a -> unit) -> 'a list -> unit` : `iter f [a1; ...; an]` applique la fonction `f` tour à tour à `[a1; ...; an]`. Cet application est équivalente à `f a1; f a2; ...; f an`.
- `List.map : ('a -> 'b) -> 'a list -> 'b list` : `map f [a1; ...; an]` applique la fonction `f` à `a1, ..., an`, et construit la liste `[f a1; ...; f an]` avec les résultats renvoyés par `f`.

Type enregistrement

- `type exemple = {mutable a : int; b int}` : permet de définir un type enregistrement.
- `let c = {a = 1; b = 3}` permet de définir une variable du type `exemple`.
- `c.a` permet d'accéder au champ `a` de `c`.
- `c.a <- 2` permet de modifier le champ mutable `a` du type enregistrement `c`.

Fonction de conversion de type

- `float_of_int : int -> float` : convertit un `int` en `float`.
- `int_of_float : float -> int` : convertit un `float` en `int`.

V Complexité du problème de la stratégie gagnante au go

Un problème classique qui survient lorsque l'on s'intéresse à des jeux est de se demander quel joueur possède une stratégie gagnante à partir d'une configuration donnée. Ce problème de décision sera étudié ici sous le nom de go généralisé.

Go généralisé

Entrée : Une configuration d'une partie de go généralisé, c'est-à-dire un goban de dimension d , avec N un entier arbitraire, sur lequel sont placées des pierres, et la donnée du joueur qui doit réaliser le prochain coup.

Sortie : Oui s'il existe une stratégie gagnante pour Blanc à partir de cette configuration, non sinon.

Il est possible de montrer que ce problème est NP-dur à partir d'une chaîne de réductions depuis le problème SAT, dont la dernière étape sera montrée dans cette partie.

Q 37. Rappeler la définition de la classe NP, et expliquer ce que signifie être NP-dur et NP-complet. Donner un exemple de problème NP-complet classique autre que SAT et 3SAT, sans prouver son appartenance à cette classe de complexité.

Dans cette partie nous considérons une variante du problème 3SAT, nommée *Rectilinear Planar 3SAT*, qui est également NP-dur. Tout comme pour 3SAT, les instances de ce problème de décision sont des formules sous forme normale conjonctive avec exactement trois littéraux par clauses, mais désormais chaque clause ne possède que des littéraux positifs, ou que des littéraux négatifs. De plus, la formule en question doit pouvoir se représenter graphiquement de manière planaire rectiligne comme illustré en figure 5, avec toutes les variables alignées horizontalement et les clauses reliées verticalement à leurs variables associées sans aucune intersection. Une telle représentation graphique est également donnée en entrée du problème avec la formule, et n'a donc pas besoin d'être trouvée.

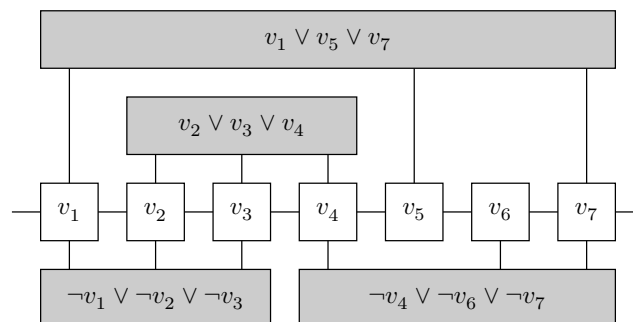


Figure 5 La formule $\varphi = (v_1 \vee v_5 \vee v_7) \wedge (v_2 \vee v_3 \vee v_4) \wedge (\neg v_1 \vee \neg v_2 \vee \neg v_3) \wedge (\neg v_4 \vee \neg v_6 \vee \neg v_7)$ représentée graphiquement sous forme planaire rectiligne. Il est à noter que sur une telle représentation graphique, certaines clauses peuvent en enjamber d'autres. Ici, la clause $(v_1 \vee v_5 \vee v_7)$ enjambe la clause $(v_2 \vee v_3 \vee v_4)$.

Soit φ et sa représentation graphique planaire rectiligne une instance de Rectilinear Planar 3SAT. Nous allons construire une configuration du jeu de go généralisé telle que Blanc possède une stratégie gagnante si et seulement si φ est satisfiable. Pour cela, nous supposons qu'une grande quantité de pierres blanches se trouvent presque encerclées par des pierres noires, de sorte que l'issue de la partie soit uniquement déterminée par le fait que Blanc parvienne à relier ces pierres blanches à une structure permettant de sécuriser son groupe (figure 6), (d)), (auquel cas il sécurise un nombre de points suffisamment grand pour lui garantir la victoire), ou pas (auquel cas Noir encercle et capture toutes ces pierres blanches, et gagne). En s'affrontant pour cette capture décisive, les deux joueurs vont placer des pierres dans des *gadgets* reliés entre eux et à la réserve de pierres blanches par des tuyaux (figure 6, (b) et (c)).

Q 38. Expliquer pourquoi si Blanc parvient à relier sa réserve à un gadget (d) de la figure 6, alors Noir ne peut plus capturer les pierres blanches de la réserve.

Ce gadget à atteindre pour sécuriser le groupe de pierres sera nommé des *yeux*. En tentant à tout prix de relier sa réserve à des yeux, Blanc va progressivement relier sa réserve à des gadgets, qui sont des dispositions

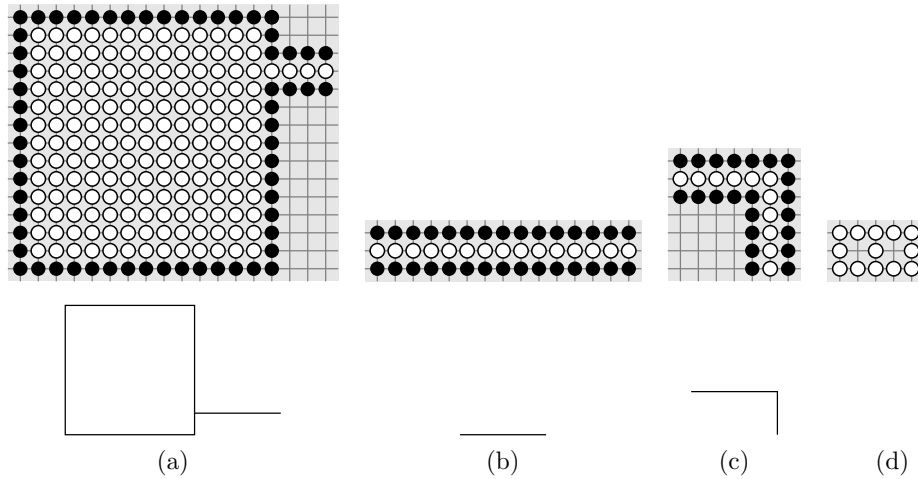


Figure 6 Un exemple de réserve de pierres blanches presque capturée par le joueur noir (a), ainsi que deux sections de tuyaux (b) et (c). Des représentations simplifiées sont données en bas. En (d) une configuration de 13 pierres blanches que Blanc va chercher à relier à sa réserve de pierres blanches pour les sécuriser.

locales de pierres qui vont permettre de simuler un comportement souhaité. Ces gadgets sont construits de sorte que le nombre de libertés du groupe de pierres blanches qui risquent d'être capturées soit toujours de 1, ou exceptionnellement de 2 dans le gadget (figure 7 (a)) ce qui permet à Blanc de faire un choix ponctuellement. Ce premier gadget (figure 7 (a)) permet de modéliser un choix pour Blanc, qui décidera s'il veut relier le tuyau du haut (supposé relié à la réserve de pierres) au tuyau de gauche ou au tuyau de droite. Nous pourrions supposer que c'est à Blanc de poser la première pierre dans chaque gadget.

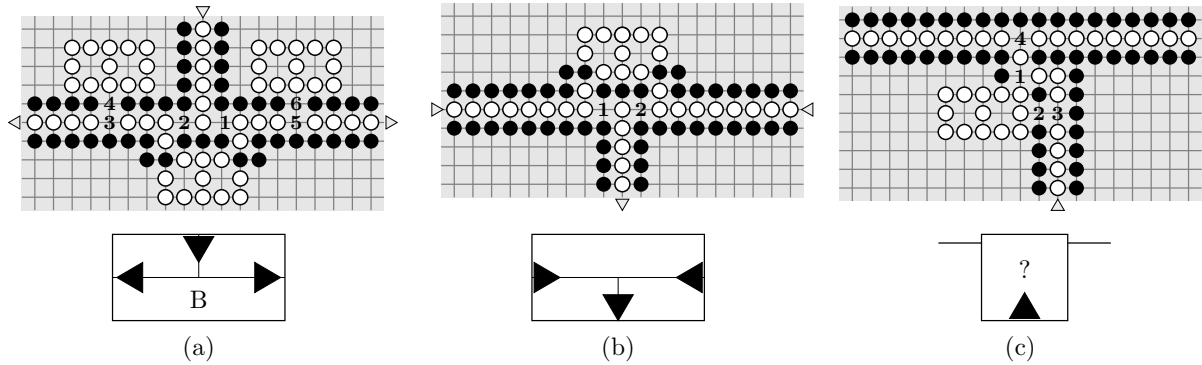


Figure 7 Trois gadgets qui seront utilisés pour la réduction, ainsi que leurs représentations simplifiées associées.

Q 39. Expliciter précisément les deux suites de coups possibles pour le gadget (a), et justifier à l'aide d'un arbre que si Blanc ou Noir s'écarte de l'une de ces deux stratégies, alors l'autre joueur peut arriver à ses fins en au plus deux coups.

Q 40. Décrire des modifications à apporter au gadget (a) pour obtenir un gadget représentant un branchement mais dans lequel c'est Noir qui décide si le tuyau de gauche ou de droite sera relié au tuyau du haut. On rappelle qu'on se place toujours dans le cas où Blanc joue en premier dans les gadgets.

Nous utiliserons, pour ce nouveau gadget, la même représentation simplifiée que le gadget (a) en remplaçant le B par un N.

Q 41. Que permet de réaliser le gadget (b) ? Justifier.

Q 42. Justifier que le gadget (c) permet, en arrivant par le bas, de tester si le tuyau transversal du haut a déjà été relié à la réserve de pierres blanches par le passé ou pas. On pourra notamment justifier que la partie se décidera sur ce gadget précis, et l'issue en sera uniquement déterminée par le fait que le tuyau transversal du haut ait été préalablement relié à la réserve ou non.

Nous nous intéressons désormais à l'assemblage de ces gadgets pour former un circuit, qui pourra être représenté en utilisant les notations simplifiées. Dans un premier temps, nous allons simuler la valuation de chaque variable par le fait d'avoir relié une section à la réserve ou non.

Q 43. Vérifier que pour chaque variable v_i , il existe une stratégie pour Noir qui permette qu'au plus un des deux tuyaux v_i ou $\neg v_i$ soit relié à la réserve de pierres blanches.

Q 44. Vérifier qu'il existe, pour toute valuation des variables, une stratégie permettant à Blanc de relier à la réserve de pierres blanches les sections associées à cette valuation.

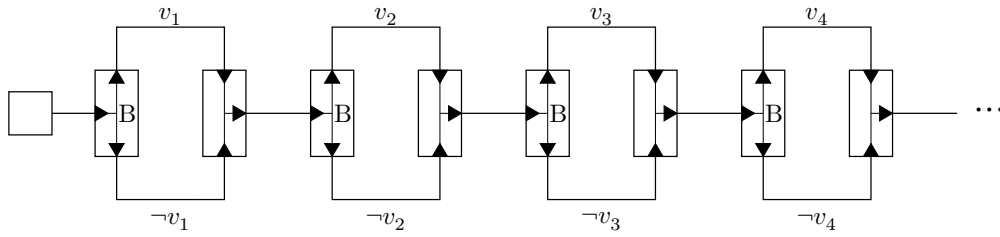


Figure 8 Un circuit représentant le choix d'une valuation, modélisée par le fait de choisir de relier le tuyau du haut ou le tuyau du bas à la réserve pour chaque variable.

Ainsi, Blanc a choisi de relier certaines sections à sa réserve de pierres blanches et la partie est encore en cours. Il reste désormais à déterminer si la valuation choisie par Blanc est un modèle de φ ou non. Pour cela, l'idée est de laisser Noir choisir la clause C qu'il souhaite dans φ . Ensuite, Blanc choisit d'accéder au littéral de son choix au sein de cette clause, afin de témoigner qu'elle est bien satisfaite. Le gadget suivant (figure 9) est une première tentative pour modéliser ce choix de Blanc, mais sa construction bloque le passage et empêche Noir d'accéder à d'éventuelles autres clauses enjambées, qui pourraient être représentées à l'intérieur dans la représentation planaire rectiligne de φ .

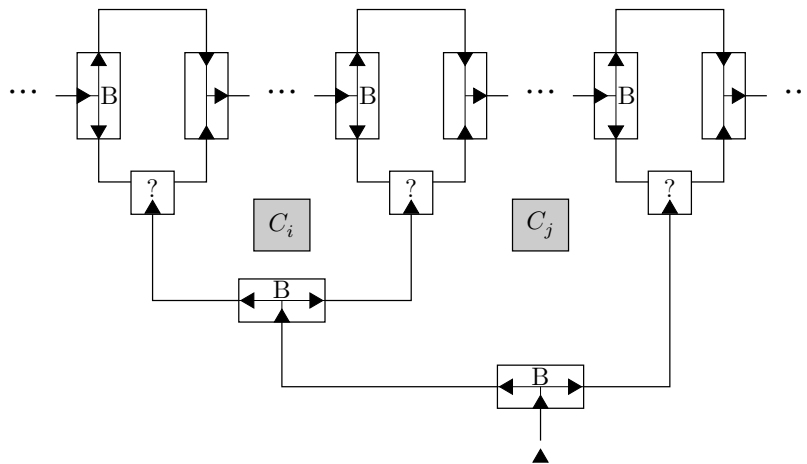


Figure 9 Un gadget permettant à Blanc de vérifier que le littéral de son choix est bien satisfait dans une clause, mais qui bloque le passage à d'autres clauses éventuelles comme C_i ou C_j qui peuvent être enjambées.

Q 45. Modifier le circuit de vérification de clause proposé, afin que Noir puisse en plus décider d'accéder à une clause intérieure s'il le souhaite, sans en modifier le comportement.

Q 46. En utilisant les questions précédentes, conclure en proposant une réduction complète de Rectilinear Planar 3SAT au jeu de go généralisé.

• • • FIN • • •

SESSION 2024



MPI5IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MPI**

INFORMATIQUE**Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, bleu clair ou turquoise, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
 - Ne pas utiliser de correcteur.
 - Écrire le mot *FIN* à la fin de votre composition.
-

Les calculatrices sont interdites.

Le sujet est composé de trois parties, toutes indépendantes.

Partie I - Algorithme Single Pass

Cette partie comporte des questions nécessitant un code en **langage C**.

On cherche à classer $n = 5$ documents textuels $doc_i, i \in \llbracket 1, n \rrbracket$ dans lesquels les termes T_1, T_2 et T_3 apparaissent un certain nombre de fois, ces occurrences étant décrites dans le tableau suivant :

	doc_1	doc_2	doc_3	doc_4	doc_5
T_1	1	2	0	1	1
T_2	3	1	1	3	0
T_3	3	0	0	1	1

Chaque document doc_i est donc représenté par un ensemble de $p = 3$ valeurs. On notera $d_i, i \in \llbracket 1, n \rrbracket$ un vecteur de $\mathbb{N}^3$, de composantes $d_{ij}, j \in \llbracket 1, p \rrbracket$, d_{ij} indiquant le nombre d'occurrences du terme T_j dans le document doc_i .

On cherche à voir si des textes traitent des mêmes thématiques, en faisant l'hypothèse que des textes sont sémantiquement proches si des termes communs apparaissent.

Q1. Recopier et remplir le tableau suivant en appliquant l'algorithme des k-moyennes avec $k = 2$ et d_2 et d_5 comme centres de classe initiaux. On utilisera la distance $\delta(d_i, d_j) = \sum_{\ell=1}^p |d_{i\ell} - d_{j\ell}|$. On notera de plus c_i les centres de classe et $\mathcal{A}_i$ les classes correspondantes.

Itération	c_1	c_2	$\mathcal{A}_1$	$\mathcal{A}_2$
1	d_2	d_5		
2				
3				

On souhaite maintenant traiter ce problème par une méthode dite "single pass" décrite dans l'**algorithme 1**.

Algorithme 1 - Algorithme Single Pass

Entrées : $(d_1 \dots d_n)$ les vecteurs des documents, θ un seuil appartient à $\mathbb{R}$.

Sorties : $(\mathcal{A}_1 \dots \mathcal{A}_j)$ les classes de centres $(c_1 \dots c_j)$

début

```

     $c_1 = d_1$  ; // Initialisation
     $\mathcal{A}_1 = \{d_1\}$  ;
     $j = 1$  ;
    pour  $i$  de 2 à  $n$  faire
        pour  $k$  de 1 à  $j$  faire
            Étape (i) Calculer  $\delta(d_i, c_k)$  ;
        si Étape (ii)  $\delta(d_i, c_k) > \theta \forall c_k$  alors
             $j = j + 1$  ; // Création d'une nouvelle classe
             $\mathcal{A}_j = \{d_i\}$  ;
             $c_j = d_i$  ;
        sinon
            // Indice du centre de classe le plus proche de  $d_i$  au sens de  $\delta$ 
            Étape (iii)  $\ell = \arg \min_{1 \leq k < i} (\delta(d_i, c_k))$  ;
             $\mathcal{A}_\ell = \mathcal{A}_\ell \cup \{d_i\}$  ; // Affectation de  $d_i$  à la classe  $\ell$ 
             $c_\ell = \frac{1}{|\mathcal{A}_\ell|} \sum_{d_i \in \mathcal{A}_\ell} d_i$  ; // Recalcul de  $c_\ell$ 

```

Q2. Appliquer l'**algorithme 1** avec $\theta = 5.0$. Détailler les résultats des étapes de l'algorithme.

On propose d'implémenter cet algorithme en langage C. À cet effet, on définit un type structuré vecteur permettant d'encoder les centres de classe et les textes.

```
struct vecteur_s {
    double *v;           // pointeur vers les coordonnées
    int taille;          // taille du vecteur
    int num_classe;      // classe du vecteur
};
typedef struct vecteur_s vecteur;
```

Puisque le nombre de centres de classe varie au cours de l'algorithme, on utilise une liste chaînée de vecteurs pour représenter l'ensemble des centres de classe.

```
struct noeud_s {
    vecteur *c;
    struct noeud_s *suivant;
};
typedef struct noeud_s noeud;
```

Q3. Écrire une fonction de prototype `void ajoutVecteur(vecteur *vec, noeud **tete)` permettant d'ajouter un vecteur `vec` en tête de la liste des centres de classe pointée par `tete`. On prendra soin de vérifier que l'allocation mémoire s'est bien passée.

On suppose dans la suite disposer :

- de la variable `vecteur *documents[nb_documents]` qui contient l'ensemble des documents, où pour tout $i \in \llbracket 0, \text{nb_documents} - 1 \rrbracket$, `documents[i]` est égal à d_{i+1} .
 - d'une fonction `void recalculCentre (vecteur *documents, noeud **tete, int l)` qui effectue le recalcul du centre c_l et met à jour le nœud correspondant dans la liste chaînée des centres de classe.
- Q4.** Écrire une fonction de prototype `double delta(vecteur *di, vecteur *c)` qui calcule la distance entre le document d_i et le centre de classe c (étape **(i)** de l'**algorithme 1**). On suppose que d_i et c ont la même taille.
- Q5.** Écrire une fonction de prototype `bool distmax(double dists[], int j, double theta)` qui réalise l'étape **(ii)** de l'**algorithme 1**. Le tableau `dists` contient les distances de d_i à tous les c_k : pour tout $k \in \llbracket 0, j - 1 \rrbracket$ l'élément `dists[k]` du tableau `dists` contient la distance de d_i à c_{k+1} . La fonction renvoie `true` si $\delta(d_i, c_k) > \theta \forall c_k$ et `false` sinon.
- Q6.** Écrire une fonction de prototype `int distmin(double dists[], int j)` qui réalise l'étape **(iii)** de l'**algorithme 1**. Le tableau `dists` contient les distances de d_i à tous les c_k comme dans la question précédente. La fonction renvoie l'entier l décrit dans l'algorithme.
- Q7.** À l'aide des questions précédentes et de la fonction `recalculCentre`, proposer une implémentation de l'**algorithme 1** sous la forme d'une fonction de prototype `noeud *algorithme1(vecteur *documents, int nb_documents, double theta)`. Évaluer la complexité de l'**algorithme 1**.

Partie II - Langages réguliers

Soit $\Sigma = \{a, b, c\}$ un alphabet. Σ^* est l'ensemble des mots de longueur finie sur l'alphabet Σ . Pour $u \in \Sigma^*$, $|u| \in \mathbb{N}$ désigne la longueur du mot u . On note $u_i, i \in \llbracket 1, |u| \rrbracket$ la i -ème lettre de u . Pour $k \in \llbracket 1, |u| \rrbracket$, on note $u[1, k]$ le préfixe de u de longueur k , c'est-à-dire le mot $u_1 \dots u_k$.

On définit sur Σ^* la relation symétrique $\mathcal{R}$ de la manière suivante : $u \mathcal{R} v$ s'il existe $w, z \in \Sigma^*$ tels que $u = wabz$ et $v = wbaz$. On note $\mathcal{R}^*$ la fermeture réflexive transitive de $\mathcal{R}$: $u \mathcal{R}^* v$ si $u = v$ ou s'il existe des mots $x_0 \dots x_n$ tels que :

- (i). $x_0 = u$,
- (ii). pour $i \in \llbracket 0, n-1 \rrbracket, i < \min(|u|, |v|), x_i \mathcal{R} x_{i+1}$,
- (iii). $x_n = v$.

Q8. Montrer que $\mathcal{R}^*$ est une relation d'équivalence.

Soit $u \in \Sigma^*$. La classe de u modulo $\mathcal{R}^*$ est l'ensemble des mots v vérifiant $v \mathcal{R}^* u$. Tous les mots de cette classe ont la même longueur. Ainsi, par exemple, $\{abcabb, abcbab, abcbba, bacabb, bacbab, bacbba\}$ est une classe modulo $\mathcal{R}^*$.

Q9. Montrer que les classes modulo $\mathcal{R}^*$ sont des langages réguliers.

Q10. Donner les classes de mots de longueur 3.

On définit la relation $\leq$ sur Σ telle que $a < b < c$ par : pour $u, v \in \Sigma^*$, $u \leq v$ si l'une des deux conditions suivantes est réalisée :

- (i). $u = v[1, |u|]$,
- (ii). il existe $i > 1$ tel que $u[1, i-1] = v[1, i-1]$ et $u_i < v_i$.

Q11. Montrer que la relation $\leq$ est réflexive et transitive. En déduire que $\leq$ est un ordre total sur Σ^* .

Soit $\mathcal{U}$ une classe modulo $\mathcal{R}^*$. Le *représentant* de $\mathcal{U}$ est le plus petit élément de cette classe pour l'ordre $\leq$. On note $\mathcal{L}$ l'ensemble des représentants des classes modulo $\mathcal{R}^*$.

Q12. Donner le représentant de la classe contenant le mot *bacbab*.

Q13. Donner une expression régulière du langage régulier $\mathcal{L}$.

Q14. Proposer un automate fini déterministe complet reconnaissant $\mathcal{L}$.

Partie III - Correspondance de Burge

La correspondance de Burge permet d'exprimer une bijection entre des graphes simples et des tableaux de Young semi-standards. Ces objets combinatoires ont de nombreuses applications, notamment dans l'étude de groupes symétriques et la géométrie algébrique. En théorie des graphes, ils permettent également de trouver, s'ils existent, les graphes simples dont les sommets sont contraints à avoir des degrés donnés.

Cette partie comporte des questions nécessitant un **code OCaml**. Pour ces questions, **les réponses ne feront pas appel aux fonctionnalités impératives du langage** (références, champs mutables, exceptions).

Notations

Dans toute la suite on notera :

- $[l] = \llbracket 1, l \rrbracket$ l'ensemble des entiers naturels de 1 à l ,
- $|E|$ le cardinal de l'ensemble E ,
- $G = ([l], A)$ un *graphe simple*, c'est-à-dire un graphe non orienté à l sommets et m arêtes, sans boucles ni arêtes multiples,
- $d_i = |\{j \in [l], (i, j) \in A\}|$ le degré du sommet $i \in [l]$ dans le graphe $G = ([l], A)$.

De plus, les sommets de G seront ordonnés par degrés décroissants, de sorte que $d_1 \geq d_2 \geq \dots \geq d_l \geq 0$.

Définition 1 (Suite de degrés d'un graphe)

Soit $G = ([l], A)$ un graphe simple. Si $d_i, i \in [l]$ est le degré du sommet i , avec $d_1 \geq d_2 \geq \dots \geq d_l \geq 0$, la *suite de degrés* de G est le l -uplet $(d_1, d_2, \dots, d_l)$.

III.1 - Partitions d'un entier

Définition 2 (Partition)

Une *partition* d'un entier $n \in \mathbb{N}^*$, notée $\lambda \vdash n$, est une suite décroissante $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_l)$ de nombres entiers strictement positifs de somme n .

Par exemple, $n = 4$ a cinq partitions : (4) , $(3,1)$, $(2,2)$, $(2,1,1)$ et $(1,1,1,1)$.

Une partition $\lambda = (\lambda_1 \dots \lambda_l) \vdash n$ peut être vue comme une suite de degrés sous certaines conditions.

Q15. Montrer que si $\sum_{i=1}^l \lambda_i$ est impaire, alors la partition $\lambda \vdash n$ ne peut pas générer de graphe simple ayant la suite de degrés $(\lambda_1, \lambda_2, \dots, \lambda_l)$.

Dans la suite, on considérera que la somme des termes de la suite d'entiers $(\lambda_1, \lambda_2, \dots, \lambda_l)$ est paire. Même avec cette contrainte, cette suite peut ne pas pouvoir générer de graphe simple.

Définition 3 (Suite graphique)

Une suite d'entiers $d_1 \geq d_2 \geq \dots \geq d_l \geq 0$ est dite *graphique* s'il existe un graphe simple dont la suite des degrés est $(d_1, d_2, \dots, d_l)$.

Notons qu'une suite graphique vérifie par définition $d_1 \geq d_2 \geq \dots \geq d_l$.

Pour déterminer si une suite d'entiers donnée est graphique, on peut utiliser l'algorithme d'Havel-Hakimi, basé sur le théorème suivant :

Théorème 1 (Havel-Hakimi)

- (i). Pour tout $(d_1, \dots, d_l) \in \mathbb{N}^l$, si $(d_1, \dots, d_l)$ est graphique, alors $d_{d_1+1} > 0$ et toute permutation décroissante de $(d_2 - 1, d_3 - 1, \dots, d_{d_1+1} - 1, d_{d_1+2}, \dots, d_l)$ est graphique.
- (ii). Réciproquement, pour tout $(d_2, \dots, d_l) \in \mathbb{N}^{l-1}$, si $(d_2, \dots, d_l)$ est graphique, alors pour $d_1 \in \llbracket d_2 + 1, l - 1 \rrbracket$, la suite $(d_1, d_2 + 1, \dots, d_{d_1+1} + 1, d_{d_1+2}, \dots, d_l)$ est graphique.

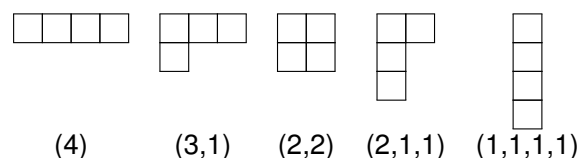
- Q16.** Montrer que si $(d_2 \cdots d_l)$ est graphique, alors il existe un graphe simple $G = (S, A)$ à l sommets, de sommet 1 de degré $d_1 \in \llbracket d_2 + 1, l - 1 \rrbracket$ tel que $(d_1, d_2 + 1 \cdots d_{d_1+1} + 1, d_{d_1+2}, d_{d_1+3}, \cdots d_l)$ soit la suite des degrés des sommets de G .
- Q17.** On suppose que $(d_1, d_2 \cdots, d_l)$ est graphique. Montrer qu'il existe $G = (S, A)$, $S = [l]$, le degré du sommet i étant d_i pour tout $i \in [l]$, tel que : $\forall j \in \llbracket 2, d_1 + 1 \rrbracket$, $(1, j) \in A$. Pour cela, on pourra raisonner par l'absurde et supposer que le sommet 1 est adjacent à un maximum de sommets dans $(2 \cdots d_1 + 1)$, mais pas à tous.
- Q18.** En déduire que si $d_{d_1+1} > d_{d_1+2}$, alors $(d_2 - 1, d_3 - 1 \cdots d_{d_1+1} - 1, d_{d_1+2}, d_{d_1+3} \cdots d_l)$ est graphique.
- Q19.** Déterminer si les suites $(4, 3, 3, 3, 3)$ et $(6, 4, 4, 2, 2, 1, 1)$ sont graphiques.
- Q20.** Écrire une fonction de signature `compare_entiers : int -> int -> int` qui compare deux entiers et telle que l'appel à `compare_entiers m n` renvoie 1 si $m < n$, -1 si $m > n$ et 0 sinon.
- Q21.** Écrire une fonction de signature `decr_n : n -> int list -> int list option` telle que pour tout $n \geq 0$ l'appel `decr_n n l` retourne `Some l'`, avec `l'` la liste `l` dont les n premiers éléments sont décrémentés, s'ils étaient tous supérieurs à 1 et `None` sinon.
- Q22.** Écrire une fonction récursive de signature `havel_hakimi : int list -> bool` qui retourne `true` si la liste passée en entrée, triée par ordre décroissant, est graphique et `false` sinon. À chaque étape utilisant le **théorème 1**, il sera nécessaire de réordonner par ordre décroissant la liste `list` construite, ce qui pourra être fait à l'aide de l'appel à `List.sort compare_entiers list`.
- Q23.** Donner deux graphes simples à $l = 5$ sommets ayant une même suite de degrés $(3, 2, 2, 2, 1)$. Ainsi, il n'existe pas de correspondance univoque entre suite de degrés et graphe simple.

III.2 - Diagramme et tableau de Young

Définition 4 (Diagramme de Young d'une partition)

Le diagramme de Young de forme λ , noté $Y(\lambda)$, d'une partition $\lambda = (\lambda_1, \lambda_2, \cdots, \lambda_l) \vdash n$ est un tableau de cases constitué de l lignes alignées à gauche, chaque ligne $i \in [l]$ ayant λ_i cases. Par convention, la ligne associée à λ_1 est la première ligne du tableau.

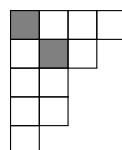
Par exemple, les diagrammes de Young des partitions de l'entier 4 sont



Définition 5 (Diagonale d'un diagramme de Young)

Soit $Y(\lambda)$ un diagramme de Young. La diagonale de $Y(\lambda)$ est définie par les r cases (i, i) , $i \in [r]$.

Dans le tableau suivant, $r = 2$ et les cases de la diagonale sont grisées.



Définition 6 (Partition conjuguée)

Soit $Y(\lambda)$ un diagramme de Young associé à $\lambda \vdash n$. La partition conjuguée de λ , notée $\lambda^* = (\lambda_1^*, \cdots, \lambda_k^*)$, est la partition obtenue en énumérant le nombre de cases de $Y(\lambda)$ par colonne, en partant de la colonne de gauche.

On remarque immédiatement que pour tout j , $\lambda_j^* = |\{i, \lambda_i \geq j\}|$.

Q24. Soit $\lambda = (5, 4, 1, 1, 1, 1) \vdash 13$. Donner λ^* .

Définition 7 (Représentation de Frobenius d'un diagramme de Young)

Soient $(\lambda_1, \lambda_2, \dots, \lambda_l) \vdash n$ une partition, $Y(\lambda)$ son diagramme de Young et r la taille de sa diagonale. Pour $i \in [r]$, soient $\alpha_i = \lambda_i - i$ le nombre de cases à droite de la case (i, i) dans la i -ème ligne de $Y(\lambda)$ et $\beta_i = \lambda_i^* - i$ le nombre de cases en-dessous de la case (i, i) dans la i -ème colonne de $Y(\lambda)$. Alors $\alpha_1 > \alpha_2 \geq \dots \alpha_r \geq 0$, $\beta_1 > \beta_2 \geq \dots \beta_r \geq 0$ et la notation de Frobenius de λ est donnée par $\lambda = \begin{pmatrix} \alpha_1 & \alpha_2 & \dots & \alpha_r \\ \beta_1 & \beta_2 & \dots & \beta_r \end{pmatrix}$.

Q25. Soit $(4, 3, 2, 2, 1) \vdash 12$. Donner la représentation de Frobenius de $Y(\lambda)$.

Définition 8 (Tableau de Young)

Soit $\lambda \vdash n$ une partition de $n > 0$. Un λ -tableau de Young est un tableau obtenu en remplissant les cases d'un diagramme $Y(\lambda)$ par des entiers dans $[n]$.

Par exemple, pour $\lambda = (2, 1) \vdash 3$, les tableaux suivants sont des λ -tableaux

1	2	2	1	1	3	3	1	2	3	3	2	1	1	1	3
3		3		2		2		1		1		2		1	

Dans la suite, on notera $T(i, j)$ l'entier en position (i, j) dans le tableau T .

Définition 9 (Tableau de Young (semi)standard)

Un tableau de Young est dit *semi standard* si les éléments de chaque ligne (respectivement colonne) forment une suite croissante de gauche à droite (respectivement strictement croissante de haut en bas). Il est dit *standard* s'il est semi-standard et si les entiers de 1 à n n'apparaissent qu'une et une seule fois.

Dans les exemples précédents, les premier, troisième et septième tableaux sont semi standards. Les premier et troisième tableaux sont standards.

III.3 - Insertion de Schensted

Pour construire un tableau de Young semi-standard à partir d'une suite d'entiers $d_1, \dots, d_l$, on utilise une méthode d'insertion proposée par Schensted.

On cherche à insérer dans un tableau de Young semi standard T un entier k , de sorte à ce que le tableau créé (contenant une case de plus) soit toujours un tableau de Young semi standard. On note cette opération d'insertion $T \leftarrow k$.

L'entier k est inséré dans T en le comparant aux valeurs de la première ligne et en déplaçant le premier entier plus grand que k en lisant la ligne de gauche à droite. S'il n'y a pas de plus grand élément, k est ajouté à la fin de la ligne. Si un élément est déplacé, il est inséré dans la deuxième ligne et les lignes suivantes du tableau sont traitées de la même manière. L'**algorithme 2** décrit l'insertion de k dans la i -ème ligne de T .

1	1	2	2	4
2	3	3		
3				
4				

Soit T_0 le tableau de Young semi standard

Q26. Insérer la valeur 5 dans T_0 et donner le tableau de Young semi standard résultant. Même question pour l'insertion de la valeur 1 dans T_0 .

Q27. Énoncer une précondition de l'**algorithme 2** permettant de prouver sa correction, c'est-à-dire qu'il retourne bien un tableau de Young semi standard contenant k .

Pour coder les tableaux de Young, on choisit d'utiliser un type OCaml type `tableau = int list list` et on encode la structure par liste de lignes.

Algorithme 2 - Algorithme d'insertion d'un entier k dans la ligne i d'un tableau de Young semi standard

Insertion(T, k, i)

Entrées : un tableau de Young semi standard T , un entier k à insérer, i la ligne traitée

Sorties : un tableau de Young semi standard contenant k

début

si (la ligne i est vide) OU (k plus grand que l'élément le plus à droite de la ligne i) **alors**

 Ajouter k en bout de la ligne i de T

sinon

 Soit j le plus petit indice tel que $k < T(i, j)$

$p = T(i, j)$

$T(i, j) = k$

Insertion($T, p, i + 1$)

Q28. Écrire une fonction récursive de signature `insereligne : int -> int list -> int*int list` qui, à partir d'un entier k et d'une ligne i d'un tableau de Young semi standard, retourne un couple (m, r) où

- $m = 0$ et r est la ligne i où k a été ajouté en queue, si k est plus grand que tous les éléments de la ligne i ,
- m , qui est dans la ligne i et r est la ligne i où m a été remplacé par k sinon.

Q29. En déduire une fonction récursive de signature `insertion : int -> tableau -> tableau` réalisant l'**algorithme 2**.

Q30. Écrire une fonction récursive de signature `construit_tableau : int list -> tableau` qui construit un tableau semi standard à partir d'une suite d'entiers.

L'**algorithme 2** permet de définir une position (s, t) , coordonnées de la case où k a été ajouté à T .

III.4 - Correspondance de Burge

Définition 10 (Tableau de Burge)

Soit $G = ([I], A)$ un graphe simple, $|A| = m$. Le *tableau de Burge* associé à G est défini par

$$\mathcal{B}_G = \begin{pmatrix} u_1 & u_2 & \cdots & u_m \\ v_1 & v_2 & \cdots & v_m \end{pmatrix}$$

où pour tout $i \in [m]$ (u_i, v_i) est une arête de G , avec $u_i > v_i$, le tableau étant formé de sorte que pour $i \in [m - 1]$, $u_i \leq u_{i+1}$ et si $u_i = u_{i+1}$ alors $v_i > v_{i+1}$.

Soit le graphe G de la **figure 1**.

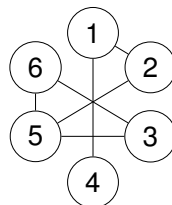


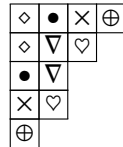
Figure 1 - Graphe exemple

Q31. Donner le tableau $\mathcal{B}_G$ correspondant au graphe de la **figure 1**.

On utilise alors $\mathcal{B}_G$ pour associer à G un diagramme de Young $Y(\lambda)$ représenté par une forme de Frobenius particulière, notée $\mathcal{F}_Y = \begin{pmatrix} \alpha_1 & \alpha_2 & \cdots & \alpha_r \\ \alpha_1 + 1 & \alpha_2 + 1 & \cdots & \alpha_r + 1 \end{pmatrix}$, où r est le nombre de cases de la diagonale principale du diagramme de Young $Y(\lambda)$ associé.

Q32. Montrer que, dans ce cas, pour tout $i \in [r]$ $\lambda_i^* = \lambda_i + 1$.

Ainsi, $Y(\lambda)$ est divisé en deux parties symétriques. La partie inférieure est constituée de toutes les cases qui se trouvent strictement sous la diagonale et la partie supérieure est constituée du reste. Chaque position dans la partie supérieure (inférieure) du diagramme de Young correspond à une position unique, appelée *position opposée*, dans la partie inférieure (supérieure) de $Y(\lambda)$. Par exemple, dans le diagramme de Young suivant, ayant comme représentation de Frobenius $\mathcal{F}_Y$, les positions opposées sont codées par le même symbole.

$$\mathcal{F}_Y = \begin{pmatrix} 3 & 1 \\ 4 & 2 \end{pmatrix}$$


Q33. Soit (s, t) une case. Donner la position opposée en fonction de s et t .

L'**algorithme 3** utilise alors la représentation $\mathcal{B}_G$ d'un graphe G pour construire un tableau de Young semi standard dont le diagramme correspondant $Y(\lambda)$ admet une représentation de Frobenius du type $\mathcal{F}_Y$. Dans cet algorithme, $T \leftarrow v_i$ insère la valeur v_i dans le tableau de Young semi standard T .

Algorithme 3 - Algorithme de Burge

Entrées : $\mathcal{B}_G$ de taille $m \times 2$

Sorties : tableau de Young semi standard T dont la représentation de Frobenius du diagramme correspondant est du type $\mathcal{F}_Y$

début

```

    T = tableau vide ;                                     // Initialisation
    pour i de 1 à m faire
        (s, t) = T ← v_i
        Placer u_i dans la position opposée à (s, t)
```

Cet algorithme produit, à partir du graphe de la **figure 1**, le tableau :

1	1	2	3
2	3	5	
4	5		
5	6		
6			

Q34. Donner les tableaux de Young semi standard intermédiaires produits à chaque itération.

Q35. À quoi correspond le nombre d'apparitions de chaque entier contenu dans les cases du tableau de Young résultat de l'**algorithme 3** ?

Notons pour terminer qu'il est à l'inverse possible de produire un tableau bidimensionnel $\mathcal{B}_G$ (et donc un graphe G) à partir d'un tableau de Young semi standard $Y(\lambda)$ de type $\mathcal{F}_Y$.

FIN